ALINX

ZYNQ 开发平台 HLS 教程

版本号：V1.01

2021-01-25 14:57:20

版权声明

Copyright ©2012-2018 芯驿电子科技（上海）有限公司

公司网址:
Http://www.alinx.com.cn

技术论坛:
http://www.heijin.org

官方旗舰店:
http://alinx.jd.com

邮箱:
avic@alinx.com.cn

电话:
021-67676997

传真:
021-37737073

ALINX 微信公众号:



前言

vitis HLS 能提高系统设计的抽象层次，为设计人员带来切实的帮助。vitis HLS 通过下面两种方法提高抽象层次：第一，使用 C/C++作为编程语言，充分利用该语言中提供的高级结构。第二，提供更多数据原语，便于设计人员使用基础硬件构建块（位向量、队列等）。与使用 RTL 相比，这两大特性有助于设计人员使用 vitis HLS 更轻松地解决常见的协议系统设计难题。最终简化系统汇编，简化 FIFO 和存储器访问，实现控制流程的抽象。

vitis HLS 便于架构研究，用户只需在代码中插入程序指令（如使用 GUI 或批处理模式时的 Tcl 命令），就可以把设计所需特性传递给综合工具。这样用户可以在不修改设计代码本身的情况下研究大量备选架构方案。研究的范围可以是模块流水线化等根本性问题，也可以是 FIFO 队列深度等较常见的问题。

vitis HLS 便于仿真。C 和 RTL 仿真是 vitis HLS 另一个大放异彩的地方。设计一般采用两步流程验证：第一步是 C 语言仿真。这个步骤中 C/C++的编译和执行与常见的 C/C++程序相同；第二步是 C/RTL 协仿真。在这步骤中，vitis HLS 会根据 C/C++测试平台自动生成 RTL 测试平台，然后设置并执行 RTL 仿真，检查实现方案吧的正确性。

如能充分发挥这些优势，这将对于用户的系统设计大有裨益。这不仅体现在开发时间和生产力上，还由于 vitis HLS 代码更加紧凑的特点，体现在代码可维护性和可读性上。此外通过高层次综合，用户仍能有效控制架构及其特性。正确理解和使用 vitis HLS 程序对实现这一控制起着根本作用。

目录

版权声明.....2
前言.....3
目录.....4
准备工作及注意事项.....7
软件环境.....7
硬件环境.....7
实验工程及目录说明.....10
实验快速复现.....10
第一章 初识 Vitis HLS.....12
1.1 Vitis HLS 基本操作.....12
1.1.1 创建 vitis HLS 工程（以第二章工程为例）.....12
1.1.2 Vitis HLS 生成 IP 核.....15
1.1.3 Vitis HLS 仿真.....18
1.2 Vivado 生成 CPU 寄存器.....23
1.3 HLS 接口综合.....29
1.3 HLS 常用优化.....31
1.4 HLS 包含库.....38
1.5 HLS 官方资料.....39
第二章 入门.....40
2.1 实验介绍.....40
2.2 实验源码.....40
2.3 接口设置和优化.....41
2.4 工程路径.....41
2.5 工程实现.....41
第三章 初识 HLS 与 CPU 寄存器交互.....43
3.1 实验介绍.....43
3.2 实验源码.....44
3.3 接口设置和优化.....44
3.4 工程路径.....44
3.5 工程实现.....45
第四章 如何使用 xfopencv 库中自带的函数.....52
3.1 实验介绍.....52
3.2 实验源码.....52
3.3 接口综合与工程优化.....53

3.4 工程路径 53
3.5 工程实现 53
第五章 图像 RGB 转灰度变换 59
4.1 实验介绍 59
4.2 实验源码 59
4.3 接口设置和工程优化 60
4.4 工程路径 61
4.5 工程实现 61
第六章 图像 RGB 转 YCrCb 62
5.1 实验介绍： 62
5.2 实验源码 62
5.3 接口综合和工程优化 64
5.4 工程路径 65
5.5 工程实现 65
第七章 图像形态学滤波 67
6.1 实验介绍 67
6.2 实验源码 67
6.3 接口综合与工程优化 69
6.4 工程路径 70
6.5 工程实现 70
第八章 图像叠加 71
7.1 实验介绍 71
7.2 实验源码 71
7.3 接口综合和工程优化： 73
7.4 工程路径 73
7.5 工程实现 74
第九章 图像对比度调整 75
8.1 实验介绍 75
8.2 实验源码 75
8.3 接口设置和工程优化： 76
8.4 工程路径 78
8.5 工程实现 78
第十章 角点检测 79
9.1 实验介绍 79
9.2 实验源码 80
9.3 接口设置与工程优化 84
9.4 工程路径 84
9.5 工程实现 85

第十一章 SOBEL 算子实现边缘检测.....86
10.1 实验介绍.....86
10.2 实验源码.....86
源文件顶层函数部分.....88
10.3 接口综合与工程优化.....88
10.4 工程路径.....89
10.5 工程实现.....89
第十二章 Canny 算子实现边缘检测.....91
11.1 实验介绍.....91
11.2 实验源码：.....91
11.3 接口综合和工程优化.....93
11.4 工程路径.....94
11.5 工程实现.....94
第十三章 如何在 vitis HLS 实现 opencv 的 c 仿真.....96
13.1 opencv 仿真环境搭建.....96
13.2 vitis hls 配置.....100
13.3 备注.....105

准备工作及注意事项

本教程基于学习者已经熟练使用 Vivado 和 vitis 的情况下编写，建议在阅读本教程之前先熟悉 course_s2_vitis 部分的教程，不详细讲解 Vivado 工程建立，vitis 裸机软件编写，中断处理等。HLS 要求学习者熟练使用 C++，具备一定硬件思维，对硬件流水线、组合逻辑、时序逻辑、FIFO、RAM、AXI 总线协议有深刻理解才能学习，免得出现低级错误。

软件环境

本教程基于软件版本 VITIS HLS，这里要用到的快捷方式图标有：



vivado 软件

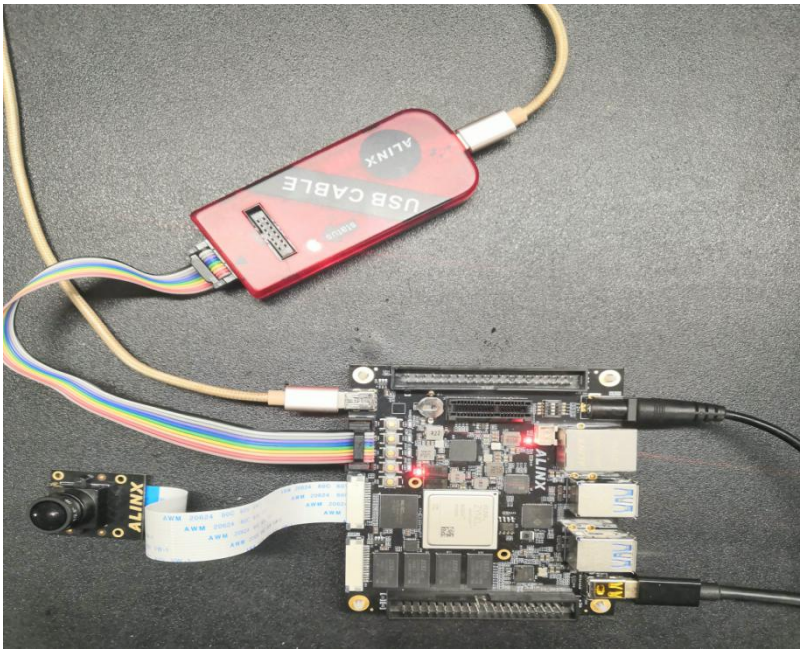


vitis hls 软件

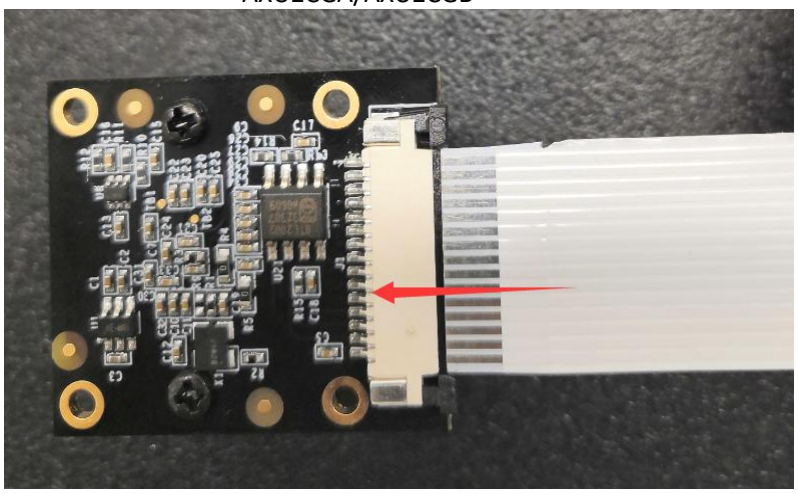
硬件环境

ZYNQ（ALINX 芯驿电子科技）开发板一块，mipi 摄像头模块一个。当前支持的型号列表如下：

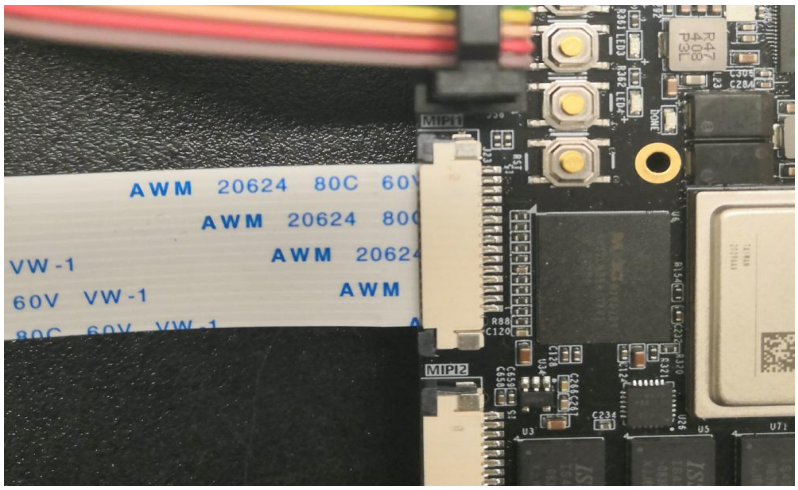
开发板型号	摄像头模块	ZYNQ 芯片型号
AXU2CGA/AXU2CGB	AN5641	Xczu2cgsfvc784-1-i
AXU3EG	AN5641	Xczu3egsfvc784-1-i
AXU4EV-E/AXU4EV-P	AN5641	Xczu4evsfvc784-1-i
AXU5EV-E/AXU5EV-P	AN5641	Xczu5evsfvc784-1-i
AXU7EV	AN5641	Xczu7evffvc1156-1-i
AXU9EG	AN5641	Xczu9egffvb1156-1-i
AXU15EG	AN5641	Xczu15egffvb1156-1-i



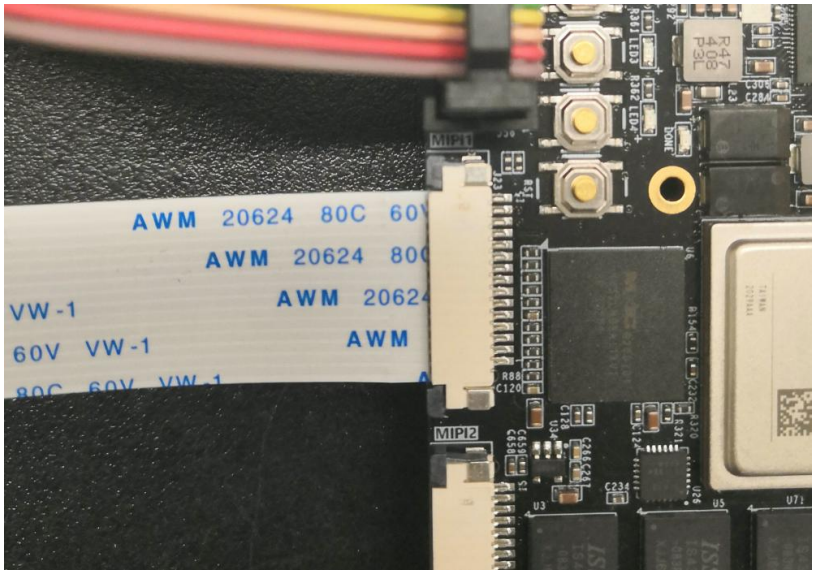
AXU2CGA/AXU2CGB



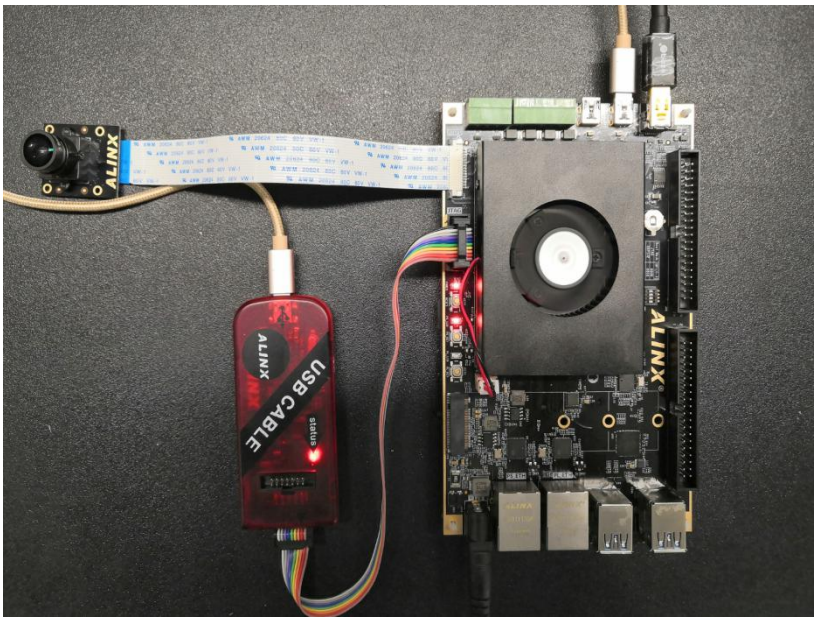
摄像头连接方式



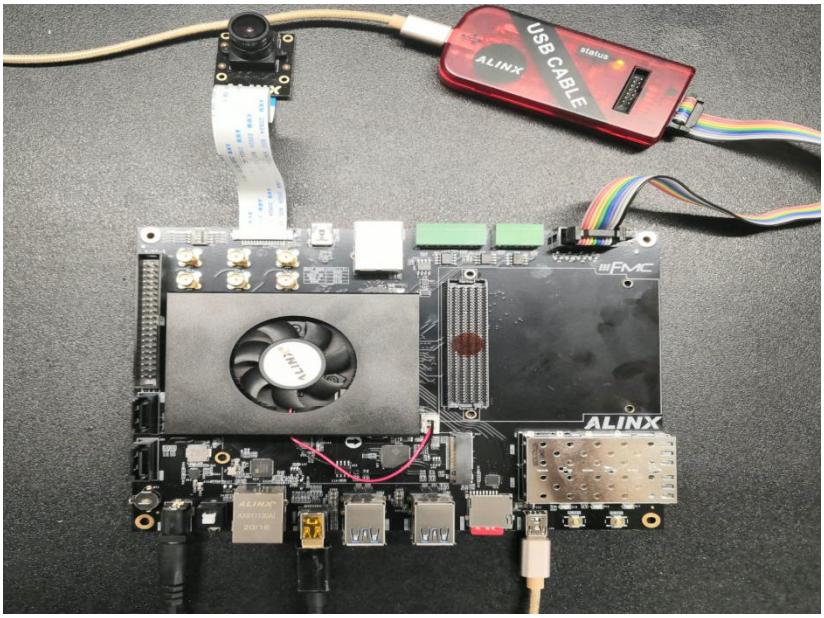
板卡排线连接



板卡连接后效果



AXU3EG/AXU4EV/AXU5EG



AXU7EV/AXU9EG/AXU15EG

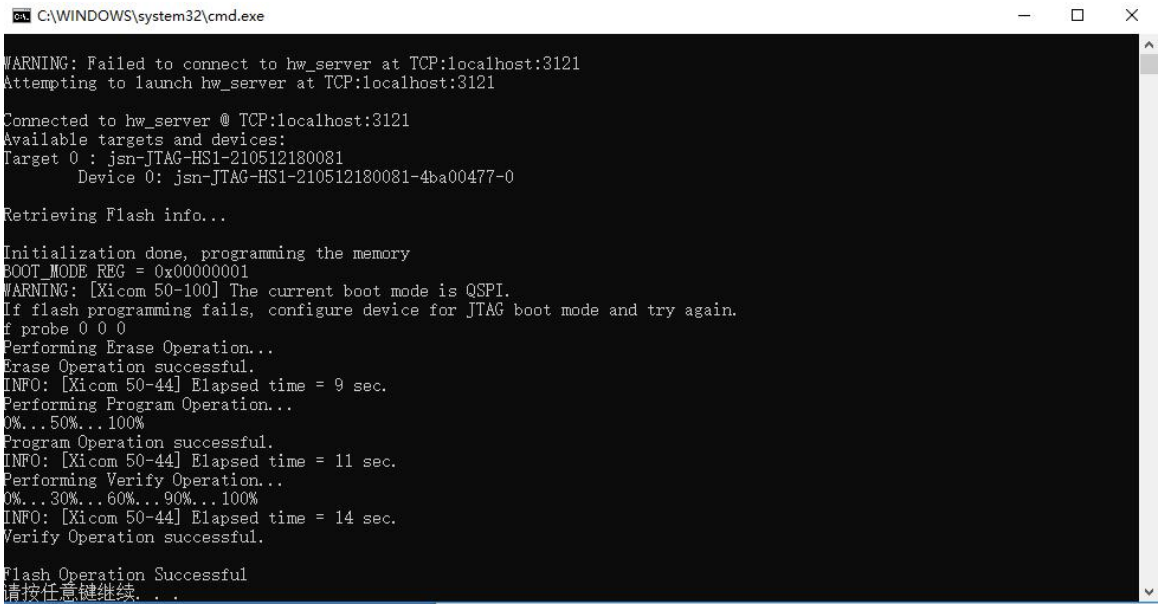
另外需要自己准备好 HDMI 显示器一台（支持 1080P60）。

实验工程及目录说明

工程名称	目录路径	说明
HLS 工程	hls	各算法导出 IP，由 vivado 引用
vivado 工程	vivado	
Vitis 工程	vivado/xx_project/xx_.vitis/	软件程序
常用 ip	vivado/ip_repo	Verilog 封装 ip
HLS 导出 IP	HLS/xx_project	Vivado 添加 IP 核时添加该路径
flash_load_comm	flash_load_comm	qspi 烧录

实验快速复现

1) 连接好 jtag 接口
2) 设置启动模式为 QSPI 启动
3) 将对应实验 SDK 工程中文件 BOOT.bin 拷贝到 flash_load_comm 目录下。
4) 开发板上电
5) 点击 program_qspi.bat 脚本运行。烧录成功后界面如下



6) 开发板重新上电

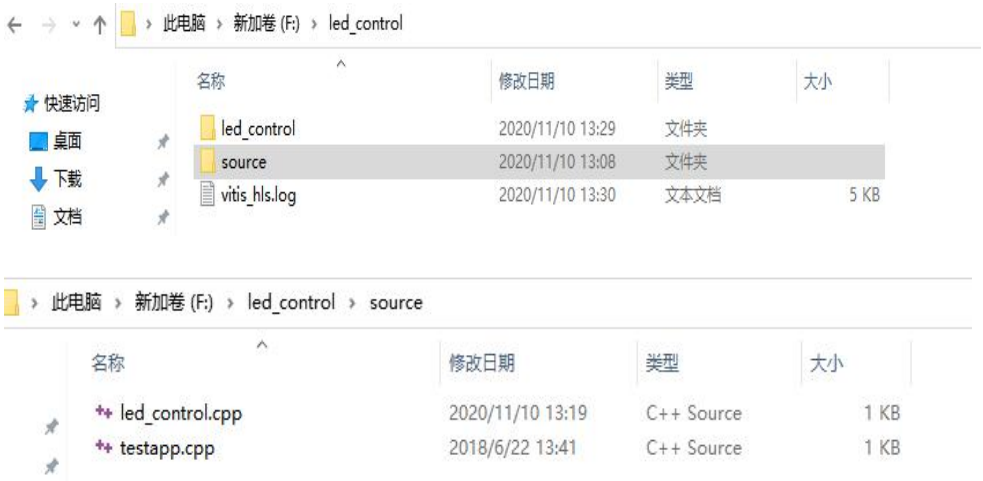
注：若 SDx 软件安装位置非默认 C 盘，则 program_qspi.bat 运行会出错，此时需要修改 program_qspi.bat 中的命令路径。

第一章 初识 Vitis HLS

1.1 Vitis HLS 基本操作

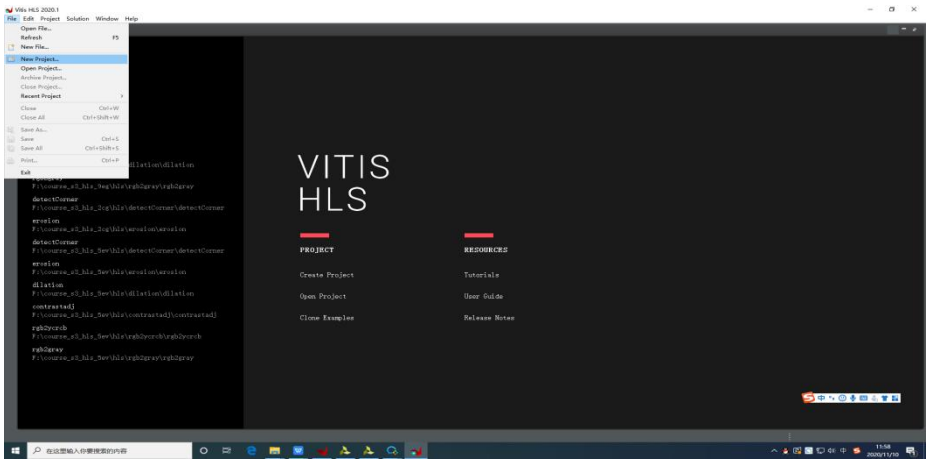
1.1.1 创建 vitis HLS 工程（以第二章工程为例）

1) 新建文件夹以工程名字命名，在文件夹中创建子文件夹 source 用于存放代码文件

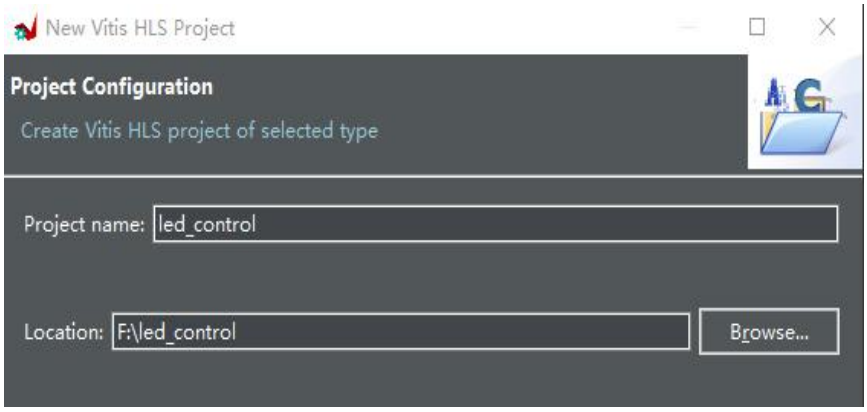


2) 打开 vitis HLS

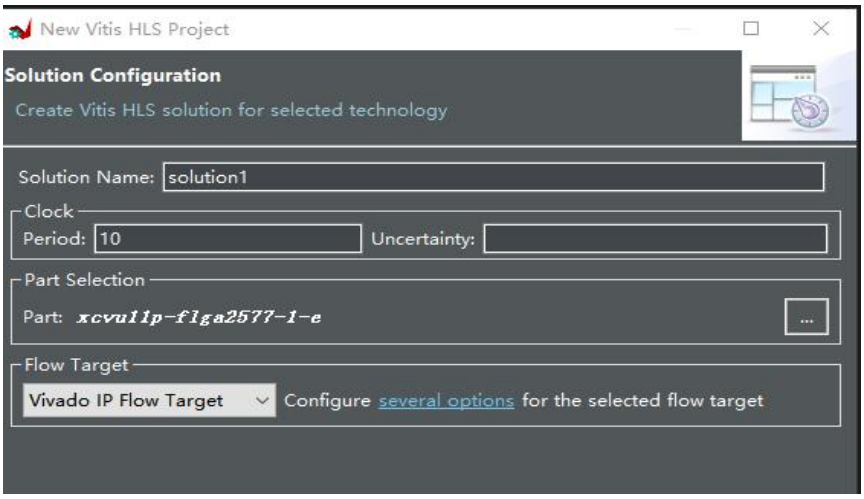
3) 在初识界面上选择 Create Project 或在左上角 File-> New Project



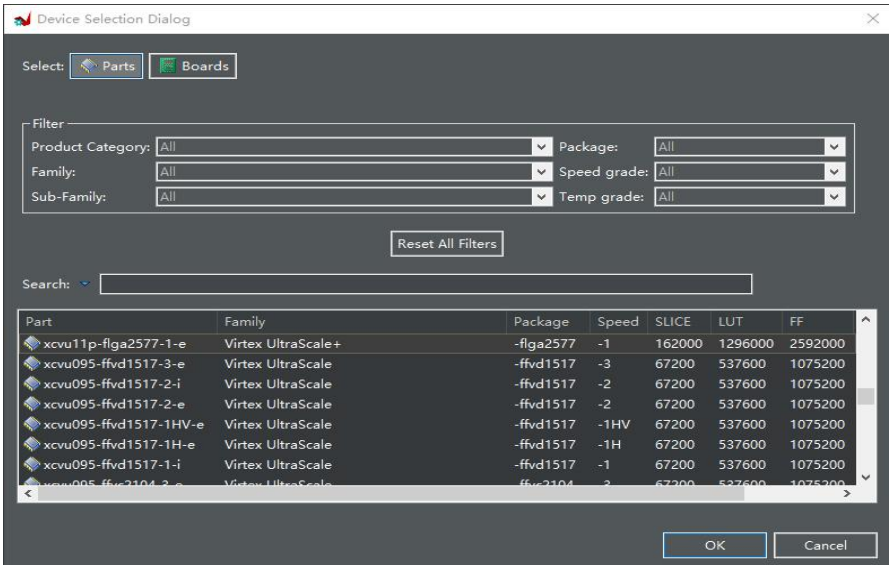
4) 设置工程名称及选择存放路径（这里以第二章的实验为例），如图



5) Next→Next 至如下界面，在 Clock Period 一栏可填入运行时钟的周期，软件会根据此时钟进行逻辑优化，可根据实际需要填写，默认是 10ns，clock uncertainty 默认为时钟周期的 12.5%。



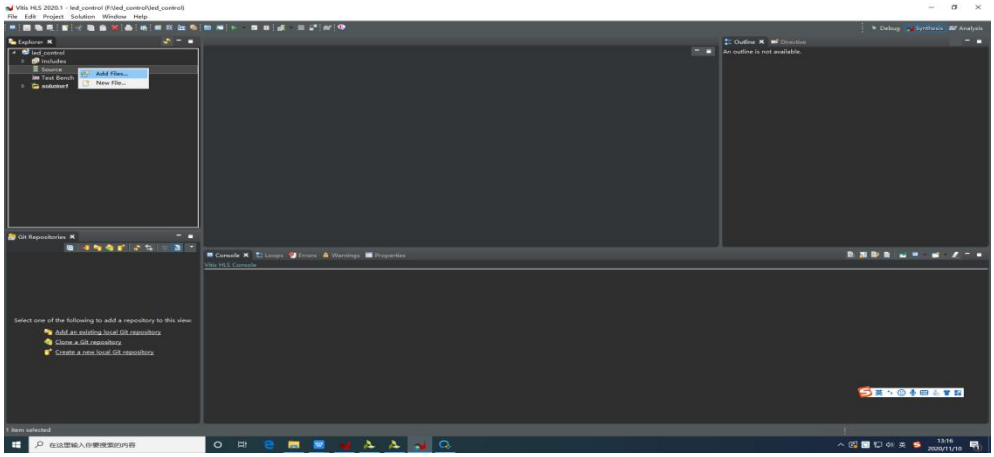
6) 点击图标 ...，至如下界面



7) Search 后输入框中输入 ZYNQ 芯片型号，在下拉列表框中选中对应器件，点击“OK”按钮

8) 点击“Finish”按钮，创建 vitis HLS 工程完成

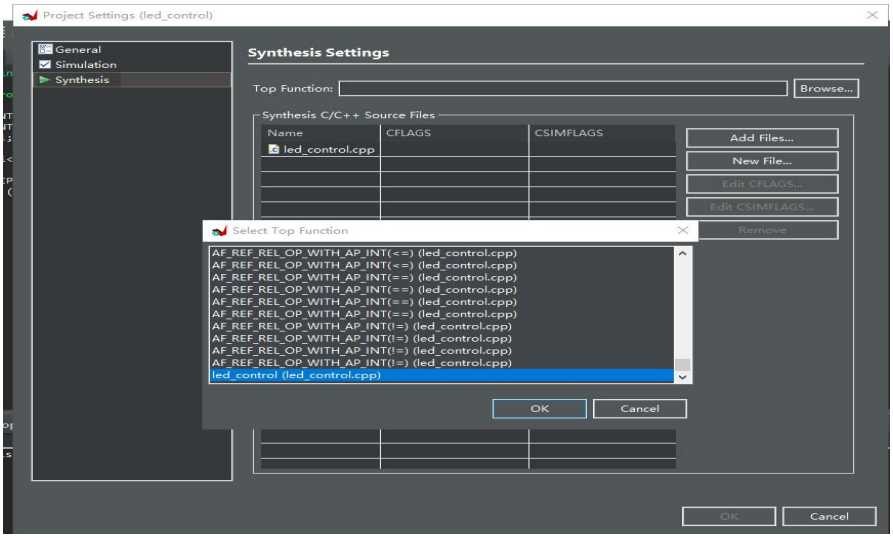
9) 选择”Source“图标右键，选择“Add Files…”，找到 source 文件夹里的“led_control.cpp”文件添加进工程



10) 双击打开“led_control.cpp”

11) “Ctrl+S”保存当前文档内容

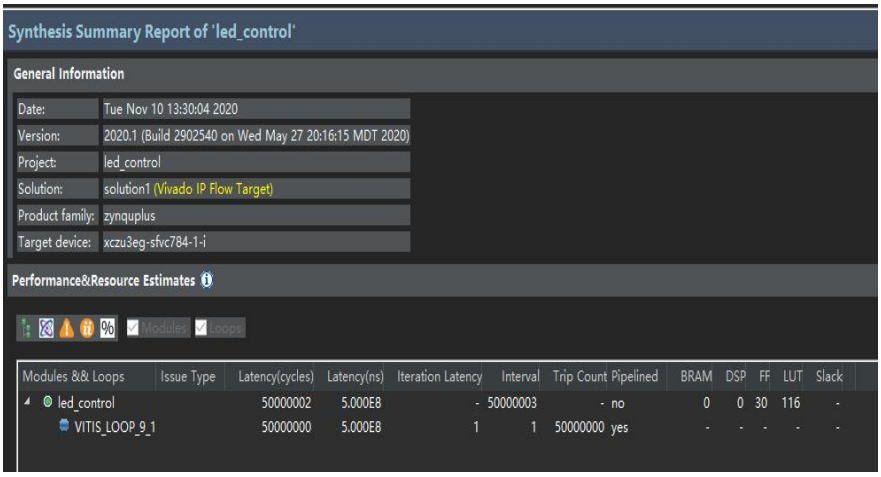
12) 点击图标 设置 top_function



注：vitis HLS 会把所有有关函数全部显示出来，通常我们需要设置的顶层函数在靠近最底部的位置

13) 点击图标 进行综合，或者点击菜单栏 Solution→Run C Synthesis→Active Solution

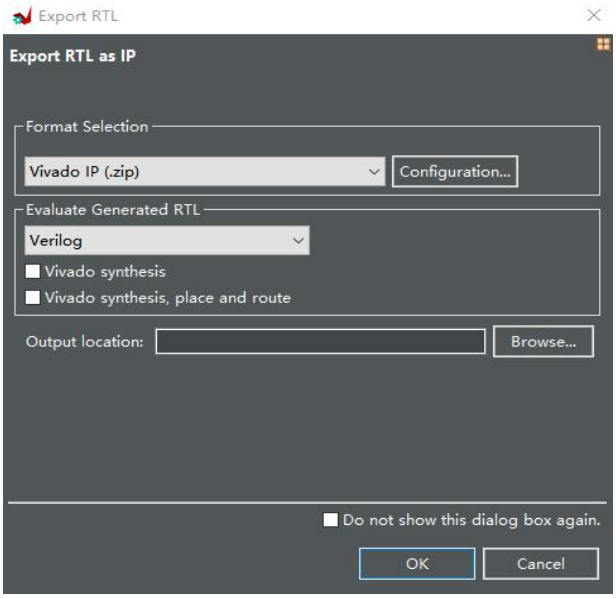
14) 综合之后通常我们要注意观察一下占用的资源和一些时序信息初步判断是否达到预期，可以通过点击图标 打开综合之后的综合评估报告



和 vivado HLS 不同 vitis HLS 会把时序和资源占用信息显示在一起，Latency（cycles）表示函数全部执行完成共需要经过多少个时钟周期，latency(ns)表示函数执行时间，单位是 ns，iteration Latency 表示循环中的迭代延时，interval 表示循环迭代执行间隔，trip Count 表示循环中预估最多执行时间，pipelined 表示是否进行流水线操作，后面章节会对此进行具体介绍，Bram 表示 Block RAM 占用率，DSP 表示 DSP 占用率，FF 表示触发器占用率，LUT 表示查找表占用率，Slack 表示是否出现时序违规，其中特别注意查找表是 FPGA 中最重要的资源，通常的资源不足指的就是 LUT 资源不足，后续章节会介绍常用的面积优化方法降低 LUT 资源的消耗。

1.1.2 Vitis HLS 生成 IP 核

1）点击图标 导出 IP 核



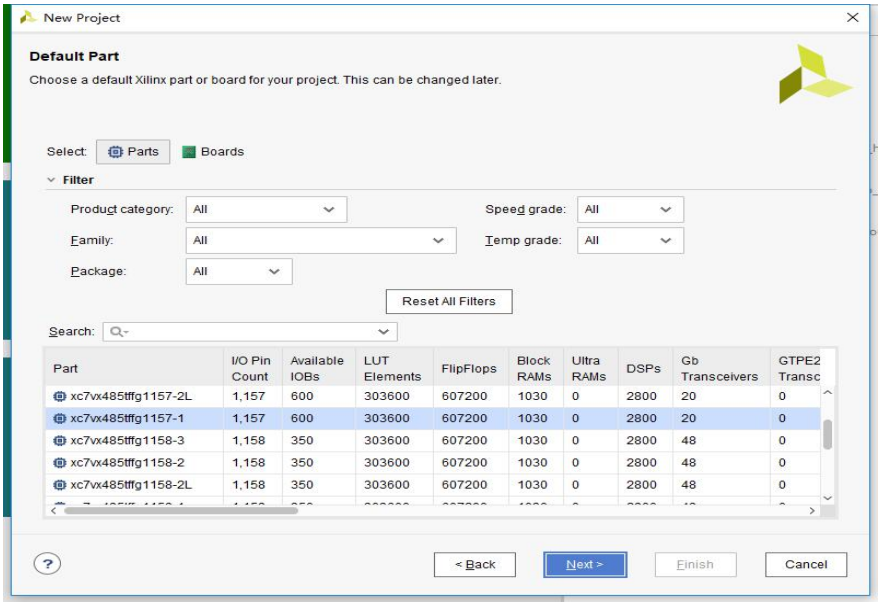
2）导出之后打开 vivado 软件

3）点击“Create Project“标签，弹出工程创建向导

4）Next

5）设置存放路径和工程名

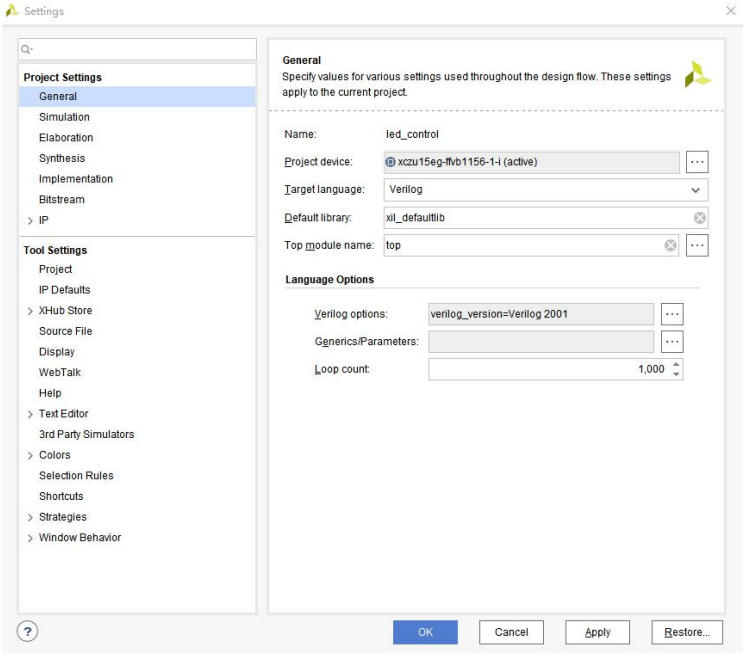
6）Next→Next→ Next→ Next，至如下界面



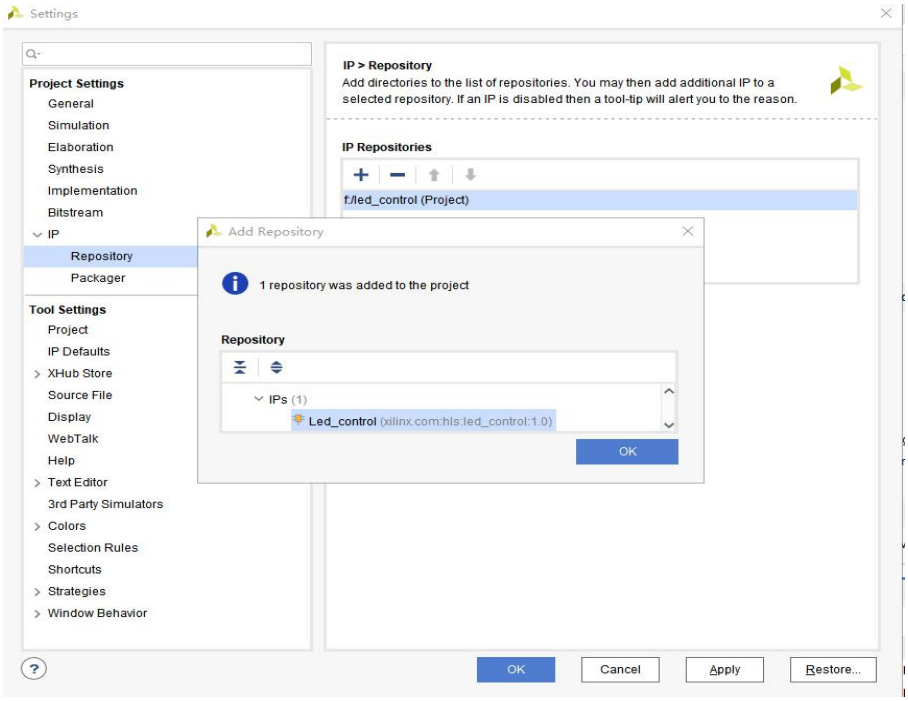
7）Search 后输入框中输入 ZYNQ 芯片型号，在下拉列表框中选择器件，注意器件要和 vitis HLS 中的器件一致

8）Next→Finish，完成 vivado 工程创建

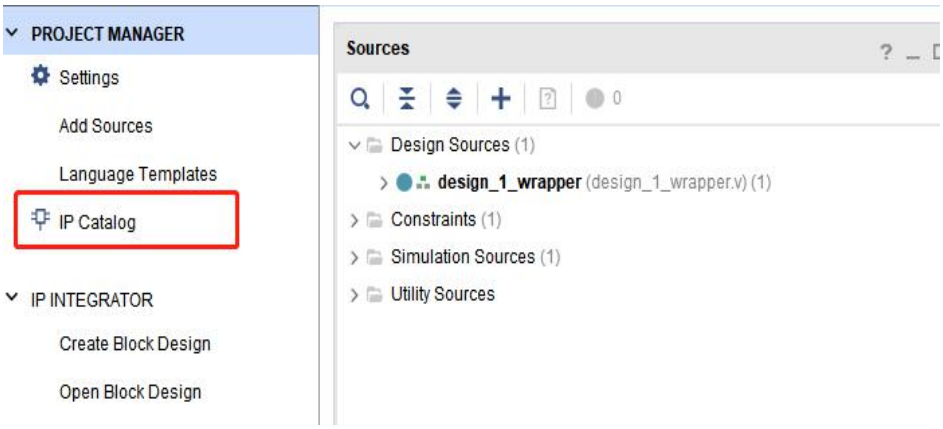
9）进入到 vivado 软件工程界面，点击图标  ，打开如下界面：



10）选择 IP->repository，添加 IP 核路径



11）选择左边 IP Catalog



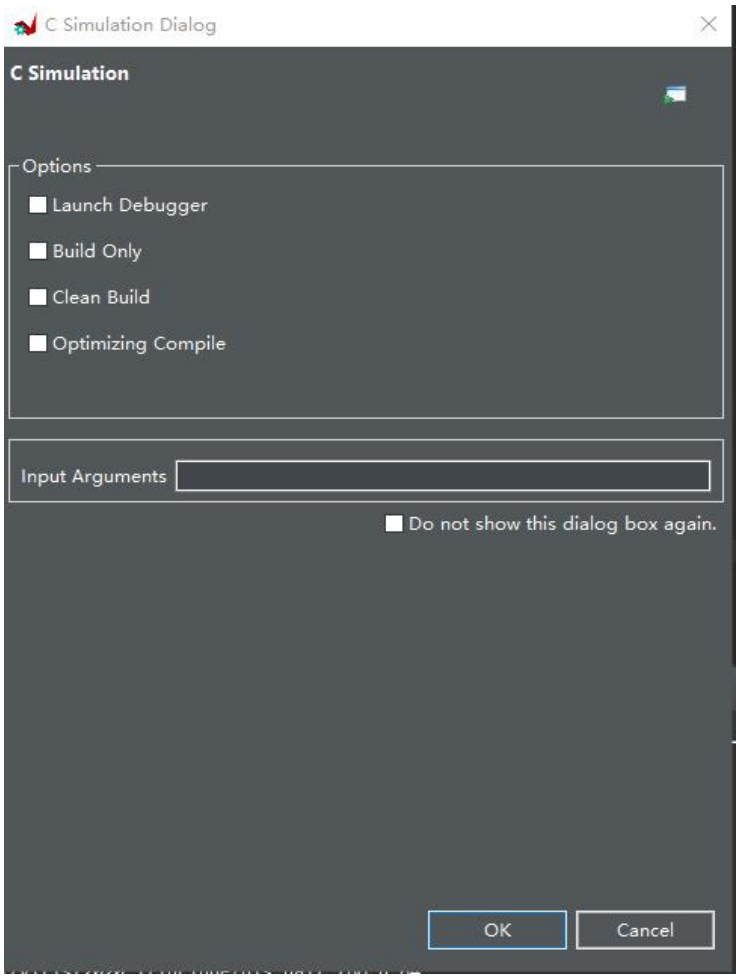
12）在下图界面中，输入 hls 能看到添加的 IP 核，此时我们的 IP 核已经添加完成



1.1.3 Vitis HLS 仿真

Vitis HLS 中的仿真分为 c 仿真和 C_RTL 交叉仿真，通常先在 HLS 中进行 c 仿真，通过 c 仿真之后生成 IP 核，将 IP 核添加到 vivado 里进行仿真观察波形图

1）点击图标  ，弹出界面如下

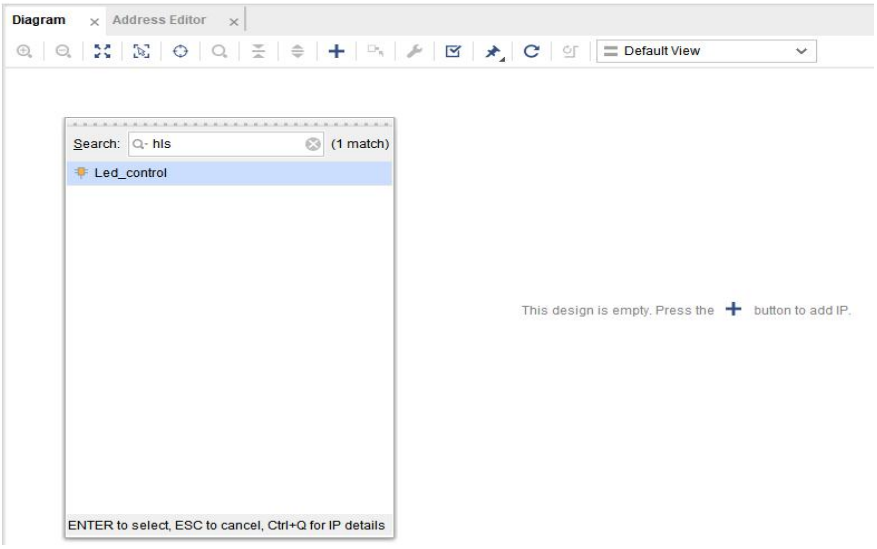


如果测试文件的主函数中没有参数可以直接保持默认设置点击 ok 观察仿真效果

3）通过 c 仿真之后进行 c_rtl 交叉仿真，按照之前的步骤将 IP 核添加到 vivado 中，打开图标 create block design



4）在显示的 block design 中点击加号添加 IP 核（以第二章的入门工程为例）



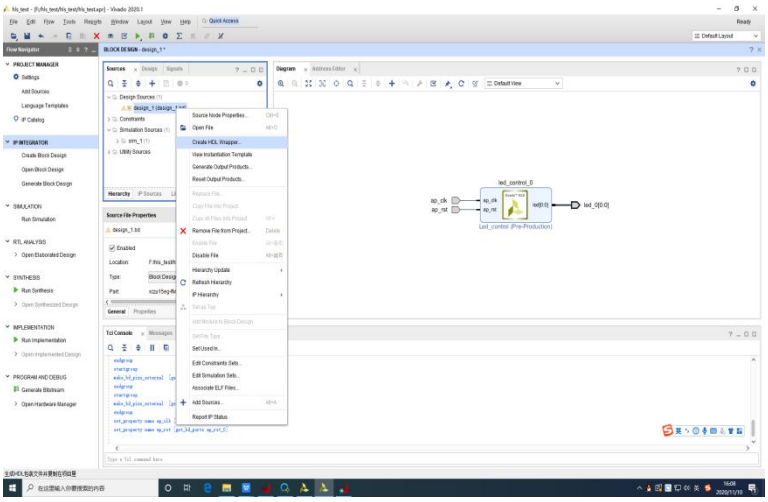
4）此时可以看到我们的 IP 核被添加到可视化界面中



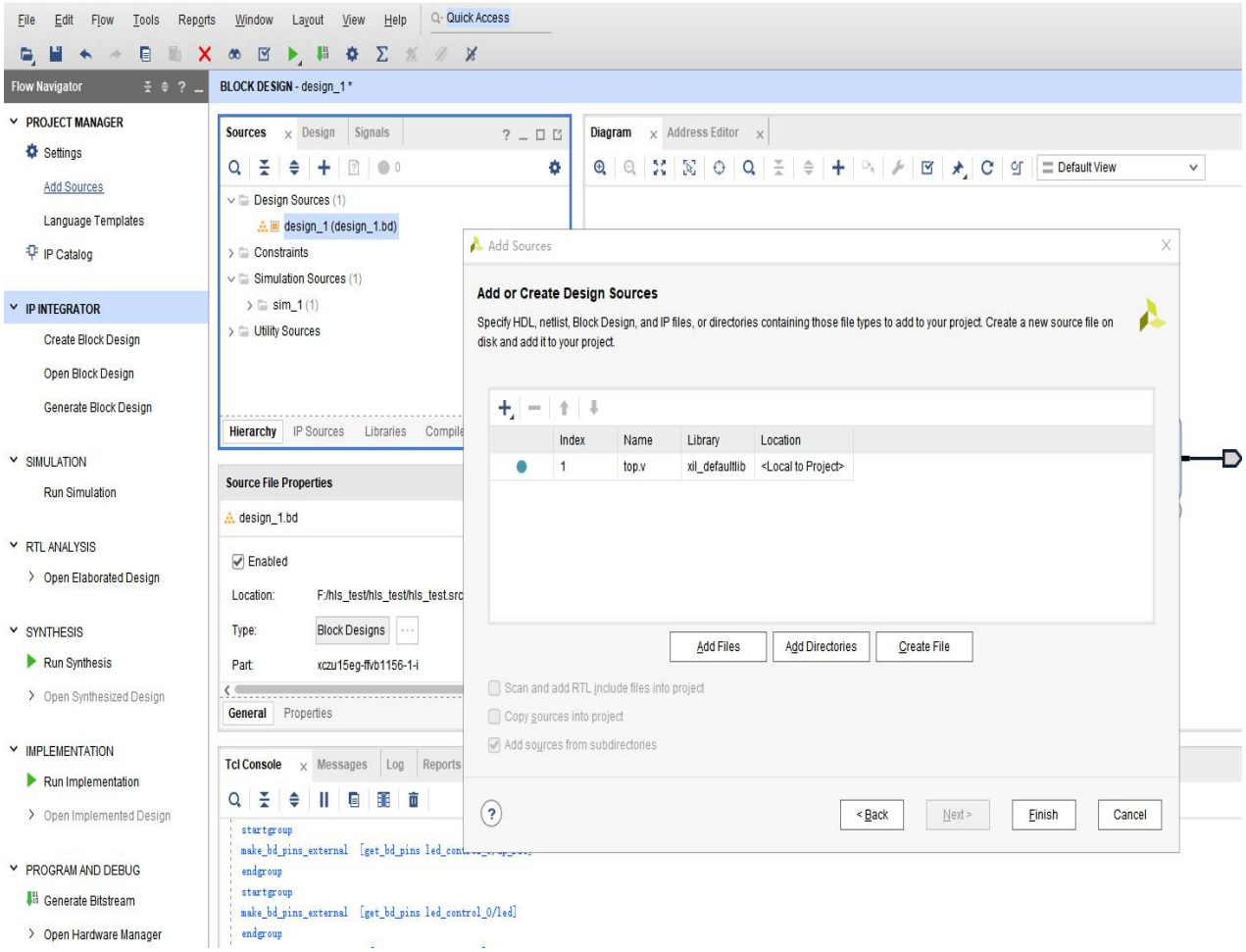
5）在 ap_clk,ap_rst,led 上按 ctrl+T 生成端口



6)右键选择 Create HDL Wrapper



7) 创建 top.v 文件



8）在 top.v 文件中输入如下代码

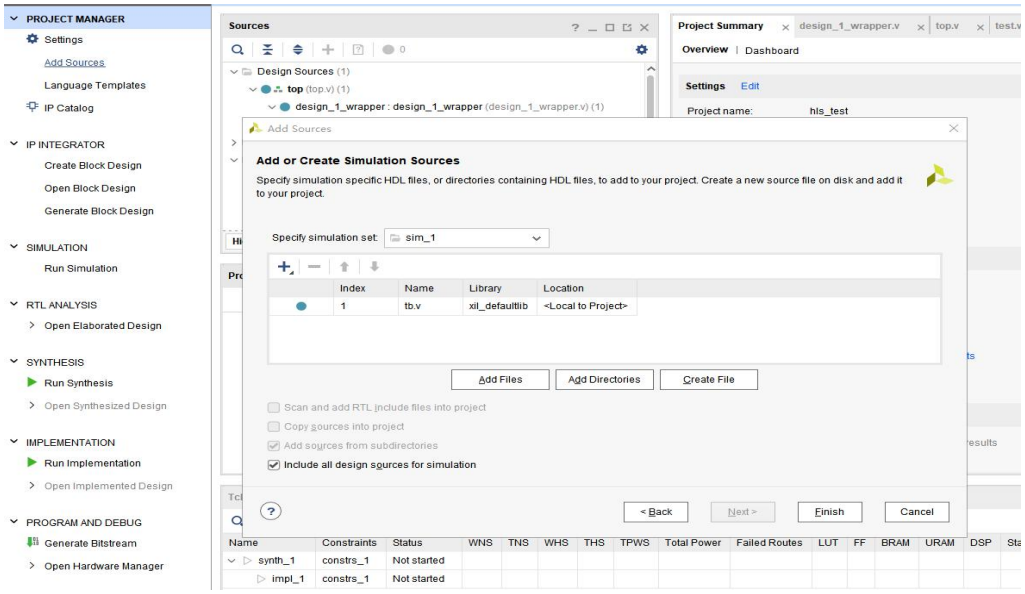
```
module top(
    input clk ,
    input ap_rst
    );
//internal variable declation

  //regs
    reg [31:0]cnt=32'd0 ;
//wires
    wire led ;
  //main codes

always @ ( posedge clk )
begin
    if ( ap_rst == 1'b1 ) begin
        cnt <= 32'd0 ;
    end
    else begin
        cnt <= cnt + 32'd1 ;
    end
end

    design_1_wrapper design_1_wrapper(
        .ap_clk( clk ) ,
        .ap_rst ( ap_rst ) ,
        .led_0 ( led )
        ) ;
endmodule
```

9）新建 test.v 仿真文件



10）在 test.v 中输入如下代码

```
module tb;

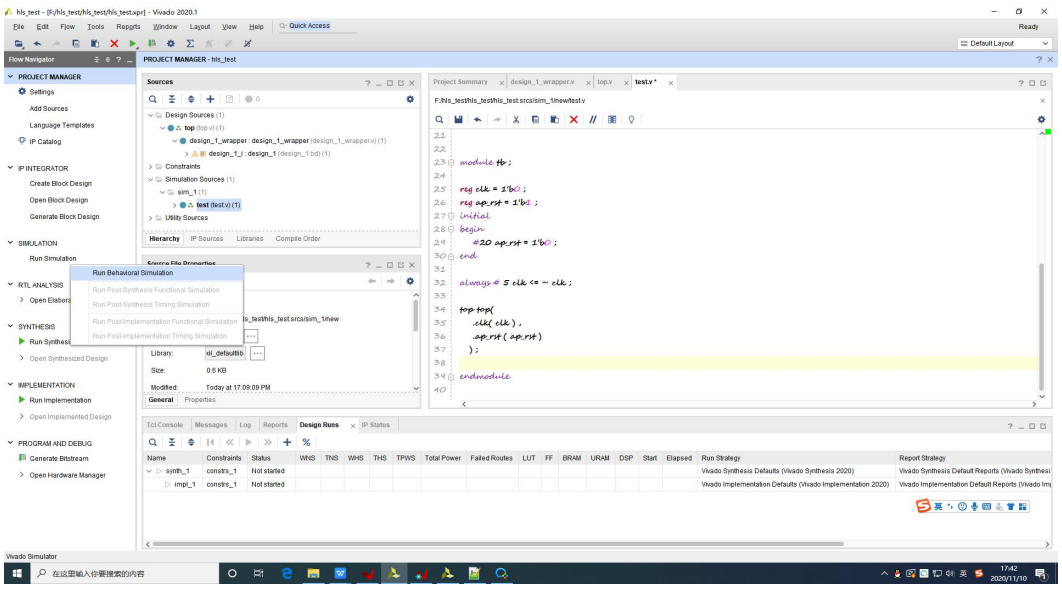
reg clk = 1'b0 ;
reg ap_rst = 1'b1 ;
initial
begin
    #20 ap_rst = 1'b0 ;
end

always # 5 clk <= ~ clk ;

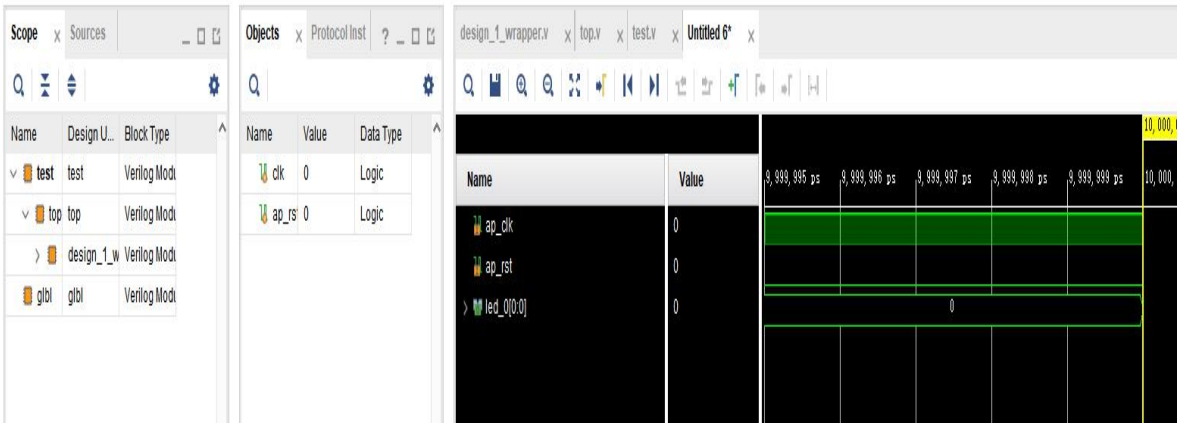
top top(
    .clk( clk ),
    .ap_rst ( ap_rst )
    );

endmodule
```

11）选择 simulation->run simulation



12)在波形图中可以看到对应信号波形，判断是否符合预期运行效果

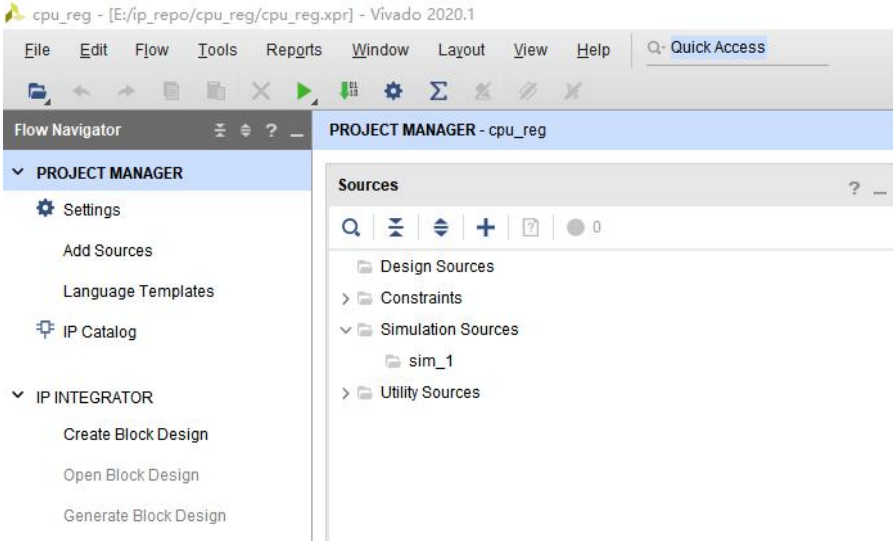


注：仿真时要按照对应 HLS 设置的对应接口输入对应的信号，后续章节会对接口类型和设置方式做具体介绍

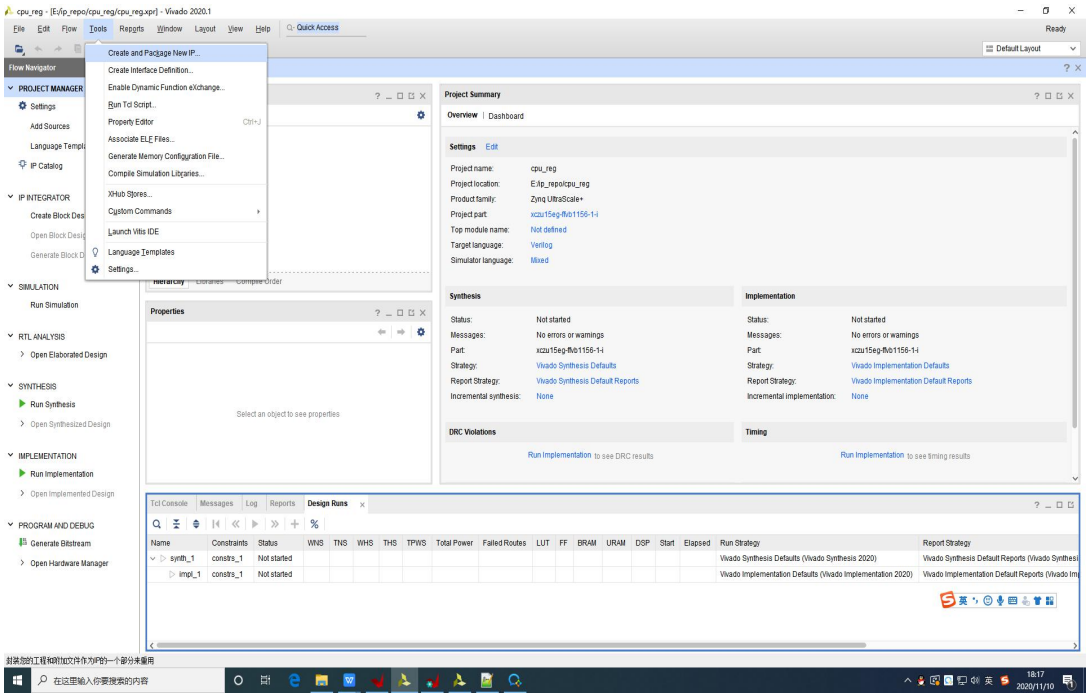
1.2 Vivado 生成 CPU 寄存器

CPU 寄存器是 PL 和 PS 端常用的交互方式，PS 和 PL 之间可以通过 AXI 总线进行数据交互，主要 AXI 总线接口包含 AXI_FULL 类型、AXI_LITE 类型，AXI_STREAM 类型，Vivado 向我们提供了接口编写的模板，只需要使用模板并在模块中 user logic 部分里添加适当的内容就可以利用 AXI 总线接口实现需求，通常情况下向 HLS 输入的待处理数据接口，以及经过处理之后的输出数据接口会被设置为 AXI 类型，便于和 CPU 进行交互，这里以设置 s_axilite 为例介绍如何利用 CPU 寄存器的模板以及和 HLS 交互的常用方法。

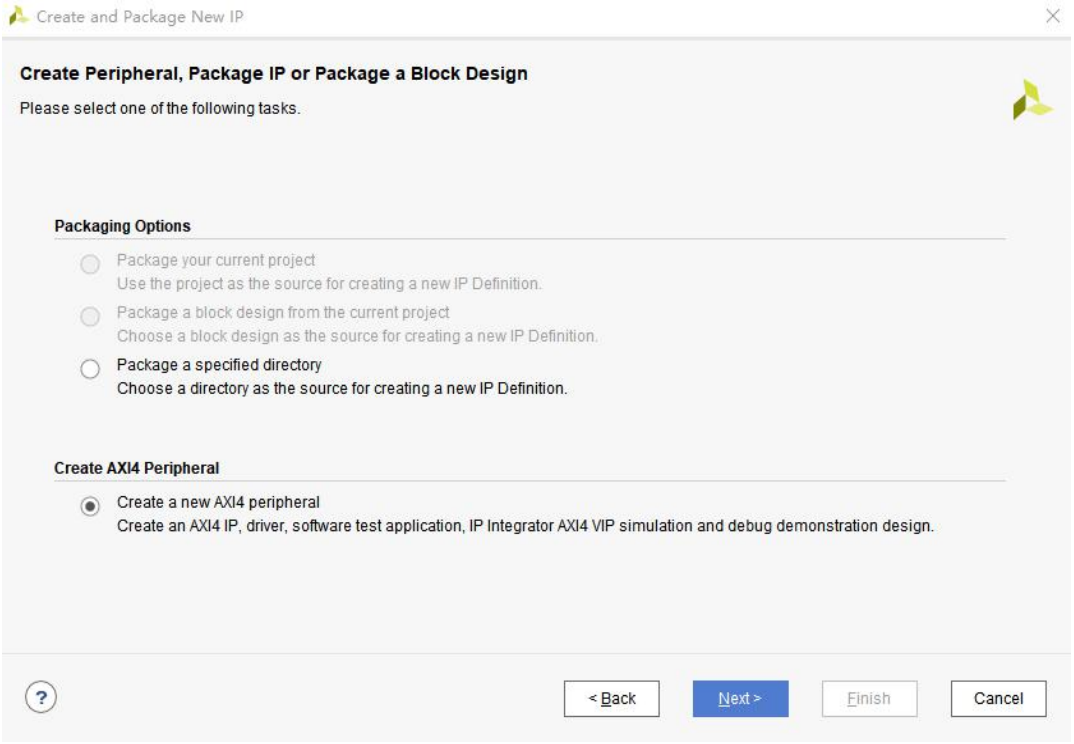
1）创建一个空白的 vivado 工程命名为 cpu_reg



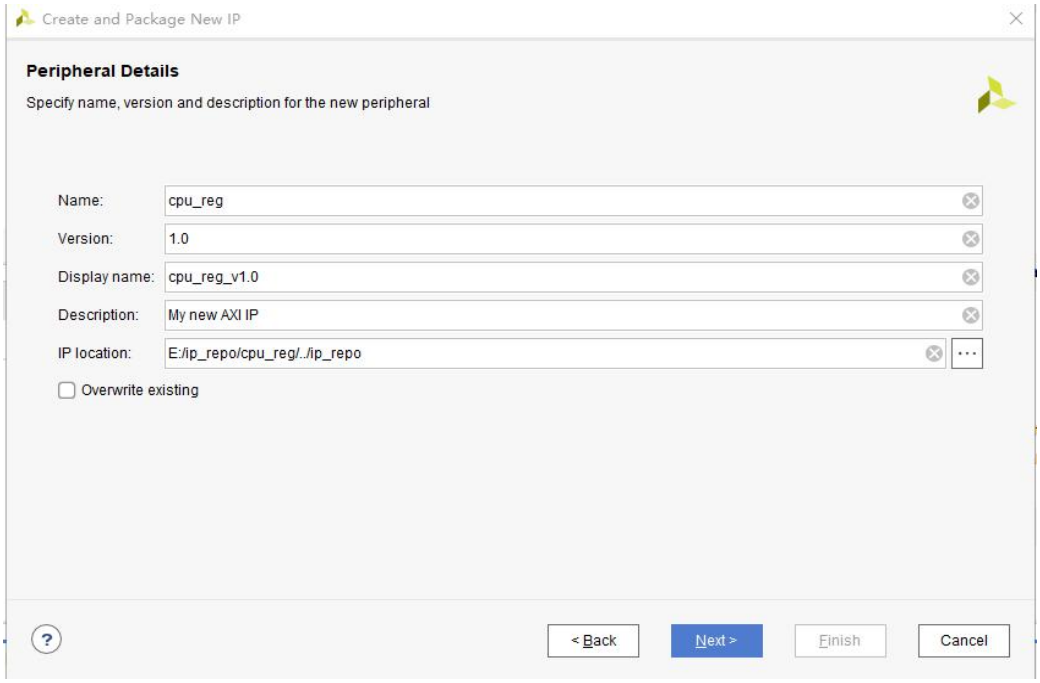
2）选择 tools->package and create new IP



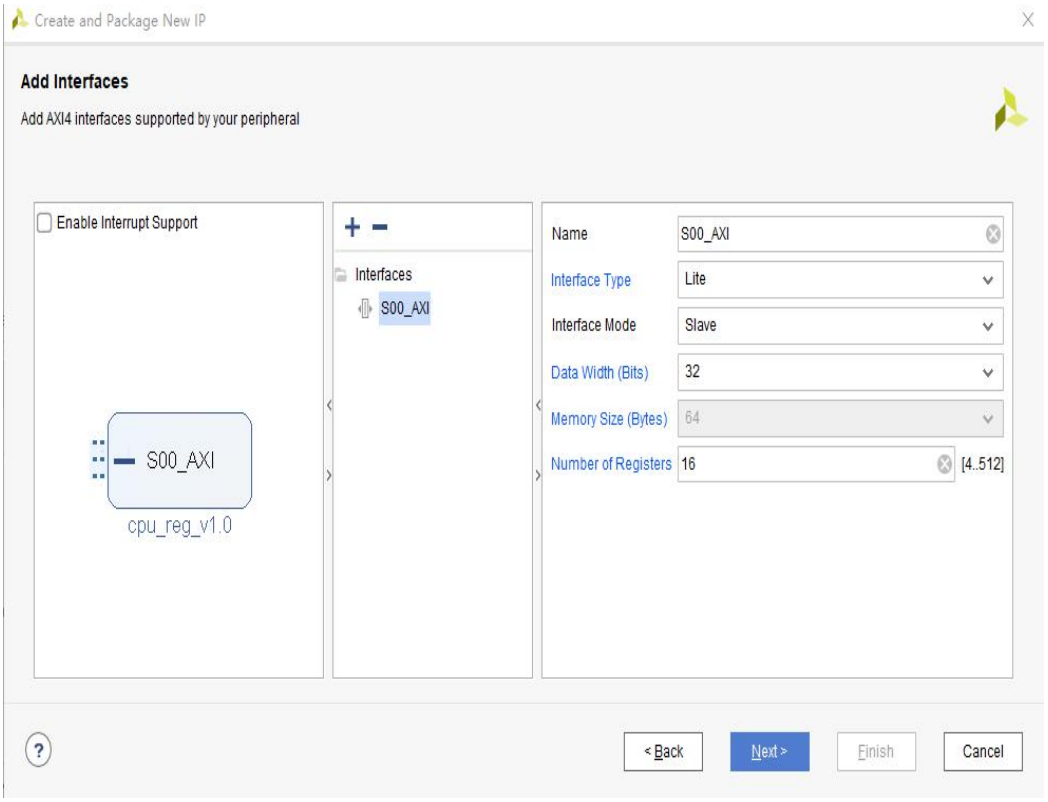
3) next->next 直到出现如下界面，选择 Create a new AXI4 peripheral



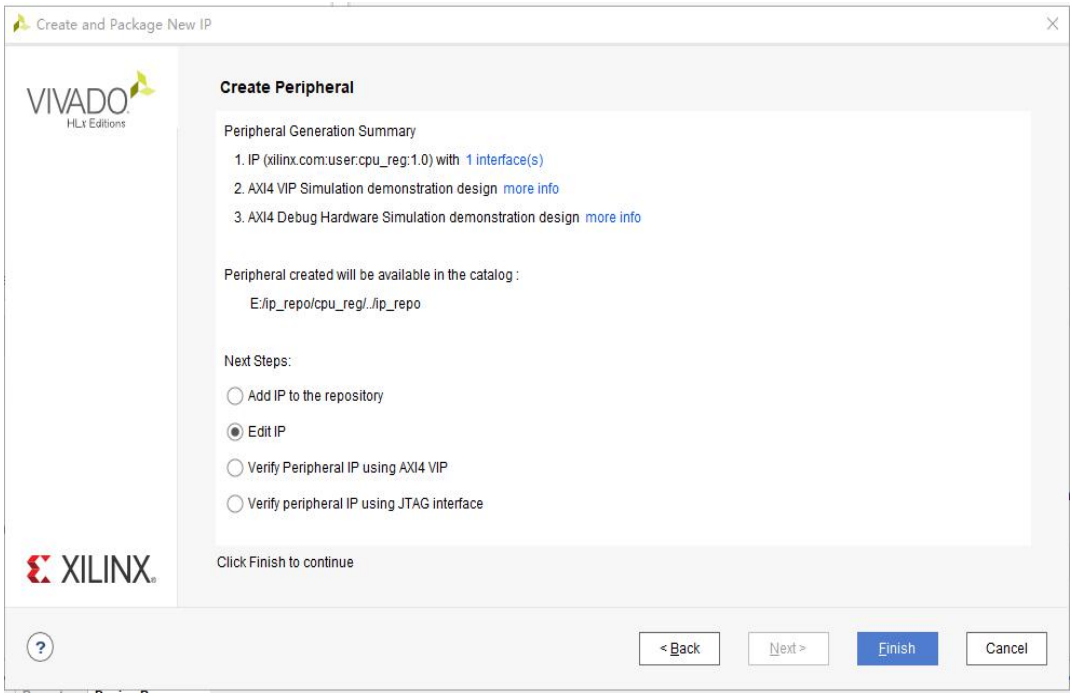
4） Next->Next 将名字修改为 cpu_reg



5）Next->Next 按照如下界面配置，Interface Type 那一栏可以选择合适的接口类型，分 stream,full,lite 三种，Interface Mode 选择主从，要注意和对接的 module 之间正确对应,这里为了便于和 HLS 交互我们选择 Slave lite



6）Next->Next 选择 edit IP 点击 Finish



7）左侧打开 cpu_reg_v1_0_S00_AXI_inst,按如图部分做一些修改

```
// Add user logic here

assign reg_out_0 = slv_reg0 ;
assign reg_out_1 = slv_reg1 ;
assign reg_out_2 = slv_reg2 ;
assign reg_out_3 = slv_reg3 ;
assign reg_out_4 = slv_reg4 ;
assign reg_out_5 = slv_reg5 ;
assign reg_out_6 = slv_reg6 ;
assign reg_out_7 = slv_reg7 ;
assign reg_out_8 = slv_reg8 ;
assign reg_out_9 = slv_reg9 ;
assign reg_out_10 = slv_reg10 ;
assign reg_out_11 = slv_reg11 ;
assign reg_out_12 = slv_reg12 ;
assign reg_out_13 = slv_reg13 ;
assign reg_out_14 = slv_reg14 ;
assign reg_out_15 = slv_reg15 ;

// User logic ends
```

```
// Users to add ports here

output wire [31:0] reg_out_0 ,
output wire [31:0] reg_out_1 ,
output wire [31:0] reg_out_2 ,
output wire [31:0] reg_out_3 ,
output wire [31:0] reg_out_4 ,
output wire [31:0] reg_out_5 ,
output wire [31:0] reg_out_6 ,
output wire [31:0] reg_out_7 ,
output wire [31:0] reg_out_8 ,
output wire [31:0] reg_out_9 ,
output wire [31:0] reg_out_10 ,
output wire [31:0] reg_out_11 ,
output wire [31:0] reg_out_12 ,
output wire [31:0] reg_out_13 ,
output wire [31:0] reg_out_14 ,
output wire [31:0] reg_out_15 ,

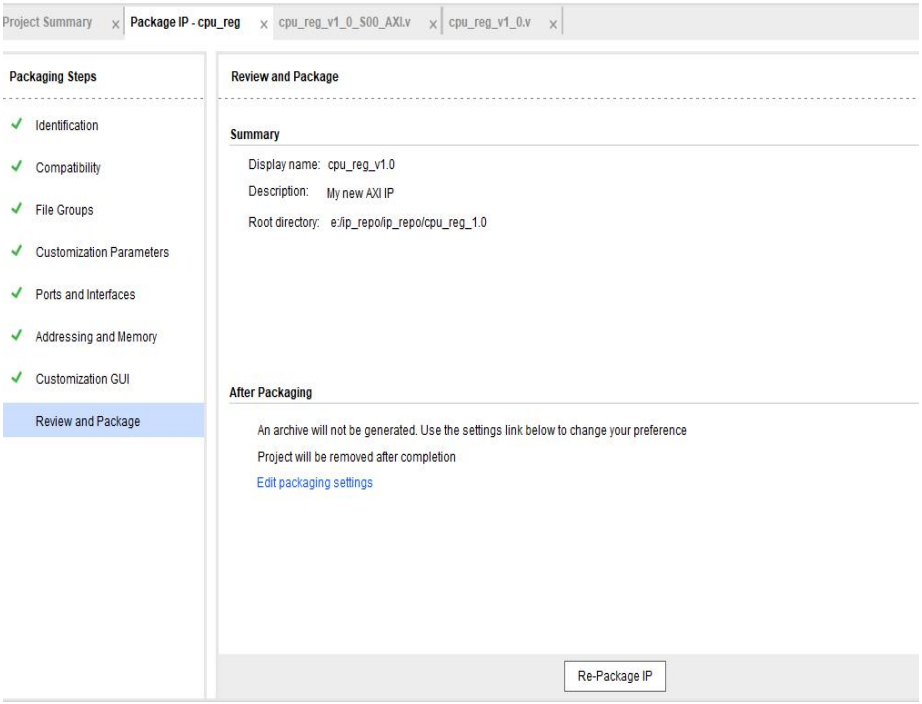
// User ports ends
```

8) 左侧打开 cpu_reg_v1_0,按如图部分做一些修改

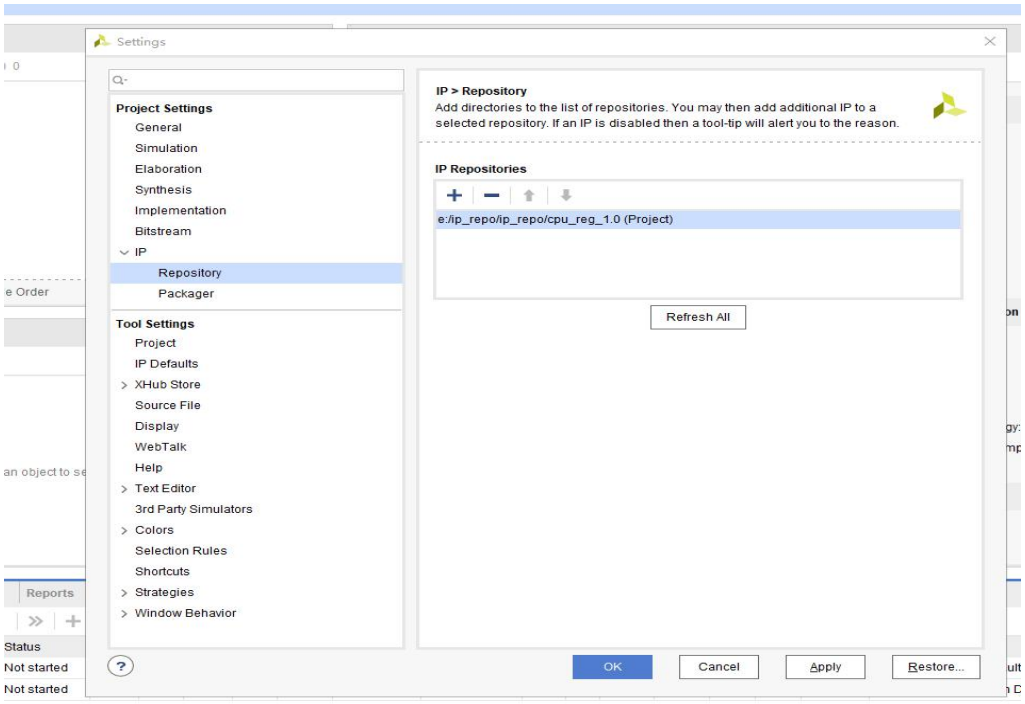
```
// Users to add ports here
output wire [31:0] cpu_reg_0 ,
output wire [31:0] cpu_reg_1 ,
output wire [31:0] cpu_reg_2 ,
output wire [31:0] cpu_reg_3 ,
output wire [31:0] cpu_reg_4 ,
output wire [31:0] cpu_reg_5 ,
output wire [31:0] cpu_reg_6 ,
output wire [31:0] cpu_reg_7 ,
output wire [31:0] cpu_reg_8 ,
output wire [31:0] cpu_reg_9 ,
output wire [31:0] cpu_reg_10 ,
output wire [31:0] cpu_reg_11 ,
output wire [31:0] cpu_reg_12 ,
output wire [31:0] cpu_reg_13 ,
output wire [31:0] cpu_reg_14 ,
output wire [31:0] cpu_reg_15 ,
// User ports ends
```

```
.cpu_reg_0 ( cpu_reg_0 ),
.cpu_reg_1 ( cpu_reg_1 ),
.cpu_reg_2 ( cpu_reg_2 ),
.cpu_reg_3 ( cpu_reg_3 ),
.cpu_reg_4 ( cpu_reg_4 ),
.cpu_reg_5 ( cpu_reg_5 ),
.cpu_reg_6 ( cpu_reg_6 ),
.cpu_reg_7 ( cpu_reg_7 ),
.cpu_reg_8 ( cpu_reg_8 ),
.cpu_reg_9 ( cpu_reg_9 ),
.cpu_reg_10 ( cpu_reg_10 ),
.cpu_reg_11 ( cpu_reg_11 ),
.cpu_reg_12 ( cpu_reg_12 ),
.cpu_reg_13 ( cpu_reg_13 ),
.cpu_reg_14 ( cpu_reg_14 ),
.cpu_reg_15 ( cpu_reg_15 ),
 .S_AXI_ACLK(s00_axi_aclk),
 .S_AXI_ARESETN(s00_axi_aresetn),
```

8）选择 review and package 点击 Re-Package IP



9) 在工程中点击设置->IP->repository



此时我们的 CPU 寄存器已经成功生成

1.3 HLS 接口综合

在 HLS 接口主要分为三类：clock and reset interface，Block Level interface ,Port Level Interface 我们能够综合的主要是 Block Level interface ,Port Level Interface

Block Level Interface:

分为 ap_ctrl_none,ap_ctrl_hs,ap_ctrl_chain 三种，ap_ctrl_none 指没有 Block Level interface,ap_ctrl_hs 包含：ap_start,ap_idle,ap_ready,ap_done 四种，ap_start 捕捉到上升沿时 hls 的 topfunction 开始执行， ap_idle 在 topfunction 开始执行之后处于低电平，执行完成之后处于高电平，ap_ready 处于高电平时表示 topfunction 准备完毕，开始执行之后处于低电平，ap_done 在 topfunction 执行完成之后处于高电平，当 ap_start 捕捉到上升沿之后重新处于低电平，ap_ctrl_chain 比 ap_ctrl_hs 多一个 ap_continue 接口，如果在 topfunction 执行完成之后 ap_continue 处于低电平，此时 IP 核会终止运行，只有在 ap_continue 被置于高电平才会开始新一轮运行

Port Level Interface :

接口和 topfunction 参数有关，主要分为 AXI4 Interface,No I/O Interface,wire

handshake,memory Interface 四种

AXI4 Interface :

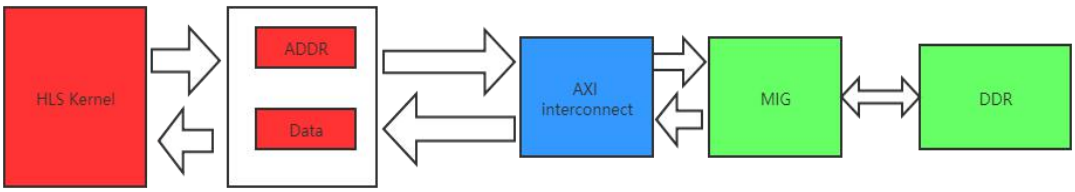
分为 axi,m_axi,s_axilite 三种，基于 AXI 总线协议，用于 PS 端和 PL 端数据交互。

AXI:

指的是 axi-stream 接口，包含 side-channel 和 without side-channel 两类，通常用于接外界连续输入的数据，比如我们想把若干组数据从 PS 端送入 PL 端，可以将 IP 核输入设置为 axi 类型接口

m_axi:

指的是 master 模式的 axi-full 接口，HLS 做为主模式，从 PS 端读写数据，多用于将数据通过 AXI-HP 接口送入 PS 端，进而送入 DDR 中，或者从 DDR 中读出，主要的读写方式和连接形式如图所示：



注意如果将接口设置为 m_axi 模式会涉及到一个读写延时的问题，整体大概在 30cycles 左右，涉及到一个叫 burst transfer 的优化，用合适的读写 DDR 的方法可以在一定程度上提升数据处理的整体速度

s_axilite:

指的是 axi-lite 接口，属于轻量级的 axi 接口，通常用于和 PS 端 CPU 数据交互，多和 AXI-GP 接口相连，用于送入 CPU 提供的配置信息或向 CPU 写入配置信息。

NO I/O Interface

分为 ap_none 和 ap_stable 两种

ap_none:

多接单个数据，没有任何相关联的其他信号，属于默认参数接口。

ap_stable:

用于配置输入参数，参数配置通常是只在复位状态下才会发生变化，用的不多

WIRE HANDSHAKE

分为 ap_vld,ap_ovld,ap_ack 三种

ap_vld:

输入信号，通常接单个数据，包含有效信号和输入信号

ap_ovld:

输出信号，通常接单个数据，包含有效信号和输出信号

ap_ack:

只包含有效信号，不包含数据，基本不用

Memory Interface

包含 ap_bram、ap_memory、ap_fifo 三种

ap_bram:

当输入数据类型是数组时使用这种类型接口，可以直接和 Bram 相连

ap_memory:

功能和 ap_bram 相同，只是 ap_bram 在综合时会把有关信号打包在一起,ap_memory 会综合成多个零散的接口

ap_fifo:

输入数据类型是数组时使用这种类型接口，用于和 fifo 直接相连

1.3 HLS 常用优化

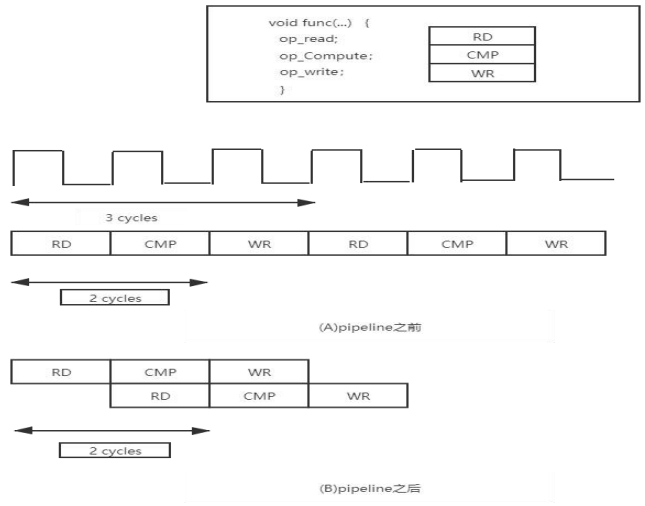
分成 Throughput,Area,Latency,logic 四种方式优化

Throughput:

吞吐量优化主要分为以下多种类型

Function and Loop PipeLine:

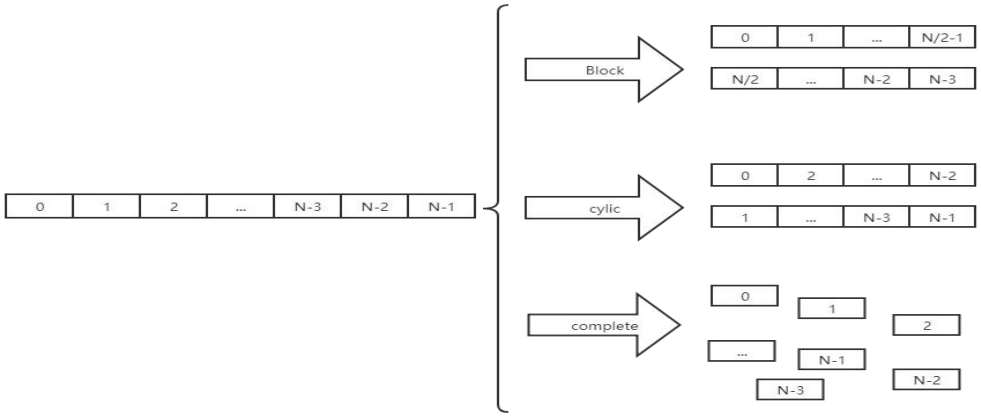
函数或循环流水化可以减少相应的执行间隔，对函数做流水化之后函数体内的循环会自动展开，提升运行速度，对循环做流水化之后循环次数计数器可以不必等循环体内的所有语句执行完之后再增加，增大数据的流通量，以下图为例：



可以看出流水线优化之后的执行时间得到缩短

Partitoning Arrays:

数组拆解指的是将一个大数组拆解为若干个小数组，有三种模式 block、cylic、complete，三种拆解方式如图所示:



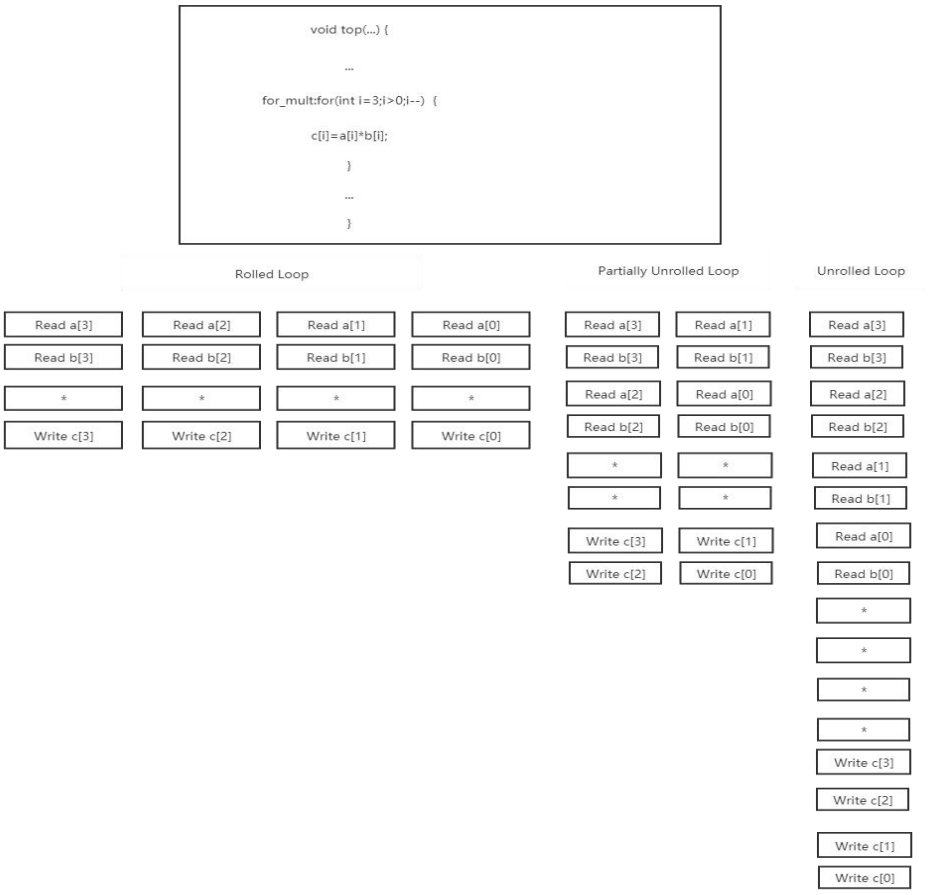
在 HLS 中数组可以被综合为 BRAM 和 FIFO 两种形式，这两种器件都无法提供两个以上接口供数据读写，如果对数组进行拆解，用多个 BRAM 表示原本的大数组，可以提供更多端口供数据读出，对于 pipeline 和 unroll 两种优化方式有非常大的影响

Remove False Dependency:

HLS 中的语句执行有顺序执行和并行两种方式，当 HLS 认为两个语句之间没有依赖关系 HLS 会选择同时执行两条语句，提升运行的速度，这种速度的提升有时候会对整体的执行速度有非常大的影响，HLS 语句之间有分四种情况，RAW(读完再写)，WAR(写完再读)，WAW(写完再写)，RAR（读完再读），其中只有 RAR 情况下才能被认为此时两个语句之间没有依赖关系可以同时执行，注意如果两个语句之间不是 RAR 的情况强行设置 Dependency=false 会导致在综合时出错或者 HLS 会用非常激进的方式满足要求，对功能会有很大的影响，因此这种优化方式要尽量慎重使用

UNROLLING:

循环展开可以降低整体循环的延时，一个周期内处理更多的数据，带来的影响会是增加更大的面积消耗，循环展开的影响具体如下图所示：



可以看到展开之前需要四个周期消耗一个乘法器，部分展开之后需要两个周期两个乘法器，完全展开之后需要一个之周期四个乘法器，同时循环展开要保证能够同时从 bram 中读取相应的数据，图中可已看出部分展开需要同时从 bram 读取两个数，可以通过指定数组资源是 dual-port-Bram 实现，完全展开需要同时从 bram 读取四个数，然而不存在四个端口的 bram，因此要对数组做完全拆解，变成触发器才能实现完全展开，且该功能对面积消耗很大，要慎重使用。

Dataflow:

HLS 执行两个有依赖关系的函数时会在上一个函数所有语句执行完之后执行下一函数，此时总的执行时间是两个函数执行时间之和，进行 DATAFLOW 优化之后总执行时间是取两个函数中执行最久的时间，极大的减少运行时间，可以将这种优化方式认为是函数之间的 pipeline,在很多视频处理的函数之间，如果分辨率较高且对帧数要求较高的情况下通常会使用这种优化方式保证较好的显示效果，使用这种优化方式有几个限制：

1）函数参数要设置为 stream 型，按照 fifo 综合数组

2）上一函数的输出只能给一个函数做输入，不可以同时给两个以上函数做输入

3）如果有多个函数按顺序执行，上一函数的输出不可以绕过中间的函数直接给之后的函数

做输入

4）函数参数不能出现反馈的情况，也就是函数输出只能输入它之后的函数，不能输入它之前的函数

5）函数条件执行，如果某个中间的函数要在一定条件下才能执行，此时不能用 DATAFLOW 进行优化

6）最后的函数多种条件下多种输出，此时不能用 DATAFLOW 优化

对于第一条要特别注意如果将数组综合为 stream，设置 depth 过小会导致整个函数出现 halt 的情况，此时新数据无法输入，旧数据未有读出，导致整个 IP 核处于停止运行的状态，整个工程可能会彻底终止运行。

Latency

延时优化主要包含以下两种方式

Flatten:

如果一个循环内嵌套一个循环，从外层循环进入内层循环会有一个时钟的延时，此时进行 Flatten 优化可以去除这个延时，提升运行速度

Merge:

通常两个循环之间要等前一个循环执行完成之后才能执行下一循环，此时如果将两个循环整合进一个循环只能可以降低整体的执行延时

Area

面积优化包含以下多种方式

DATA_Types and width:

HLS 中数据位宽可以灵活定义，需要根据所需要的位宽定义，位宽定义太大会增加资源消耗，定义太小容易出现数据溢出，同时当数据计算结果超过 8、16、32 位时，要注意避免出现 Integer promotion 的情况，比如两个 18 位的数相乘，定义积是 36 位，越过了 32 位，此时编译器会将两个 18 位的数转换成 32 位的数，变成两个 32 位的数相乘，产生一个 32 位的积，再把这 32 位的积转换成 36 位送给之前定义的 36 位的积，此时得到的计算结果并不是正确的。

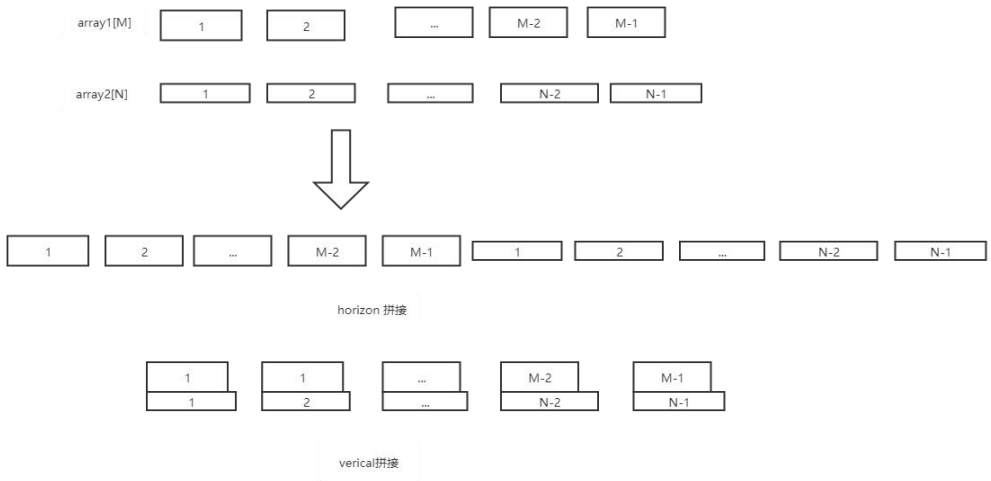
Function Inlining:

函数内联对资源消耗有很大的影响，函数内联指的是消除函数等级，当一个函数被多个函数调用，且被调用的函数处于同一等级时，此函数可以被复用，我们可以通过修改函数等级来达到函数服用的效果，内联方式分为三种，直接对函数设置 inline，这种内联是对上内联，也就是告诉调用它的函数该函数已经被是为内联，此时被设置为内联的函数无法被复用，调用多少次就会产生多少倍的资源消耗，对于多次被调用的函数建议设置 inline = off,region inline 这种内

联是对下内联，即此函数调用子函数时，子函数被视为内联，不需要经过一个时钟跳转到子函数，要注意 region inline 只能作用于等级比它低一级的子函数，子函数之内调用的函数不被视为内联，recursive inline 这种内联是对所有被它调用的函数，且被调用函数的子函数也被视为内联，也就是此时该函数之下没有比它等级更低的函数，有时对于 inline 方式设置不当可能会导致更大的面积消耗，要根据具体情况做合适的 inline 类型设置

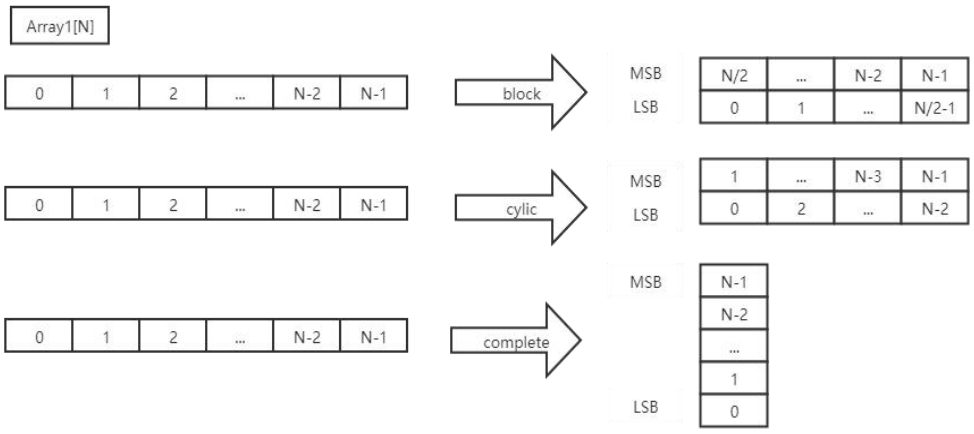
Map many arrays to one large array:

将多个小数组组合成较大的数组，由于每个数组都会被综合成 bram 或 fifo，太多的小数组会增大 bram 的资源消耗量，此时我们可以尝试将小数组拼接组合成较大的数组，减少 bram 消耗，拼接方式有 horizon 和 veritcal 模式两种，horizon 拼接之后数组大小是拼接的小数组之和，veritcal 模式是增大数据位宽，大小是原先较大的数组，效果如下图所示：



Array Reshape:

数组重组，效果类似于 array Partition,用 veritcal 的方式拼接数组达到对数组并行输出的效果，重组方式如图所示：



Controlling hardware resources:

控制逻辑资源，可以通过指定逻辑资源使用的总数要求 HLS 做该资源的复用，指定某种操作用何种逻辑资源实现对应操作，这个可以直接降低资源消耗，对于面积的优化帮助最大

LOGIC

逻辑优化通过以下两种方式实现

Controlling Operator Pipelining :

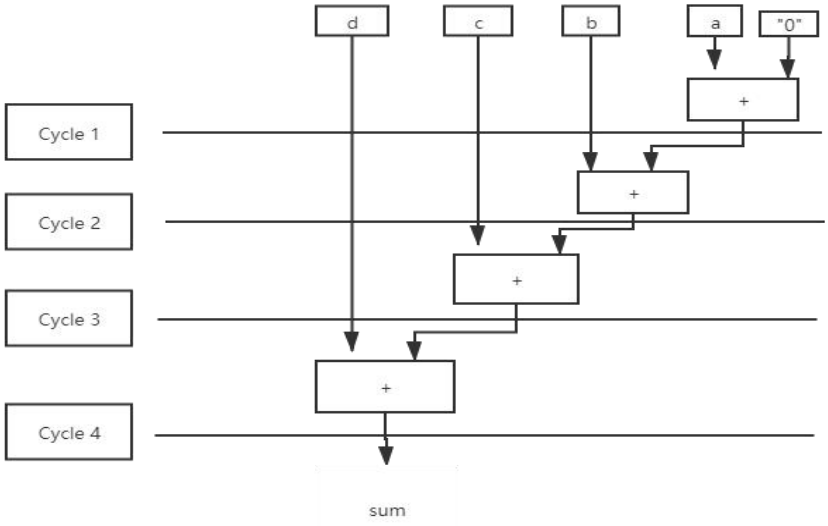
HLS 自动控制操作流水线等级，设置 latency 人为调整操作延时，通过牺牲时滞的方式提升时序，优化逻辑

Optimizing Logic Expression:

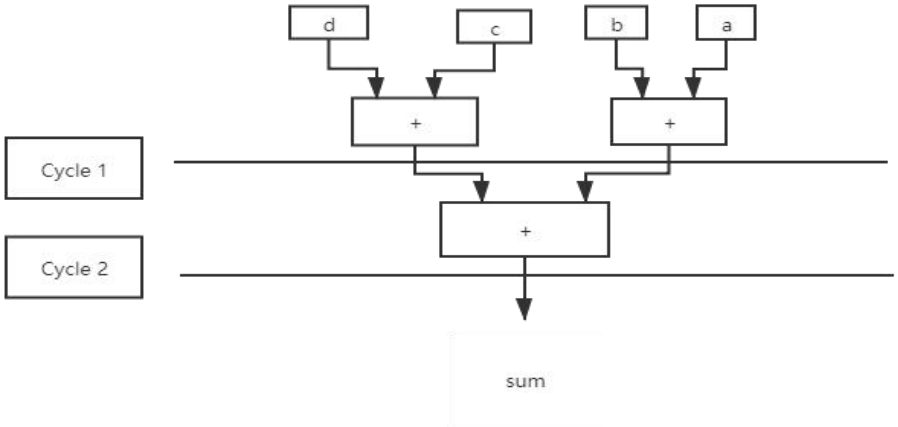
Expression balance 在默认状态下，定点运算时出于开启状态，浮点运算时处于关闭状态，可以手动调整，注意 Expression balance 开启状态时面积消耗会增大，比如一下这段代码

```
    data_t foo_top (data_t a, data_t b,
data_t c, data_t d)
    {
    data_t sum;
    sum = 0;
    sum += a;
    sum += b;
    sum += c;
    sum += d;
    return sum;
    }
```

设置 Expression balance=on 之前的综合方式是这样：

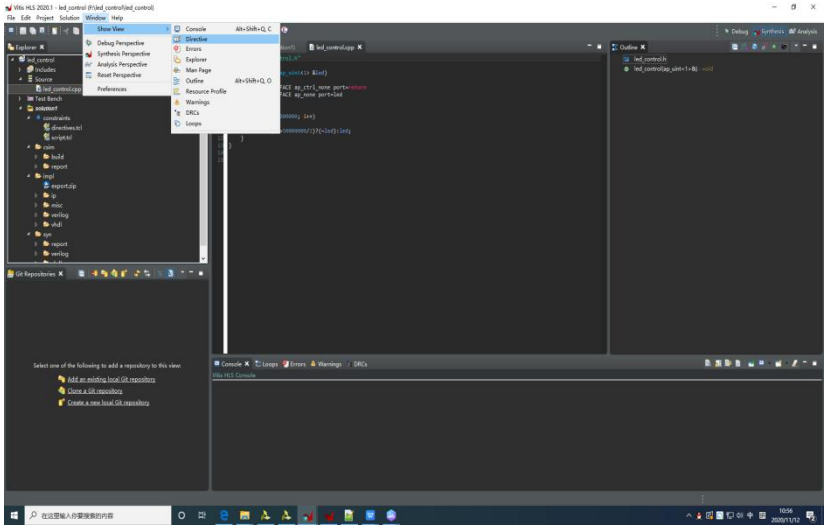


设置 expression balance=on 之后的综合方式是这样：

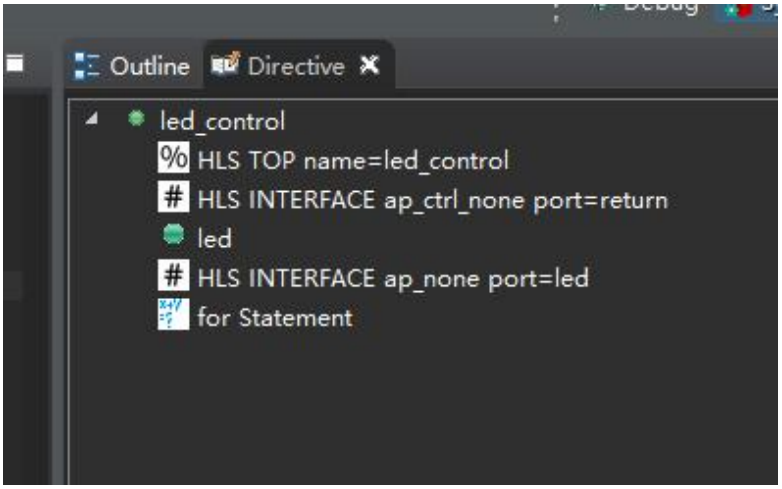


如何做接口综合和优化操作：

打开 window->show view->directive



在 directive 中双击对应信号进行接口综合或是优化



1.4 HLS 包含库

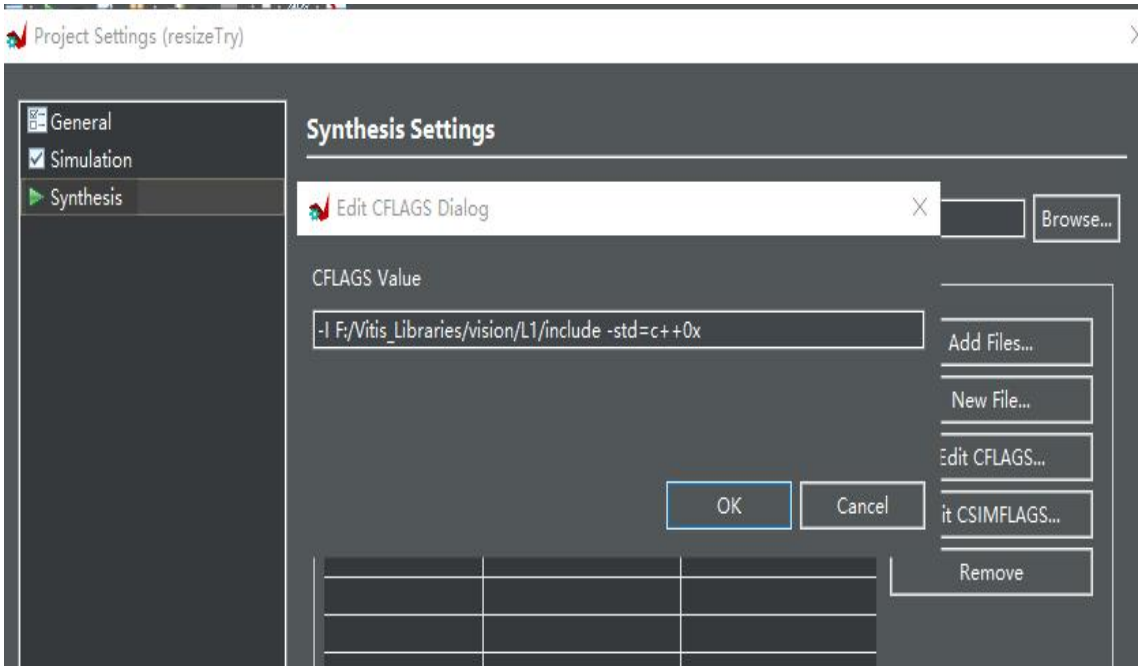
任意精度的数据类型	整数和定点 (ap_cint.h, ap_int.h and systemc.h)
HLS 流	流数据结构模型 旨在实现最佳性能和面积 (hls_stream.h)
HLS Math	广泛支持标准 C (math.h) 和 C++ (cmath.h) 数学库的综合。支持浮点和定点功能：abs, atan, atanf, atan2, atan2, ceil, ceilf, copysign, copysignf, cos, cosf, coshf, expf, fabs, fabsf, floorf, fmax, fmin, logf, fpclassify, isfinite, isinf, isnan, isnormal, log, log10, modf, modff, recip, recipf, round, rsqrt, rsqrtf, 1/sqrt, signbit, sin, sincos, sincosf, sinf, sinhf, sqrt, tan, tanf, trunc
HLS IP	集成 LogiCORE IP FFT 和 FIR Compiler (hls_fft.h, hls_fir.h, ap_shift_reg.h)
HLS 线性代数	cholesky, cholesky_inverse, matrix_multiply, qrf, qr_inverse, svd (hls_linear_algebra.h)
HLS DSP	atan2, awgn, cmpy, convolution_encoder, nco, qam_demod, qam_mod, sqrt, viterbi_decoder (hls_dsp.h)

注： Vitis HLS 的视频处理函数使用 xfopencv 的函数，在赛灵思官方 github 上下载 https://github.com/Xilinx/Vitis_Libraries

其中 xfopencv 的图像处理函数在 L1 的 include 文件夹里面，下载完成之后把其中的 xf_infra.hpp 文件用教程中提供的同名文件替换掉

使用 xfopencv 库文件之前要设置文件路径

点击图标  选择 synthesis->edit cflags，在其中输入：-I <include-path> -std=c++0x 如图所示：



其中 F:/Vitis_Libraries/vision/L1/include 是在下之前下载的路径，要根据下载的地址替换对应路径，之前版本的 hls_video 函数在和 hls_opencv 函数在 vivado HLS 2020.1 和 vitis HLS 2020.1 版本中已经放弃支持,这点要特别注意。

1.5 HLS 官方资料

1) *Introduction to FPGA Design with Vivado High-Level Synthesis* (UG998)
2) *Vivado Design Suite Tutorial: High-Level Synthesis* (UG871)
3) *Vivado Design Suite User Guide: High-Level Synthesis* (UG902)
4) *Xilinx Opencv User Guide* (UG1233)
5) *Vitis HLS Migration Guide* (UG1391)

第二章 入门

2.1 实验介绍

我们和 HLS 的故事从 LED 灯闪烁开始，就如无数高手在开始书写他们传奇之前的第一句“Hellow World”，这是一个比较简单的程序，通过 HLS 实现 LED 灯闪烁并且观察现象，这期间要注意有时候 HLS 综合我们所写的代码会和预期有所出入，不能简单的把它当成 C++来写，对于学习 HLS 的人来说最好对 FPGA 的工作方式和原理有一个初始的了解之后再结合 C++的知识来编写 HLS 的代码，实现 HLS 的开发

2.2 实验源码

```
#include <ap_int.h>

void led_control(ap_uint<1> &led)
{
#pragma HLS INTERFACE ap_ctrl_none port=return
#pragma HLS INTERFACE ap_none port=led
unsigned int i;
  led=0;
  for(i=0; i<50000000; i++)
  {
#pragma HLS PIPELINE
       led = (i<50000000/2)?(~led):led;
  }
}
```

2.3 接口设置和优化

接口设置：

这里 Block level Interface 设置为 ap_none 类型去除多余的控制信号，参数设置为 ap_none 类型

综合优化：

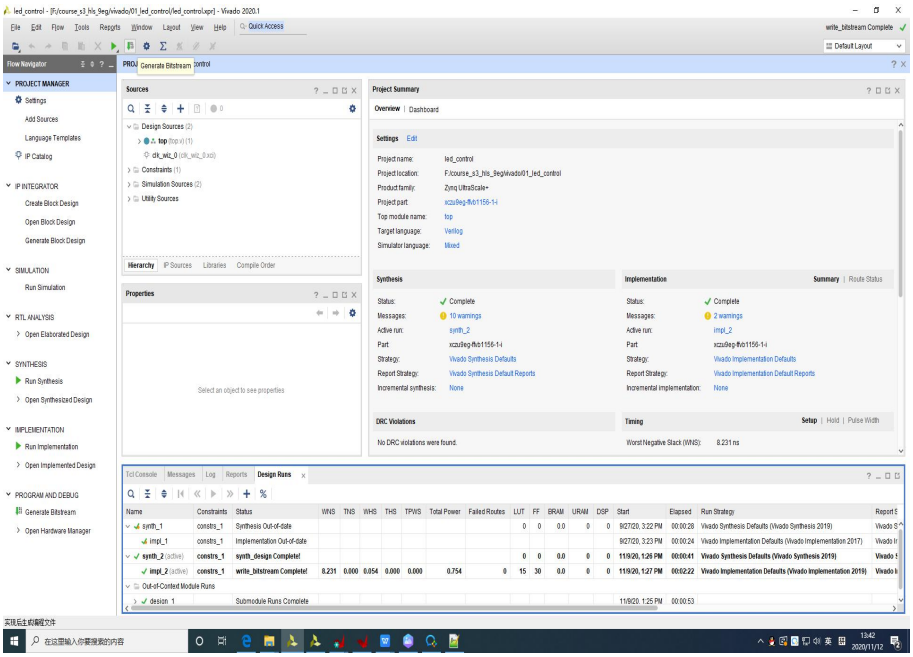
对循环内部使用 Pipeline 优化，降低延时提升运行速度

2.4 工程路径

名称	路径
Vivado 工程	vivado/led_control
HLS 工程	hls/led_control
bit 文件	/vivado/led_control/led_control.runs/impl_1/top.bit

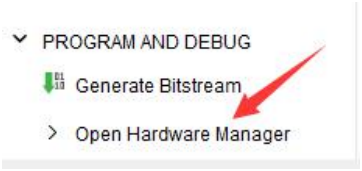
2.5 工程实现

我们重点说一说工程实现，这个入门的工程需要的时钟直接使用开发板上的时钟，根据开发板上的时钟类型用对应的时钟原语 IP 核驱动我们的 HLS 核实现功能。
首先我们按照前一章的步骤把 IP 核添加到 vivado 工程中，生成 system wrapper,并创建顶层文件之后点击 generate bitstream 生成比特流，这个过程可能需要几分钟时间，请耐心等待，如果超过一个小时还没完成，建议换台新电脑



接下来我们就可以加载刚生成的 bit 文件至板卡，先将板卡上电，并确定连接好 JTAG。

1) 点击“Open Hardware Manager”

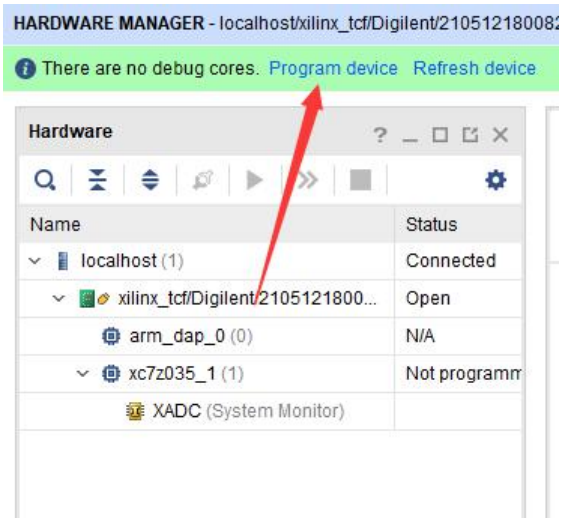


2) 点击“Open Target”→”Auto Connect”

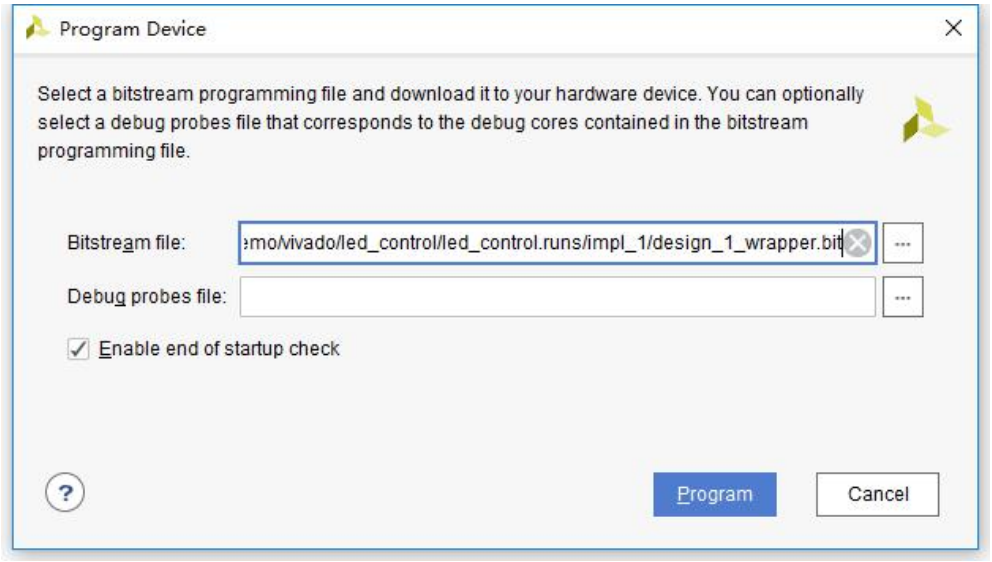




3) 点击”Program device“



4) 点击“Program“



5) 完成后，即可看到 led 灯以 1 秒的频率闪烁，至此，整个实验完成。

第三章 初识 HLS 与 CPU 寄存器交互

3.1 实验介绍

这章的实验现象和上一章类似，区别在于这一章我们通过 CPU 寄存器来控制灯的闪烁时间，

除了使用 Vivado 之外还需要使用 Vitis 辅助，时钟通过 PS 端生成，在 PS 端要运行一个程序，当 PS 端运行之后才能生成电路的时钟。

3.2 实验源码

```
#include <ap_int.h>

void led_register(ap_int<1> &led, int total_cnt, int high_cnt)
{
#pragma HLS INTERFACE ap_none port=led
#pragma HLS INTERFACE ap_none port=total_cnt
#pragma HLS INTERFACE ap_none port=high_cnt
#pragma HLS INTERFACE ap_ctrl_none port=return
  led=0;
  unsigned int i=0;
  for(i=0;i<total_cnt;i++)
  {
#pragma HLS LOOP_TRIPCOUNT max=50000000 min=50000000
        led = (i==high_cnt)?(~led):led;
  }
}
```

3.3 接口设置和优化

接口设置:

Block level Interface 设置为 ap_ctrl_none 类型，因为这里不需要额外控制信号，其他实验中也通过这种方式设置 Block level Interface 所有输入参数设置为 ap_none,去除多余的关联信号

综合优化:

因为循环次数是变量，需要设置 TRIPCOUNT 否则无法得到准确的运行时间和延时预估，当循环次数是变量时大部分时候都要做这类优化。

3.4 工程路径

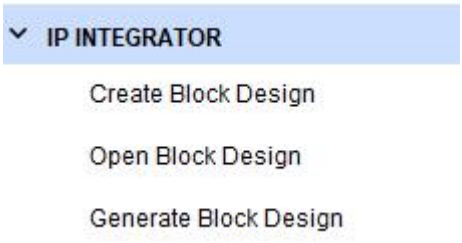
名称	路径
Vivado 工程	vivado/led_control
HLS 工程	hls/led_control
bit 文件	/vivado/led_control/led_control.runs/impl_1/design_1_wrapper.bit

3.5 工程实现

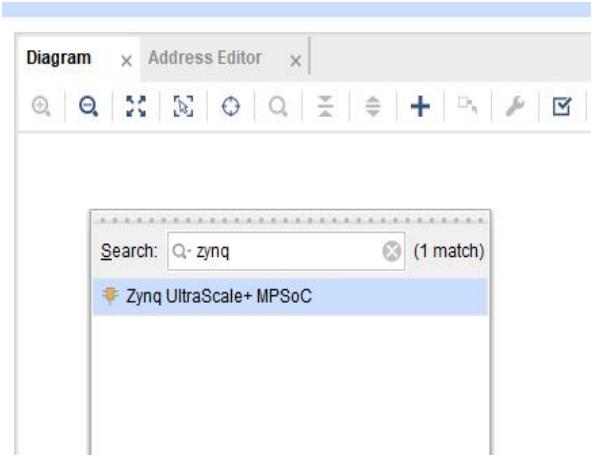
我们还是重点说一下工程实现，工程实现按照如下步骤操作

1) 按照之前章节介绍的步骤将 IP 核添加到工程中

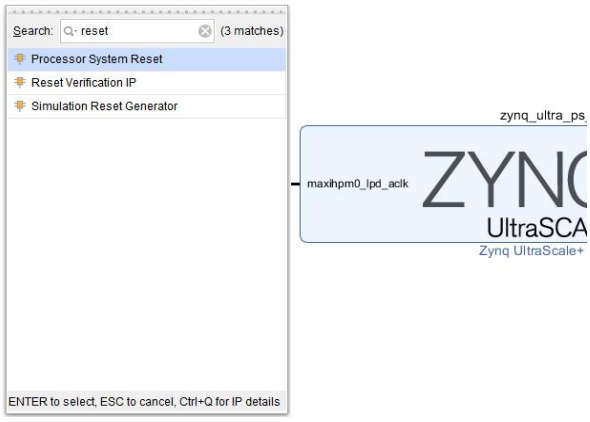
2) 左侧选择 Create Block Design



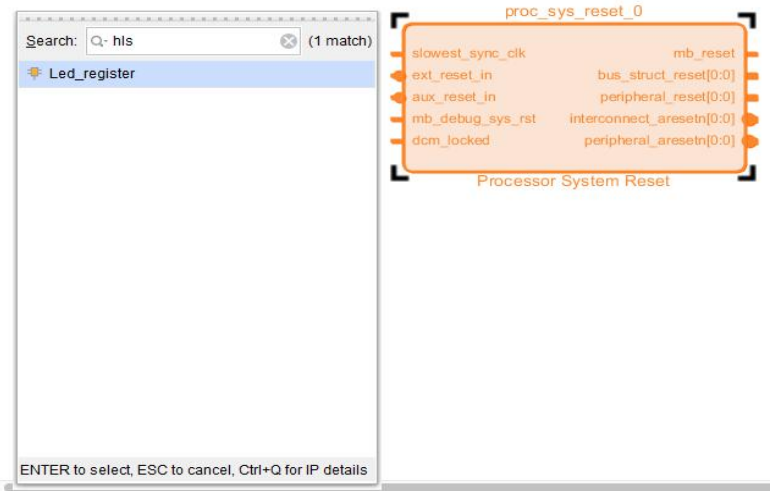
3) 在界面中单击加号输入 zynq，按照 course_s2 中 Hellow world 那章的方式配置 IP 核



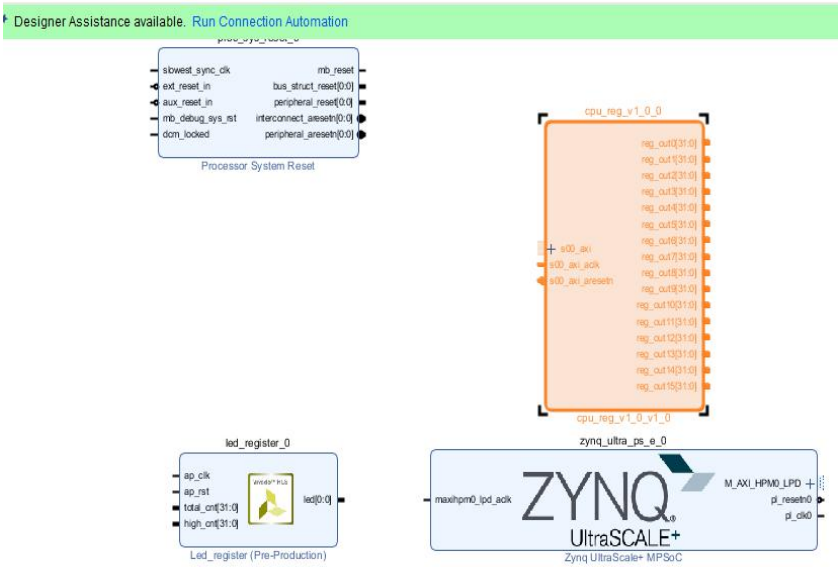
4）在界面中单击加号输入 reset，添加复位 IP 核



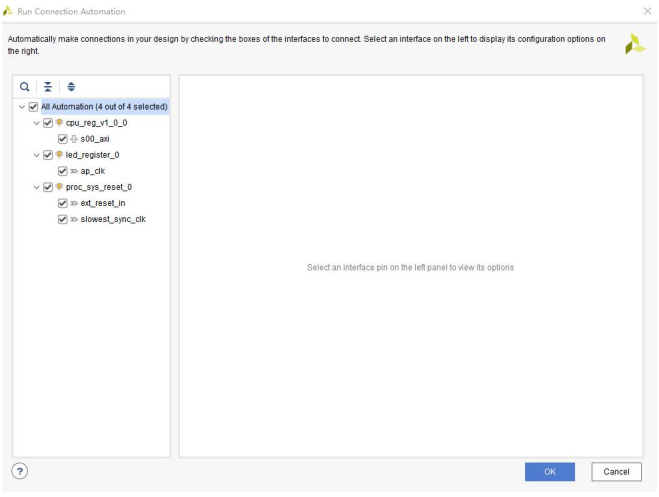
5）在界面中单击加号输入 hls,添加 HLS 生成的 IP 核



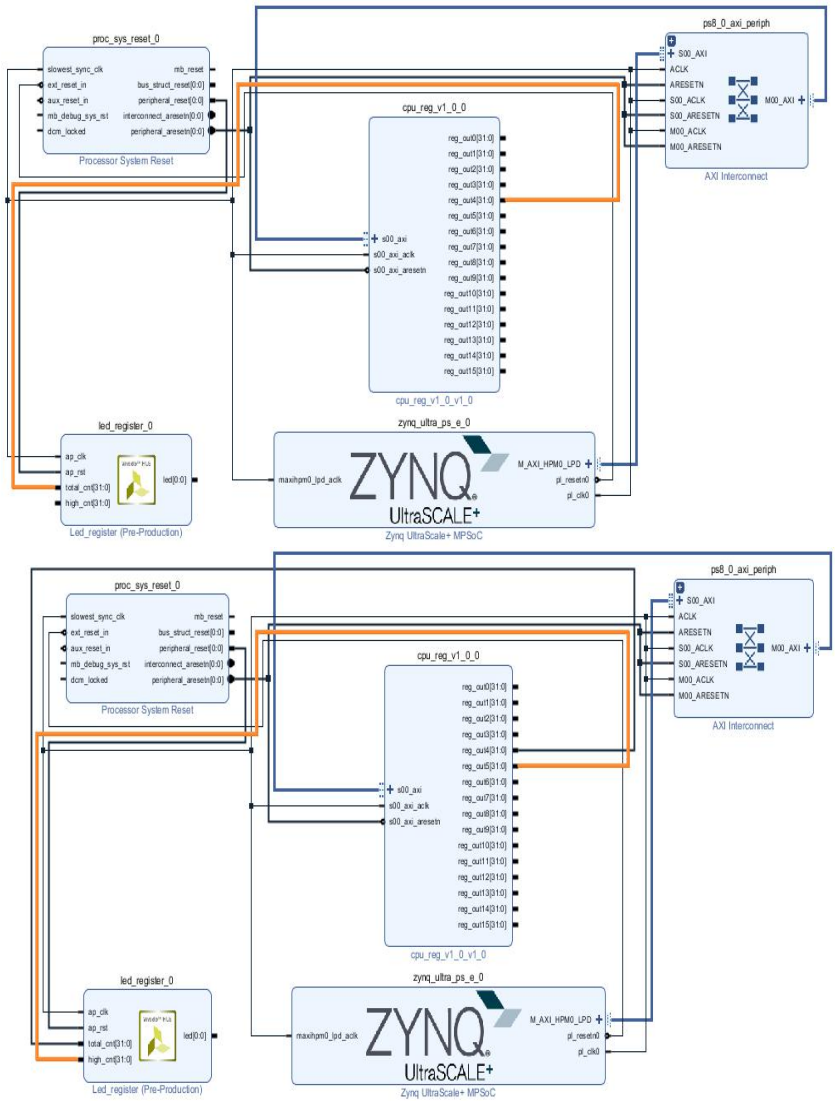
6）在界面中继续单击加号输入 CPU_reg 添加我们生成的 CPU 寄存器，之后单机 run auto connection



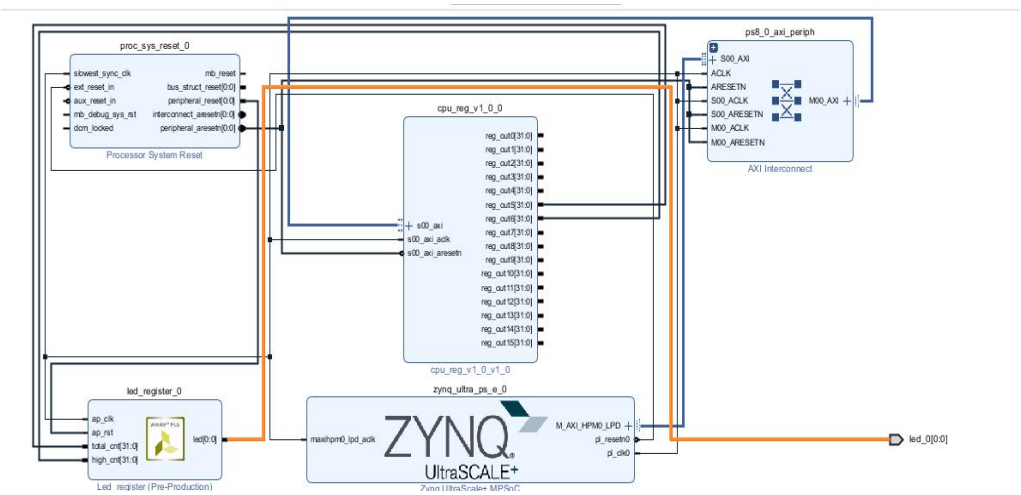
7）所有全部勾选



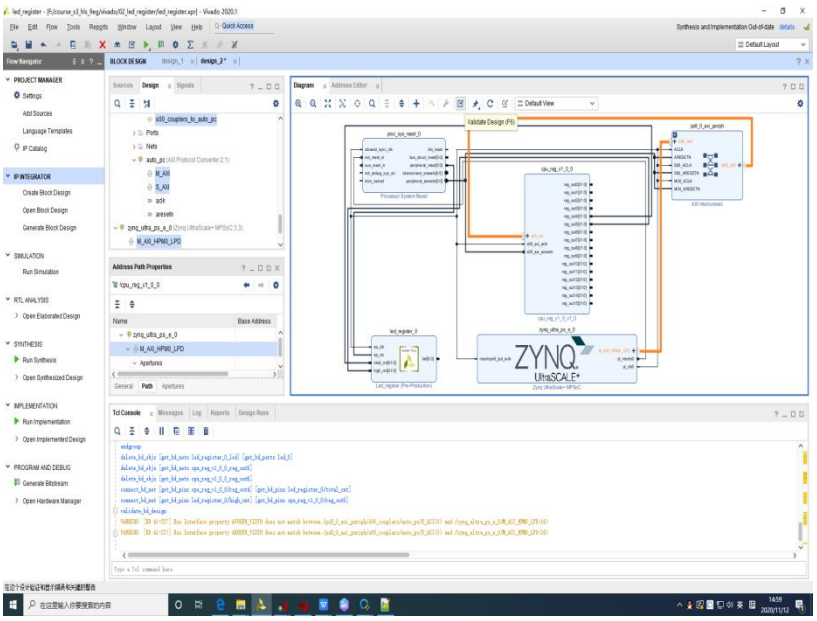
8）单击 ok 之后按照如下方式连接 IP 核

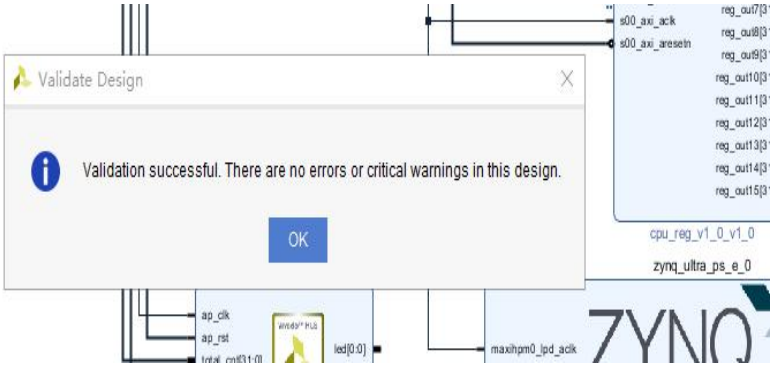


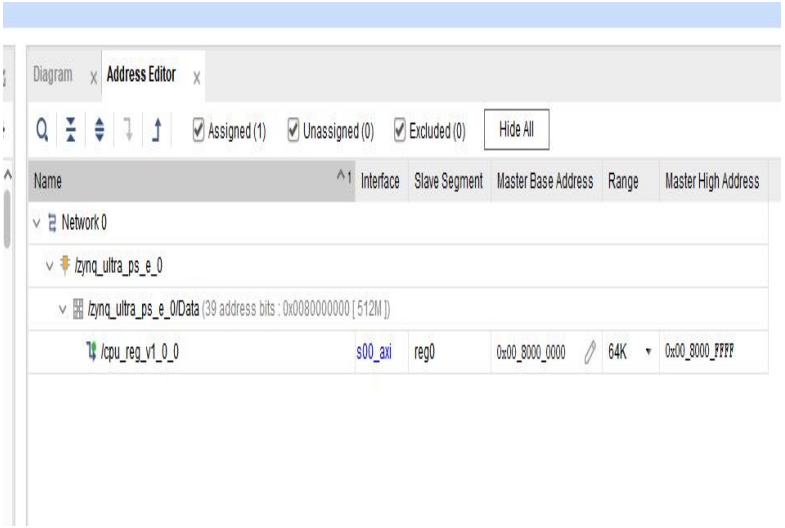
9）在 led_register_0 右边的输出端口单击 ctrl+T 生成输出端口



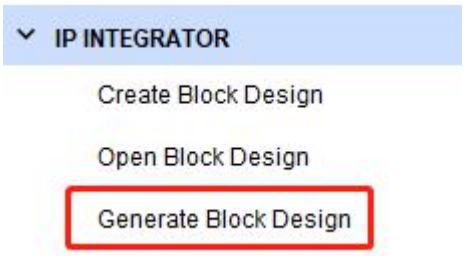
10）单击 F6,确认接线没有错误之后，为 IP 核分配地址



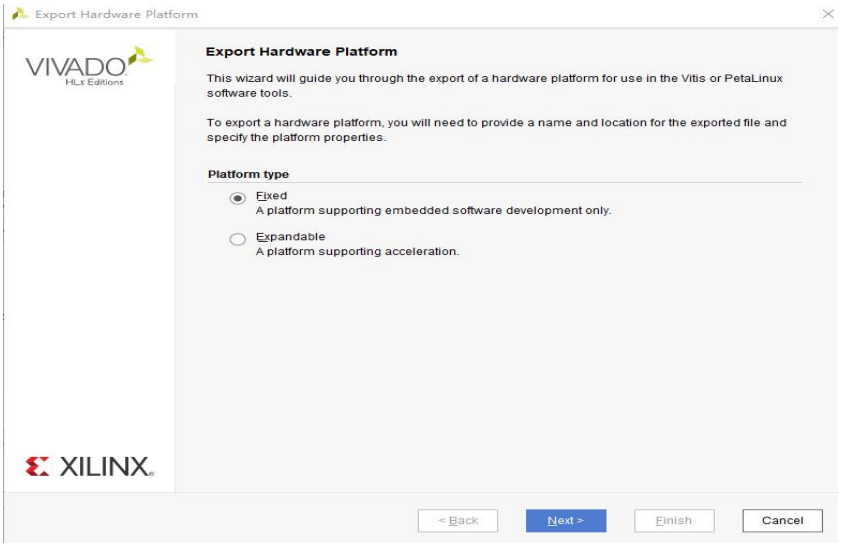




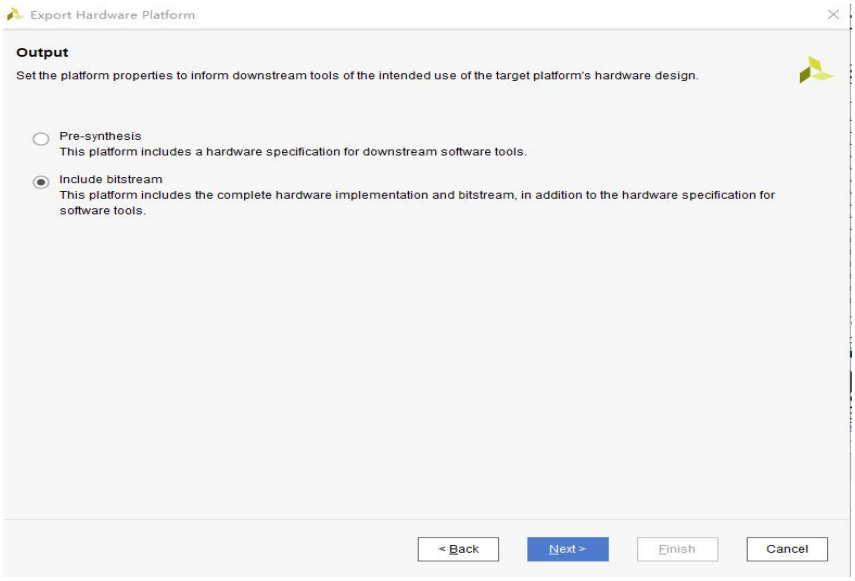
11）完成之后单击 generate block design



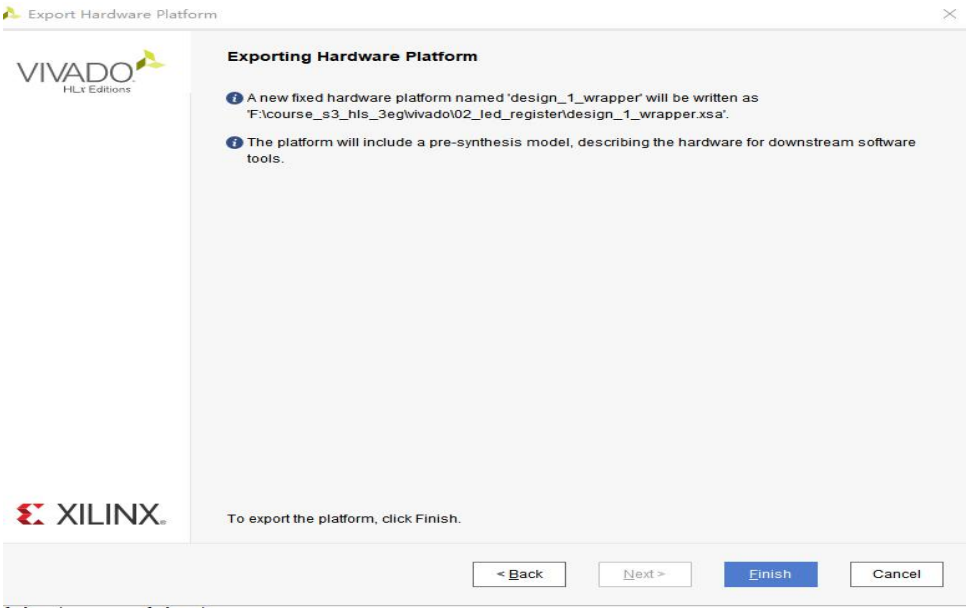
12) 生成比特流文件，完成之后左上角 Files->Export->Export Hardware,选择 fixed



13) Next->Next ,选择 include bitstream

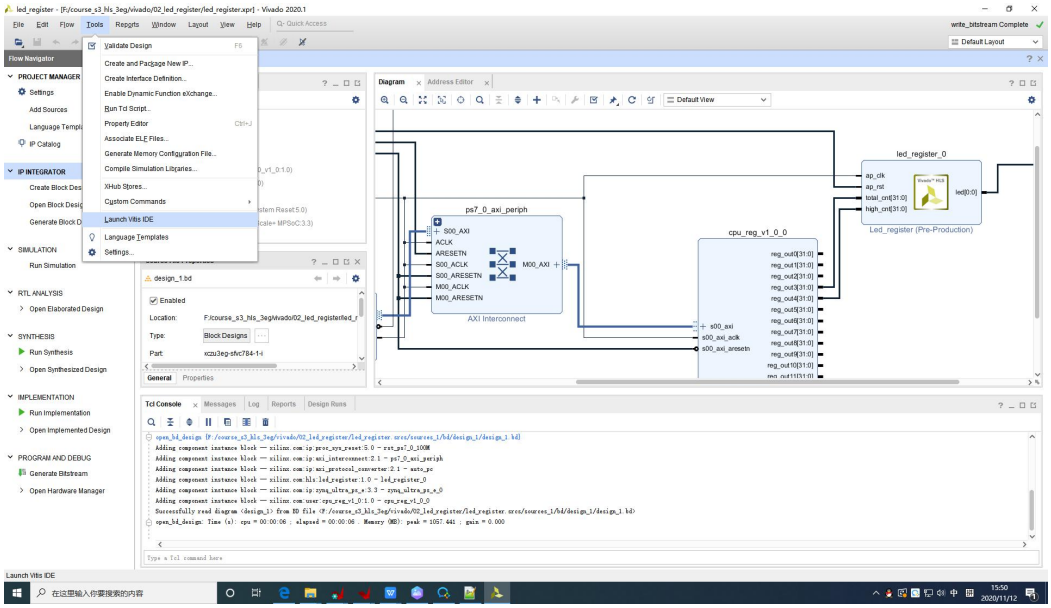


14) Next->Next ,点击 finish

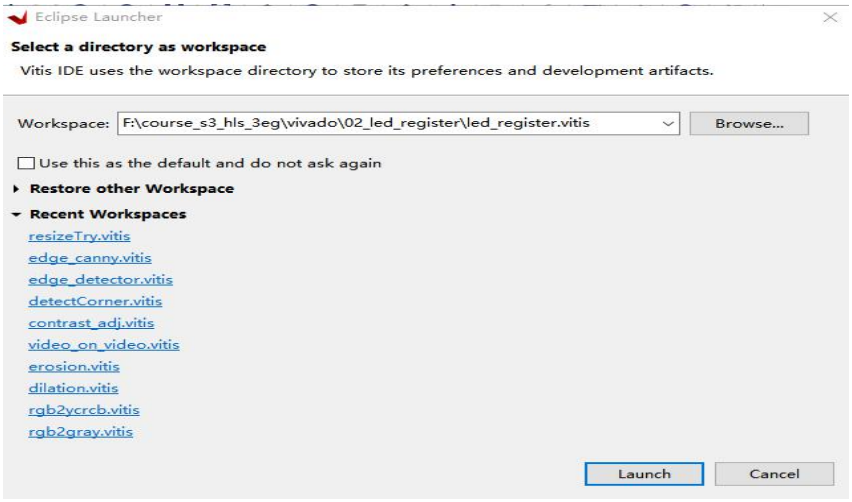


15) 在工程中新建文件夹 led_register.vitis

16) Tools->launch vitis

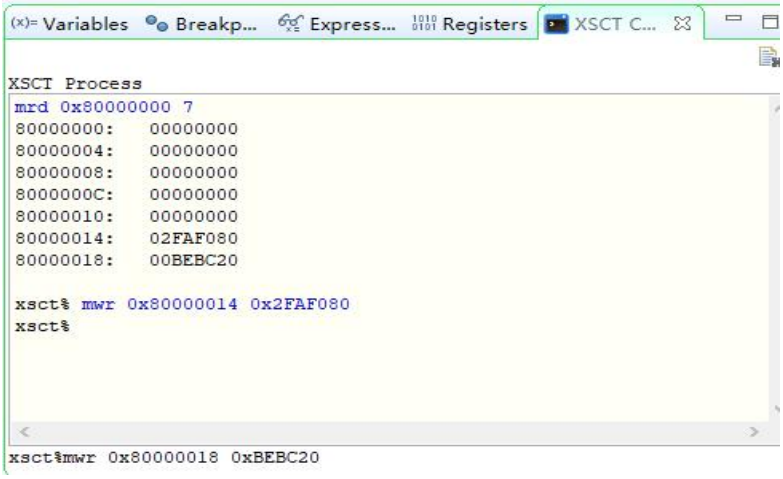


17) Workspace 选择创建的文件夹



18) 之后按照 course_s2 中操作，将比特流烧录进板卡

19) 烧录完成之后，单击 debug,在 XSTC console 输入以下命令



注：如果我们想要往寄存器中读取数据往命令行里输入：mrd+空格+地址+读取个数

如果想要从寄存器中写入数据往命令行中输入：mwr+空格+地址+写入数据值

20) 可以看到 led 灯按照设置的闪烁周期和时间持续闪烁

第四章 如何使用 xfopencv 库中自带的函数

4.1 实验介绍

这章实验我们主要实现的是图像分辨率的调整，通过对图像做双线性插值法实现，赛灵思官方 xfopencv 库带有该函数，我们主要做的是一个应用型的实验，通过这个实验熟悉如何使用赛灵思官方 xfopencv 库中的内置函数实现我们的需求，读者有机会可以对赛灵思库中常见的函数进行实践，熟悉其中的代码，有些函数可能烧录之后运行效果不太理想，此时需要结合自身实际需求进行一些修改，后续几章的实验中包含了几个直接使用 xfopencv 库中函数发现效果不理想进行修改的例程，有些是延时过大，有些是显示黑屏或彩条，这些函数在下经过修改放在了工程中的 source 文件夹内，可以直接使用。

4.2 实验源码

顶层函数参数定义

```
#ifndef __resizeTry_h__
#define __resizeTry_h__

#include "ap_int.h"
#include "hls_stream.h"
#include "ap_axi_sdata.h"

#include "common/xf_common.hpp"
#include "common/xf_infra.hpp"
#include "common/xf_utility.hpp"

#include "xf_resize_nn_bilinear.hpp"

#define IMG_MAX_ROWS 1080
#define IMG_MAX_COLS 1920

void resizeTry(hls::stream<ap_axiu<24,1,1,1>>&src,hls::stream<ap_axiu<24,1,1,1>>&dst);

#endif
```

源文件顶层函数部分：

```
#include "resizeTry.h"

void resizeTry(hls::stream<ap_axiu<24,1,1,1>>&src,hls::stream<ap_axiu<24,1,1,1>>&dst)
{
#pragma HLS DATAFLOW
#pragma HLS INTERFACE axis port=dst register_mode=both register
#pragma HLS INTERFACE axis port=src register_mode=both register
#pragma HLS INTERFACE ap_ctrl_none port=return

  xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS,IMG_MAX_COLS,XF_NPPC1>img_src;
#pragma HLS STREAM variable=img_src.data depth=1920 dim=1
  xf::cv::Mat<XF_8UC3,640,640,XF_NPPC1>img_dst;
#pragma HLS STREAM variable=img_dst.data depth=1920 dim=1

  xf::cv::AXIvideo2xfMat(src,img_src);
  xf::cv::resizeNNBilinear<XF_8UC3,IMG_MAX_ROWS,IMG_MAX_COLS,XF_NPPC1,640,640,2,2>(img_src,img_dst);
  xf::cv::xfMat2AXIvideo(img_dst,dst);
}
```

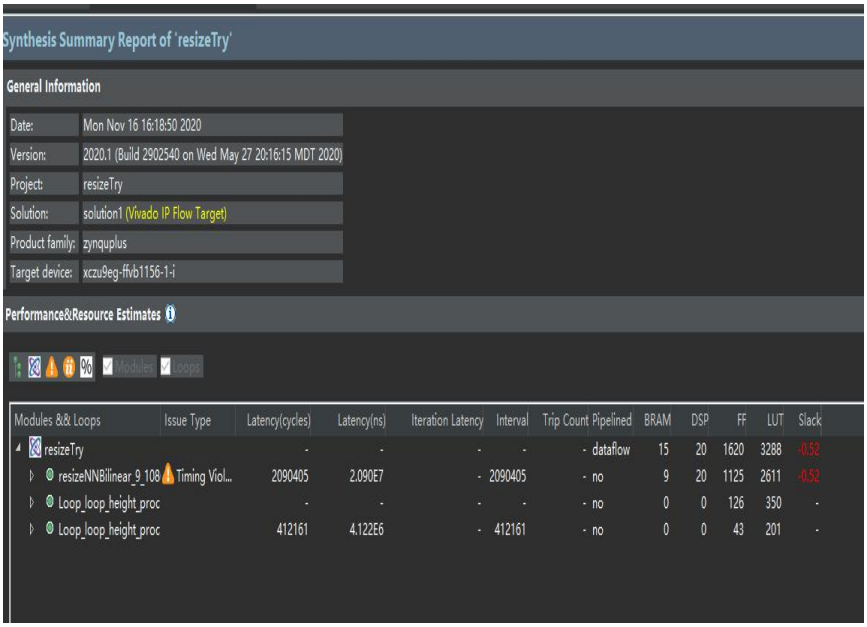
4.3 接口综合与工程优化

接口综合：

Block level Interface 设置为 ap_ctrl_none 去除多余的关联信号，输入输出因为是连续变换的数据设置为 axi 类型，连接的时候可以直接和之前的对应 IP 核相连

工程优化：

主函数为了降低处理延时使用 Dataflow 优化，整体函数运行时间在 20ms 左右，时序方面如果将时钟周期设置为 10ns,该函数在 vitis HLS 的综合中可能出现违规，但整体以 vivado 实现之后的报告为准，如果 vivado 未出现时序违规，则实际不存在时序违规，可以设置为 10ns，可以放心设置。



4.4 工程路径

名称	路径
Vivado 工程	vivado/rgb2gray
HLS 工程	hls/ rgb2gray

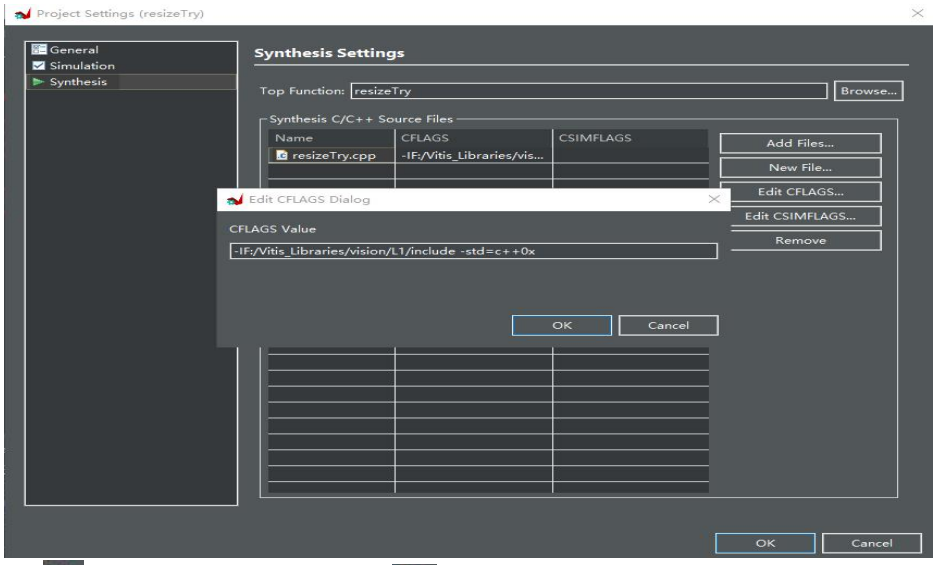
4.5 工程实现

我们重点聊一聊工程实现，连接方式和 IP 核设置参照 course_s2 中 mipi 摄像头显示的工程设置，在 HLS 中要对数据路径进行设置，否则会出现找不到头文件的报错，按照以下步骤进行操作

1）按照一章入门工程所说创建 HLS 工程，命名为 resizeTry

2）在源文件和头文件中输入实验源码中的代码

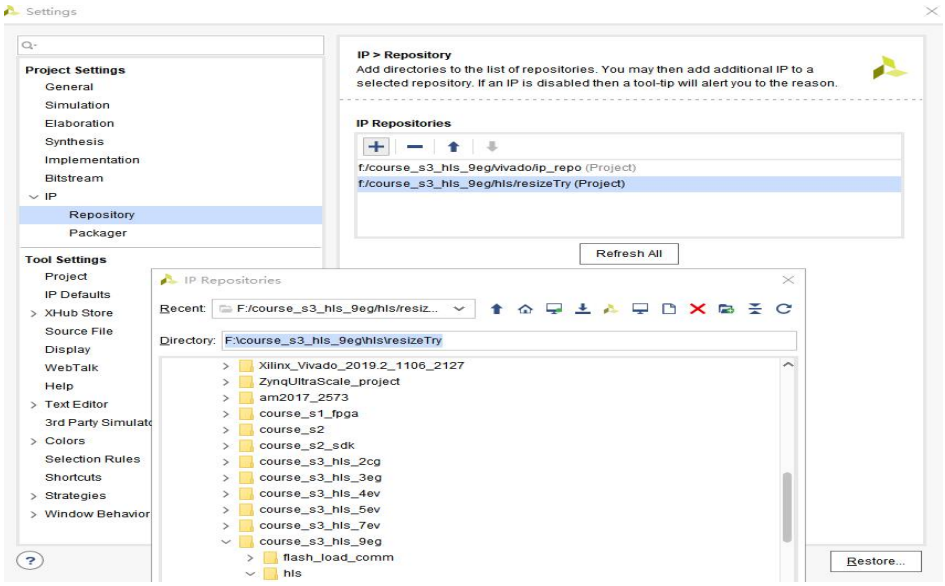
3）输入完成之后保存，单击  选择顶层函数，选择 edit cflags，输入-I <get-path> -std=c++0x



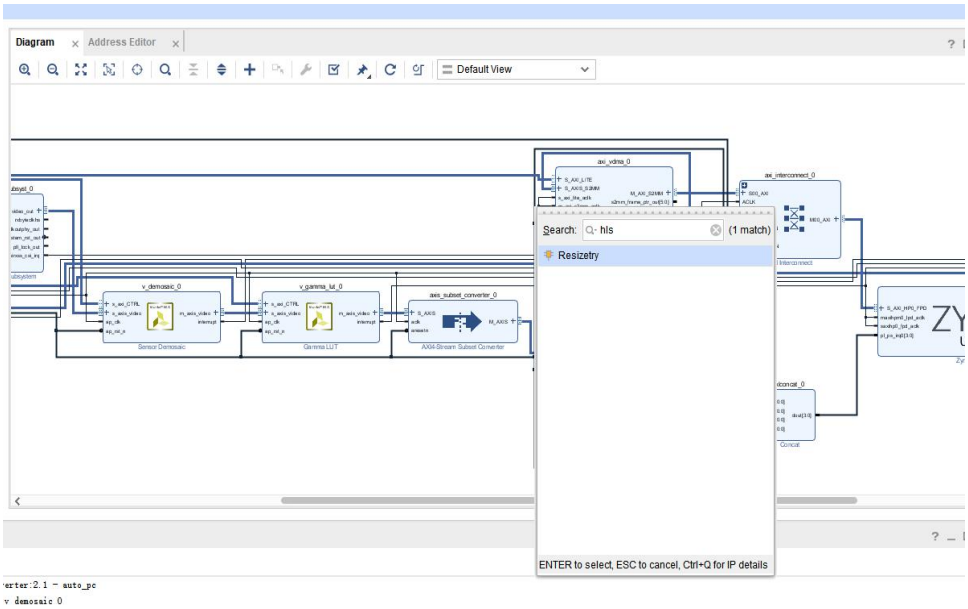
4）单击  进行综合，综合之后单击  导出 IP 核

5）打开 Vivado，按照 course_s2 中 mipi 工程的连接方式和 IP 核设置方操作

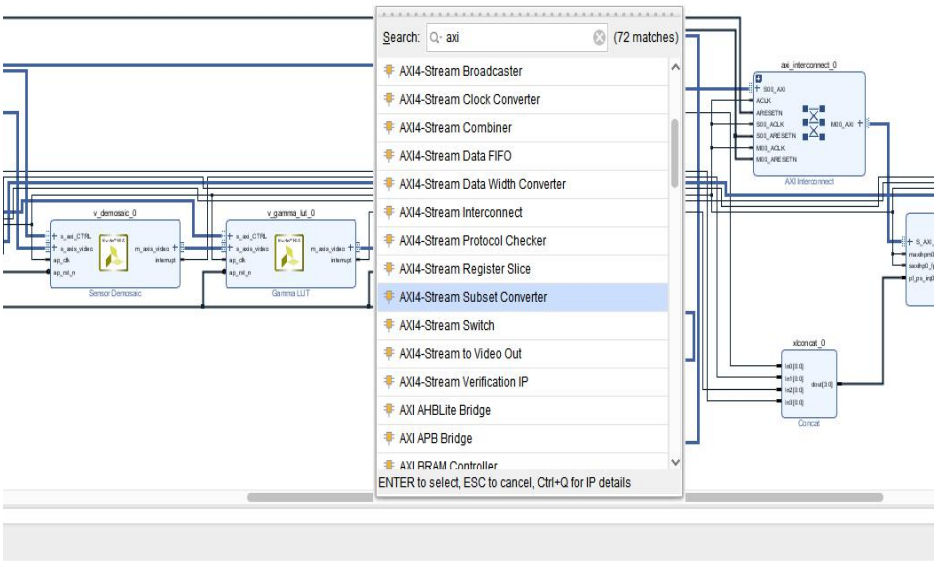
6）单击  选择 IP->repository,添加 IP 核路径



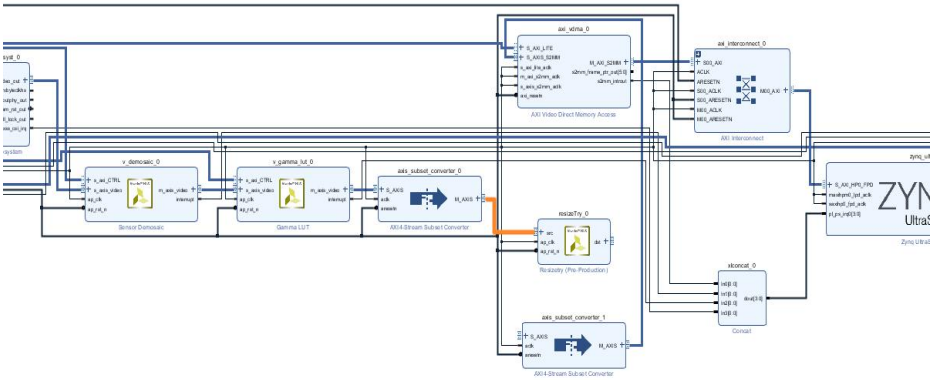
7）单击确定，在 block design 中右键添加 IP 核，输入 HLS

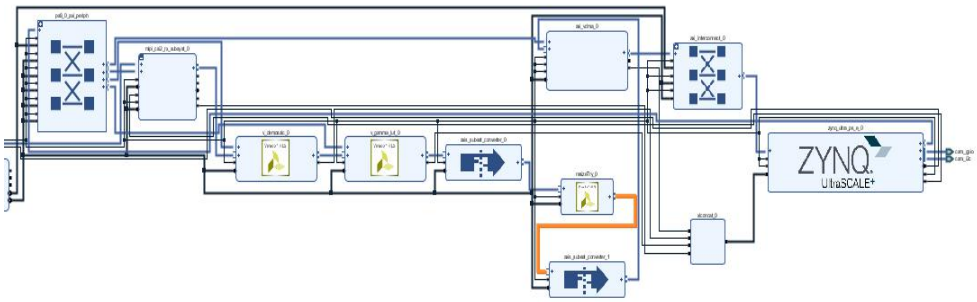


8) 在 block design 中添加 IP 核，输入 axi,选择 axi_subset_convert



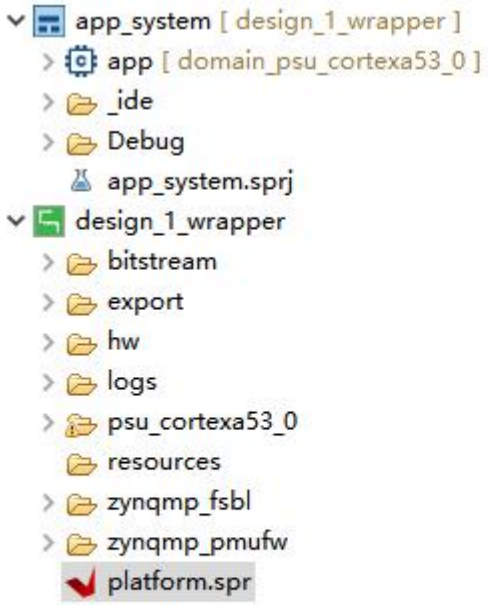
9) 按如图方式连接并保存，之后生成比特流



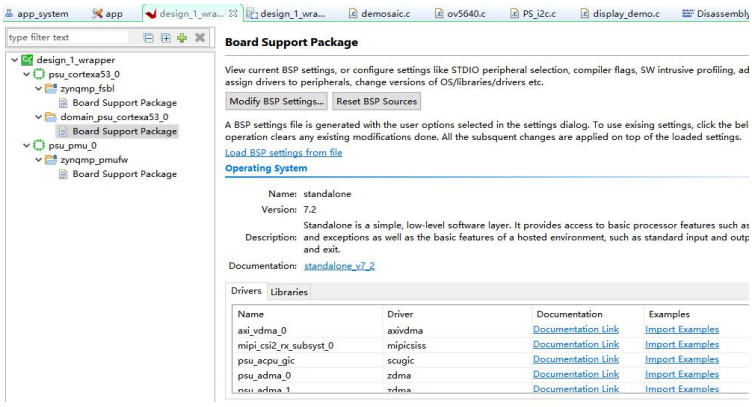


9）按第二章步骤生成 vitis 工程

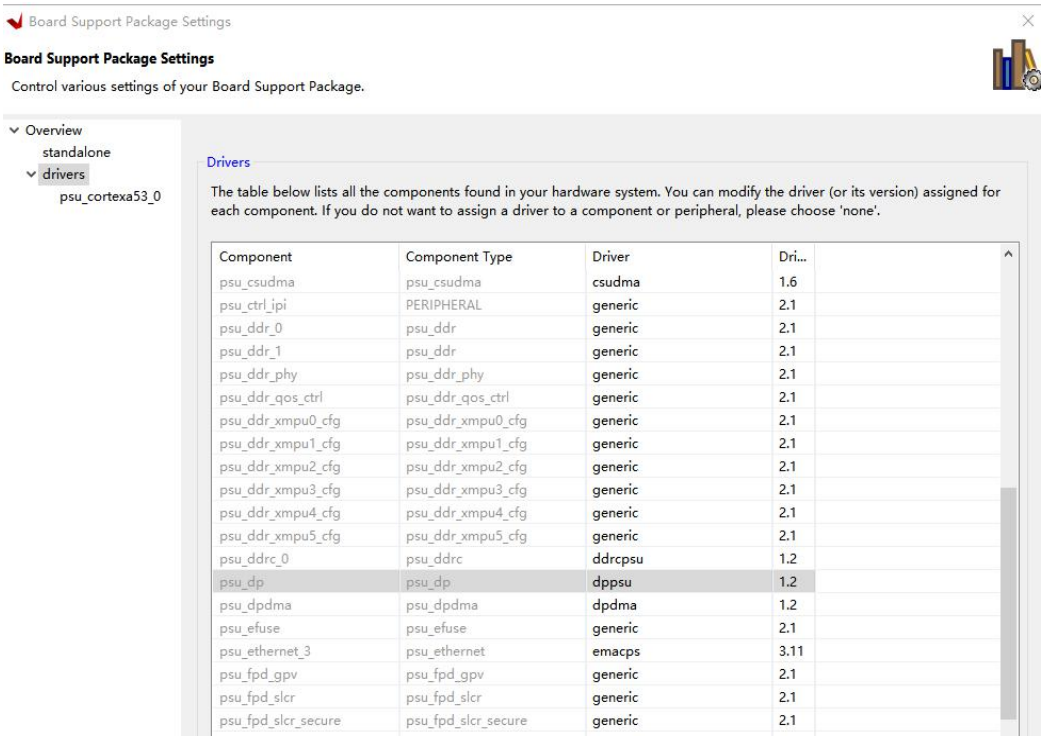
10）双击 platform.spr 修改 dp 驱动



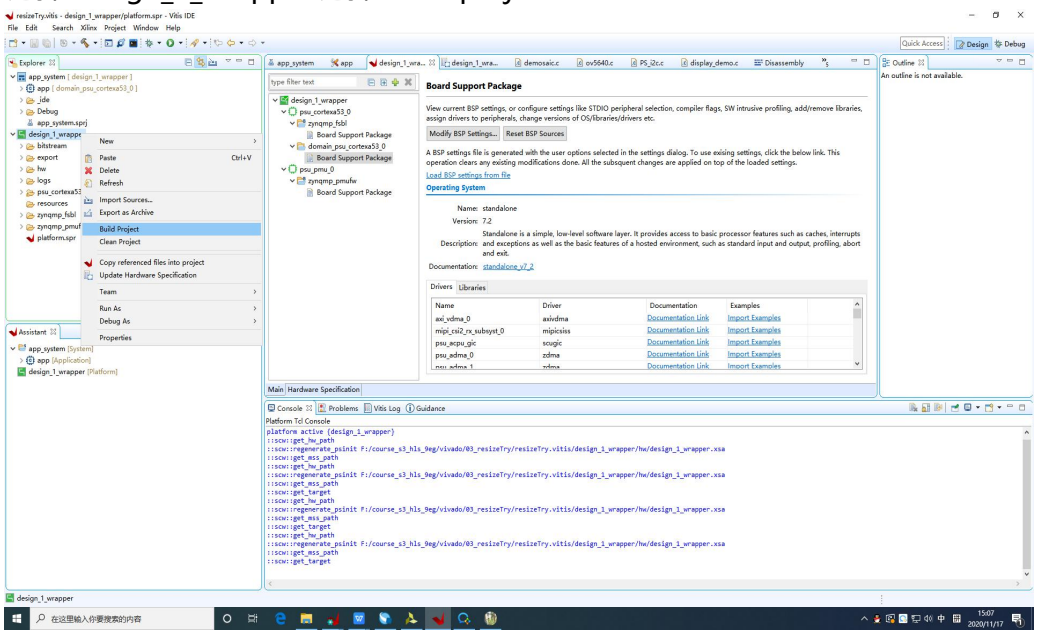
11）选择合适的 bsp 设置



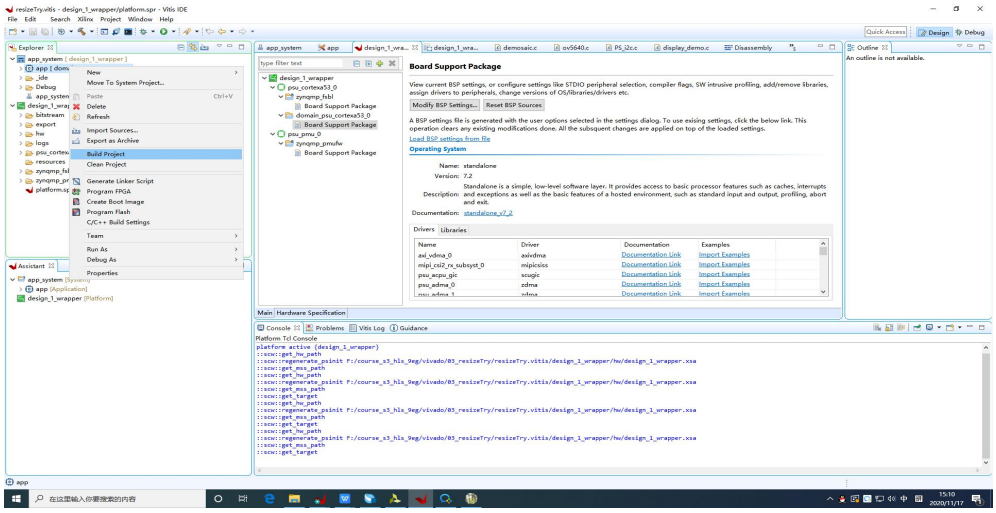
12）找到 psu_dp 将驱动修改为 dppsu



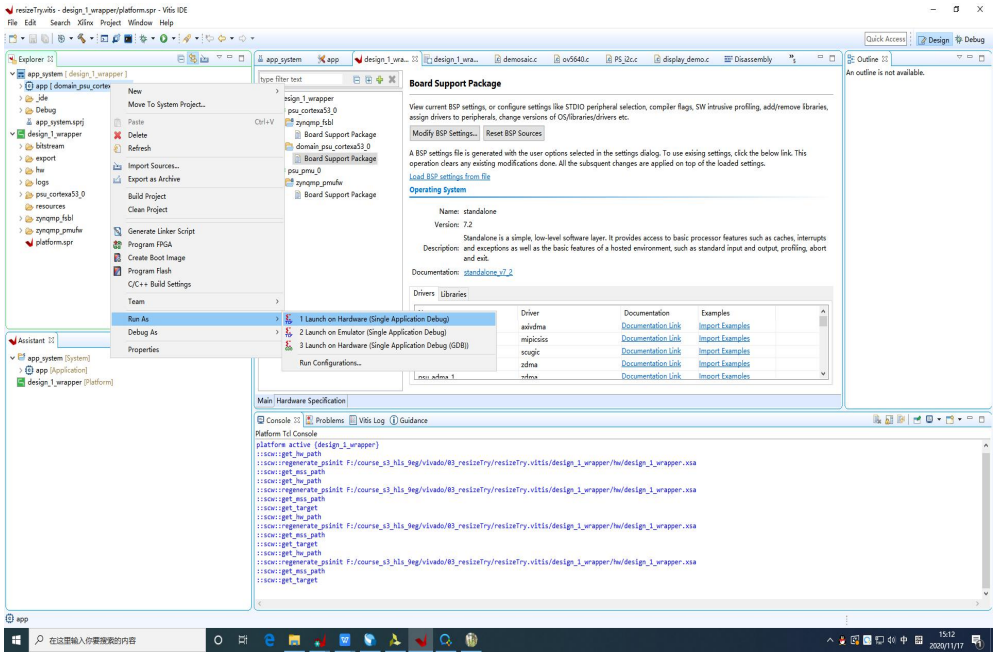
13）选择 design_1_wrapper 选择 build project



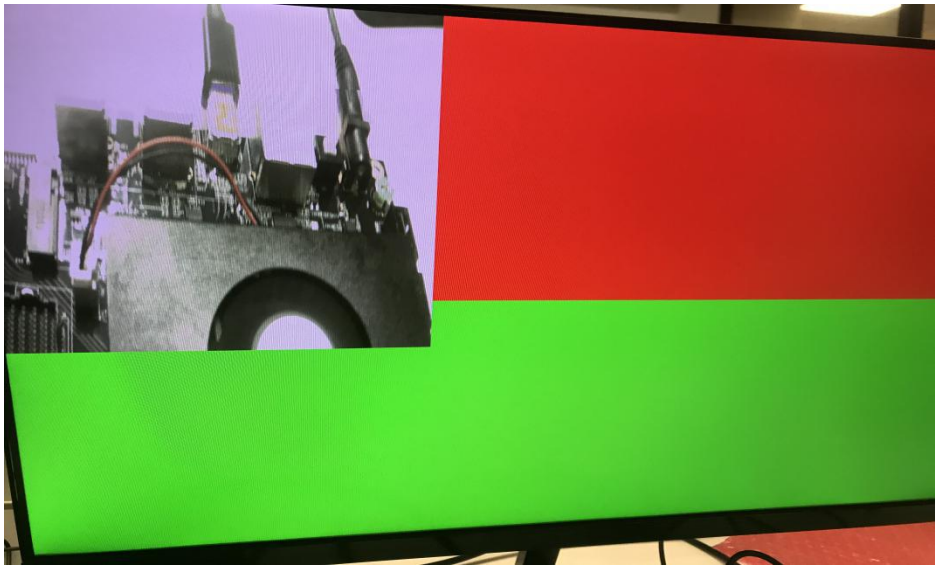
15）完成之后选择创建的工程选择 build project



16) 完成之后选择创建的工程选择 run as



17) 烧录过几分钟之后能看到屏幕上显示如下图像



第五章 图像 RGB 转灰度变换

5.1 实验介绍

RGB 图像是三通道的，有时做图像处理算法时为节省资源将输入三个通道信息转换成单个通道信息，该通道信息要包含原先的三个通道信息，最常见的做法是做 RGB 到灰度的转换，转换方式按照如下公式实现 $Y = 0.299 * R + 0.587 * G + 0.114 * B$,转换之后图像会变成灰白色，灰色信息为 8 位，输入 24 位数据高八位包含 B 信息，中间八位包含 G 信息，低八位包含 R 信息，先做了提取并使用函数 CalculateGray 做了灰度转换，之后将转换到的灰度信息拼接成 24 位，并将这 24 位的图像信息输出，显示出来是灰色。

5.2 实验源码

头文件参数定义

```
#ifndef __rgb2gray_h__
#define __rgb2gray_h__

#include "hls_stream.h"
#include "ap_int.h"
#include "ap_axi_sdata.h"

#include "common/xf_common.hpp"
#include "common/xf_utility.hpp"
#include "common/xf_infra.hpp"

#include "imgproc/xf_cvt_color.hpp"

#define IMG_MAX_ROWS 1080
#define IMG_MAX_COLS 1920

Void rgb2gray(hls::stream<ap_axiu<24,1,1,1>>&video_in,hls::stream<ap_axiu<24,1,1,1>>&video_out);
#endif
```

源文件通道提取部分

```
#include "rgb2gray.h"

void ExtractPixel(XF_TNAME(XF_8UC3,XF_NPPC1)&src,ap_uint<8>dst[3])
{
  unsigned int i,j=0;
  for(i=0;i<24;i+=8)
  {
#pragma HLS UNROLL
        dst[j]=src.range(i+7,i);
        j++;
  }
}

template<int ROWS,int COLS>
void xfrgb2gray(xf::cv::Mat<XF_8UC3,ROWS,COLS,XF_NPPC1>&src,xf::cv::Mat<XF_8UC1,ROWS,COLS,XF_NPPC1>&dst)
{
  XF_TNAME(XF_8UC3,XF_NPPC1)rgb_packed;
  XF_TNAME(XF_8UC1,XF_NPPC1)gray_packed;
  ap_uint<8>rgb[3];
  ap_uint<8>gray;
  unsigned int i,j=0;
  for(i=0;i<ROWS;i++)
  {
        for(j=0;j<COLS;j++)
        {
#pragma HLS PIPELINE
              rgb_packed=src.read(i*COLS+j);
              ExtractPixel(rgb_packed,rgb);
              gray=CalculateGRAY(rgb[0],rgb[1],rgb[2]);
              gray_packed.range(7,0)=gray;
```

```
                dst.write(i*COLS+j,gray_packed);
            }
        }
    }
```

源文件通道还原部分

```
template<int ROWS,int COLS>
void xfgray2rgb(xf::cv::Mat<XF_8UC1,ROWS,COLS,XF_NPPC1>&src,xf::cv::Mat<XF_8UC3,ROWS,COLS,XF_NPPC1>&dst)
{
  XF_TNAME(XF_8UC1,XF_NPPC1)gray_packed;
  XF_TNAME(XF_8UC3,XF_NPPC1)rgb_packed;
  ap_uint<8>gray;
  unsigned int i,j=0;
  for(i=0;i<ROWS;i++)
  {
        for(j=0;j<COLS;j++)
        {
#pragma HLS PIPELINE
              gray_packed=src.read(i*COLS+j);
              gray=gray_packed.range(7,0);
              rgb_packed.range(7,0)=gray;
              rgb_packed.range(15,8)=gray;
              rgb_packed.range(23,16)=gray;
              dst.write(i*COLS+j,rgb_packed);
        }
    }
}
```

源文件顶层函数部分

```
void rgb2gray(hls::stream<ap_axiu<24,1,1,1>>&video_in,hls::stream<ap_axiu<24,1,1,1>>&video_out)
{
#pragma HLS INTERFACE ap_ctrl_none port=return
#pragma HLS DATAFLOW
#pragma HLS INTERFACE axis port=video_out register_mode=both register
#pragma HLS INTERFACE axis port=video_in register_mode=both register
  xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS,IMG_MAX_COLS,XF_NPPC1>img_rgb_src;
#pragma HLS STREAM variable=img_rgb_src.data depth=1920 dim=1
  xf::cv::Mat<XF_8UC1,IMG_MAX_ROWS,IMG_MAX_COLS,XF_NPPC1>img_gray_src;
#pragma HLS STREAM variable=img_gray_src.data depth=1920 dim=1
  xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS,IMG_MAX_COLS,XF_NPPC1>img_gray_dst;
#pragma HLS STREAM variable=img_gray_dst.data depth=1920 dim=1
  xf::cv::AXIvideo2xfMat(video_in,img_rgb_src);
  xfrgb2gray<IMG_MAX_ROWS,IMG_MAX_COLS>(img_rgb_src,img_gray_src);
  xfgray2rgb<IMG_MAX_ROWS,IMG_MAX_COLS>(img_gray_src,img_gray_dst);
  xf::cv::xfMat2AXIvideo(img_gray_dst,video_out);
}
```

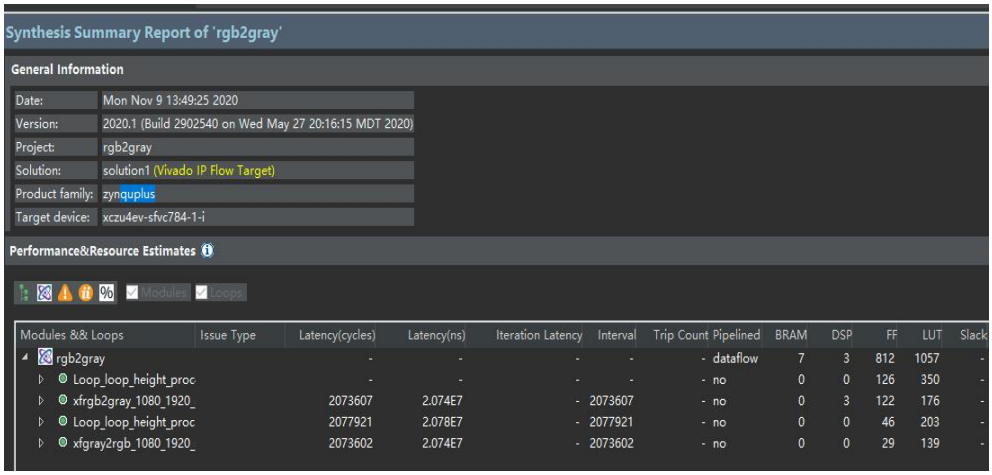
5.3 接口设置和工程优化

接口设置:

Block level Port 设置为 ap_ctrl_none，输入和输出参数因为是连续送入的数据设置为 axi 类型接口

工程优化:

为提升图像帧数顶层函数中设置 Dataflow，因此中间两个函数的输入输出参数 img_rgb_src、img_gray_src 和 img_gray_dst 设置为 stream 类型，深度设置为 1920，循环中设置 pipeline 优化，提升运行速度，整体资源消耗与延时信息如下图所示，可以看出延时在 20ms 左右，能满足帧数要求



5.4 工程路径

名称	路径
Vivado 工程	vivado/rgb2gray
HLS 工程	hls/ rgb2gray

5.5 工程实现

烧录之后能在分辨率为 1920*1080 的显示器中看到如下图像



可以看到图像从 RGB 变成了灰色

第六章 图像 RGB 转 YCrCb

6.1 实验介绍:

由于上一章所讲到的 RGB 图像转换为灰度图像之后无法还原为 RGB 图像，对于一些要经过处理之后输出依旧为 RGB 图像的情况，可以使用 RGB 转 YCrCb 图像的算法将 RGB 转换为 YCrCb，经过处理之后再还原为 RGB 图像输出，其中 Y 指的就是之前的灰度图，RGB 转 YCrCb 转换公式如下:

Y = 0.299*R + 0.587*G + 0.114*B

Cr= (R-Y)*0.713+delta

Cb= (B-Y)*0.564+delta

$$delta = \begin{cases} 128 & for\ 8\text{-}bit\ images \\ 32768 & for\ 16\text{-}bit\ images \\ 0.5 & for\ floating\ point\ images \end{cases}$$

YCrCb 转 RGB 转换公式如下:

- R= Y+1.403*(Cr-delta)
- G= Y-0.714*(Cr-delta)-0.344*(cb-delta)
- B= Y+1.773+(Cb-delta)

$$delta = \begin{cases} 128 & for\ 8\text{-}bit\ images \\ 32768 & for\ 16\text{-}bit\ images \\ 0.5 & for\ floating\ point\ images \end{cases}$$

实验中会先展示 YCrCb 图像，之后通过寄存器控制再显示 RGB 图像

6.2 实验源码

头文件中参数

```
#ifndef __rgb2ycrcb_h__
#define __rgb2ycrcb_h__

#include "ap_int.h"
#include "hls_stream.h"
#include "ap_axi_sdata.h"

#include "common/xf_common.hpp"
#include "common/xf_utility.hpp"
#include "common/xf_infra.hpp"

#include "imgproc/xf_cvt_color.hpp"

#define IMG_MAX_ROWS 1080
#define IMG_MAX_COLS 1920
```

```
    void rgb2ycrcb(hls::stream<ap_axiu<24,1,1,1>>&src,hls::stream<ap_axiu<24,1,1,1>>&dst,ap_uint<1>en);

#endif
```

源文件中提取通道和整合通道部分

```
    void ExtractPixel(XF_TNAME(XF_8UC3,XF_NPPC1)&src,ap_uint<8>dst[3])
    {
    #pragma HLS INLINE off
      unsigned int i,j=0;
      for(i=0;i<24;i+=8)
      {
    #pragma HLS UNROLL
            dst[j]=src.range(i+7,i);
            j++;
      }
    }

    void PackPixel(XF_TNAME(XF_8UC3,XF_NPPC1)&dst,ap_uint<8>src[3])
    {
    #pragma HLS INLINE off
      unsigned int i,j=0;
      for(i=0;i<24;i+=8)
      {
    #pragma HLS UNROLL
            dst.range(i+7,i)=src[j];
            j++;
      }
}
```

源文件中 RGB 转 YCrCb 部分

```
    template<int ROWS,int COLS>
    void
xfrgb2ycrcb(xf::cv::Mat<XF_8UC3,ROWS,COLS,XF_NPPC1>&src,xf::cv::Mat<XF_8UC3,ROWS,COLS,XF_NPPC1>&dst)
    {
      XF_TNAME(XF_8UC3,XF_NPPC1)rgb_packed;
      XF_TNAME(XF_8UC3,XF_NPPC1)ycrcb_packed;
      ap_uint<8>rgb[3];
      ap_uint<8>ycrcb[3];
      unsigned int i,j=0;
      for(i=0;i<ROWS;i++)
      {
            for(j=0;j<COLS;j++)
            {
    #pragma HLS PIPELINE
                  rgb_packed=src.read(i*COLS+j);
                  ExtractPixel(rgb_packed,rgb);
                  ycrcb[0]=CalculateGRAY(rgb[0],rgb[1],rgb[2]);
                  ycrcb[1]=Calculate_CR(rgb[0],ycrcb[0]);
                  ycrcb[2]=Calculate_CB(rgb[2],ycrcb[0]);
                  PackPixel(ycrcb_packed,ycrcb);
                  dst.write(i*COLS+j,ycrcb_packed);
            }
      }
}
```

源文件中 YCrCb 转 RGB 部分

```
    template<int ROWS,int COLS>
    void
xfycrcb2rgb(xf::cv::Mat<XF_8UC3,ROWS,COLS,XF_NPPC1>&src,xf::cv::Mat<XF_8UC3,ROWS,COLS,XF_NPPC1>&dst,ap_uint<1>en
)
    {
      XF_TNAME(XF_8UC3,XF_NPPC1)rgb_packed;
      XF_TNAME(XF_8UC3,XF_NPPC1)ycrcb_packed;
      ap_uint<8>rgb[3];
      ap_uint<8>ycrcb[3];
      unsigned int i,j=0;
      if(en==0)
      {
            for(i=0;i<ROWS;i++)
            {
                  for(j=0;j<COLS;j++)
                  {
    #pragma HLS PIPELINE
                        ycrcb_packed=src.read(i*COLS+j);
                        rgb_packed=ycrcb_packed;
                        dst.write(i*COLS+j,rgb_packed);
```

```
                }
            }
        }
        else
        {
            for(i=0;i<ROWS;i++)
            {
                for(j=0;j<COLS;j++)
                {
    #pragma HLS PIPELINE
                    ycrcb_packed=src.read(i*COLS+j);
                    ExtractPixel(ycrcb_packed,ycrcb);
                    rgb[0]=Calculate_Ycrcb2R(ycrcb[0],ycrcb[1]);
                    rgb[1]=Calculate_Ycrcb2G(ycrcb[0],ycrcb[1],ycrcb[2]);
                    rgb[2]=Calculate_Ycrcb2B(ycrcb[0],ycrcb[2]);
                    PackPixel(rgb_packed,rgb);
                    dst.write(i*COLS+j,rgb_packed);
                }
            }
        }
}
```

源文件中顶层函数部分

```
    void rgb2ycrcb(hls::stream<ap_axiu<24,1,1,1>>&src,hls::stream<ap_axiu<24,1,1,1>>&dst,ap_uint<1>en)
    {

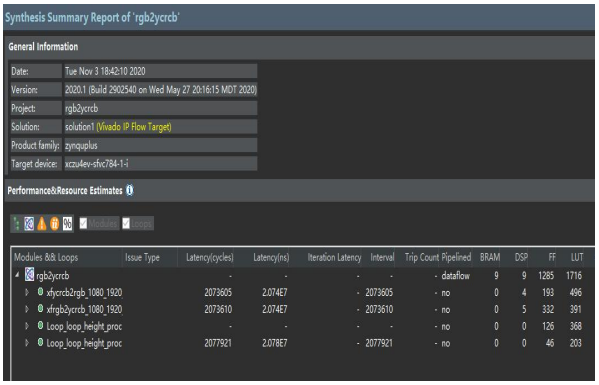
    #pragma HLS DATAFLOW
    #pragma HLS INTERFACE ap_vld port=en register
    #pragma HLS INTERFACE axis port=dst register_mode=both register
    #pragma HLS INTERFACE axis port=src register_mode=both register
    #pragma HLS INTERFACE ap_ctrl_none port=return
      xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS,IMG_MAX_COLS,XF_NPPC1>img_src;
    #pragma HLS STREAM variable=img_src.data depth=1920 dim=1
      xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS,IMG_MAX_COLS,XF_NPPC1>img_gray_src;
    #pragma HLS STREAM variable=img_gray_src.data depth=1920 dim=1
      xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS,IMG_MAX_COLS,XF_NPPC1>img_dst;
    #pragma HLS STREAM variable=img_dst.data depth=1920 dim=1

      xf::cv::AXIvideo2xfMat(src,img_src);
      xfrgb2ycrcb(img_src,img_gray_src);
      xfycrcb2rgb(img_gray_src,img_dst,en);
      xf::cv::xfMat2AXIvideo(img_dst,dst);
}
```

6.3 接口综合和工程优化

接口综合：Block Level Interface 设置和之前章节相同，参数设置和之前章节相同，参数 en 设置为 ap_vld 型，因为该参数用于判断是否进行 YCrCb 到 RGB 转换，有效信号和帧起始信号 user 相接，参数每帧送入一次

工程优化：工程中的优化和之前基本相同，由于 ExtractPixel 和 PackPixel 调用次数较多设置了 inline=off 防止被内联化，降低资源消耗，同时对循环做了展开减少延时，整体资源消耗情况如图所示

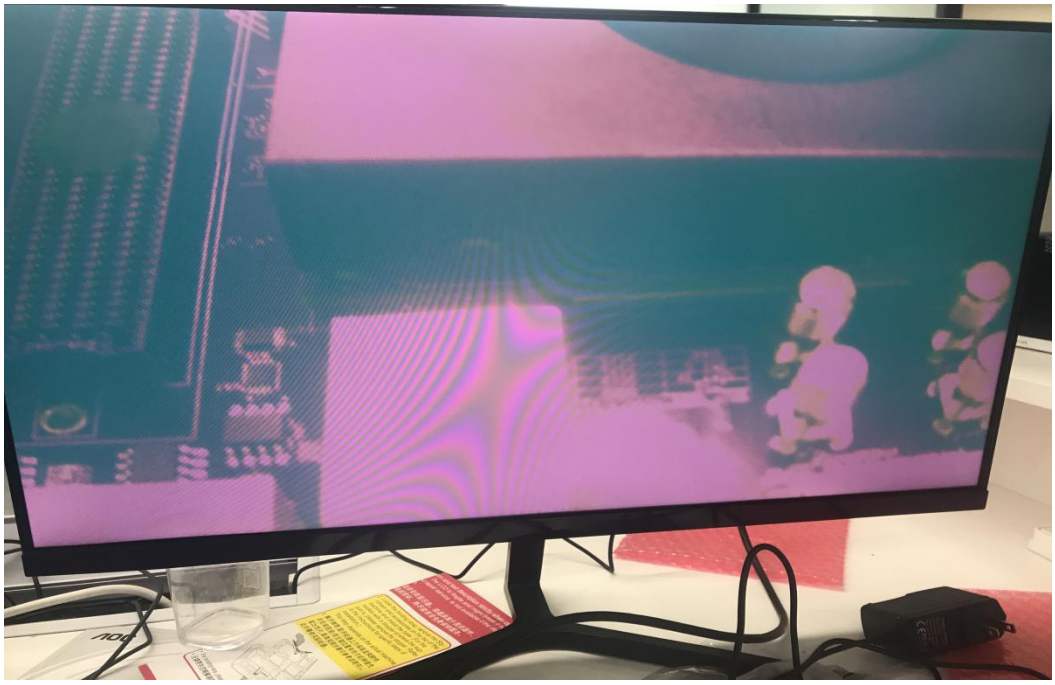


6.4 工程路径

名称	路径
Vivado 工程	vivado/rgb2ycrcb
HLS 工程	hls/ rgb2ycrcb
BOOT.bin 文件	vivado/rgb2ycrcb/rgb2ycrcb.sdk/app/bootimage/BOOT.bin

6.5 工程实现

烧录之后会先显示 Ycrcb 图像，效果如图所示



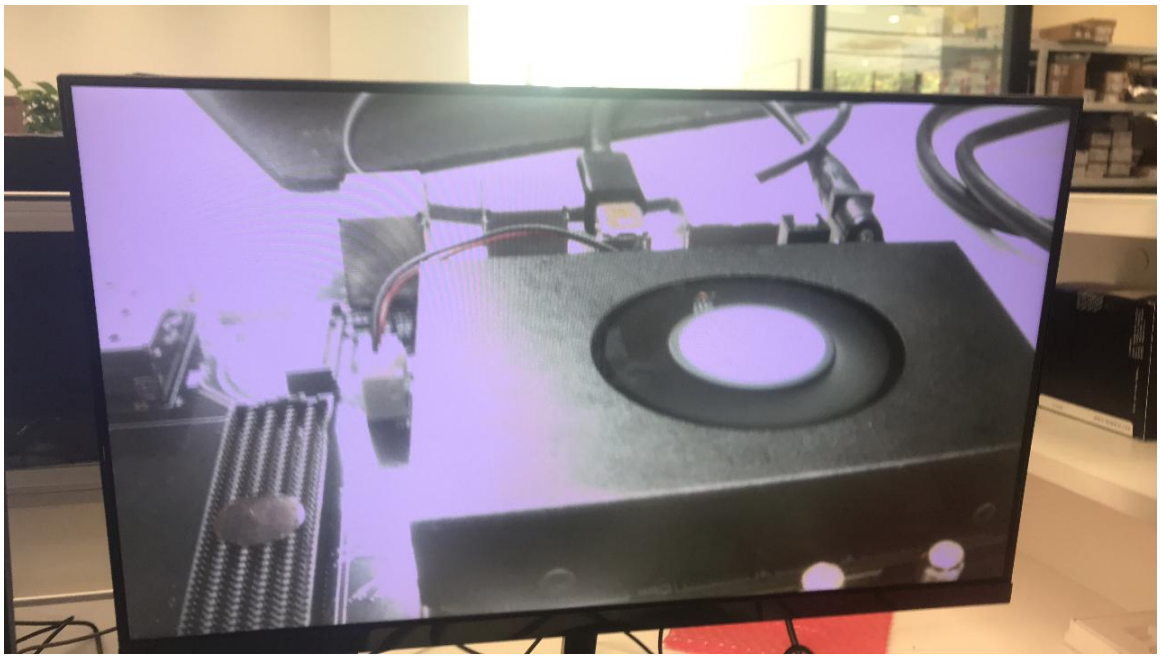
在 vitis 中输入如下命令

```
XSCT Process
185:    b    _boot
xsct% Info: Cortex-A53 #0 (target 9) Running
mrd 0x80040000 8
80040000:   00000000
80040004:   00000000
80040008:   00000000
8004000C:   00000000
80040010:   00000000
80040014:   00000000
80040018:   00000000
8004001C:   00000000

xsct% mwr 0x8004001c 0x1
xsct%

xsct%
```

显示效果如下：



第七章 图像形态学滤波

7.1 实验介绍

我们这章尝试两种比较常见形态学滤波方式，分别是腐蚀与膨胀，腐蚀指的是在目标像素周边若干个点出取其中值最小的像素赋予目标像素，其效果会使图像相较于原先的图像滴两滴区域增加，膨胀与腐蚀相反，在目标像素周边取值最大处赋予目标像素，图像相较于原来高亮度区域增加，可用于增强图像画质，减少其中的不连续点。

7.2 实验源码

顶层文件膨胀部分参数定义

```
#ifndef __dilation_h__
#define __dilation_h__

#include "ap_int.h"
#include "ap_axi_sdata.h"
#include "hls_stream.h"

#include "common/xf_common.hpp"
#include "common/xf_utility.hpp"
#include "common/xf_infra.hpp"

#define IMG_MAX_ROWS 1080
#define IMG_MAX_COLS 1920

#define NEW_K_ROWS  3
#define NEW_K_COLS 3

void dilation(hls::stream<ap_axiu<24,1,1,1>>&src,hls::stream<ap_axiu<24,1,1,1>>&dst);

#endif
```

顶层文件腐蚀部分参数定义

```
#ifndef __erosion_h__
#define __erosion_h__

#include "ap_int.h"
#include "ap_axi_sdata.h"
#include "hls_stream.h"

#include "common/xf_common.hpp"
#include "common/xf_utility.hpp"
#include "common/xf_infra.hpp"

#include "imgproc/xf_cvt_color.hpp"
#include "hsv2rgb.hpp"
#include "rgb2hsv.hpp"

#define IMG_MAX_ROWS 1080
#define IMG_MAX_COLS 1920

#define NEW_K_ROWS  3
#define NEW_K_COLS 3

void erosion(hls::stream<ap_axiu<24,1,1,1>>&src,hls::stream<ap_axiu<24,1,1,1>>&dst);

#endif
```

源文件膨胀部分核心代码

```
template <int PLANES, int DEPTH, int K_ROWS, int K_COLS>
void dilate_function_apply(XF_PTUNAME(DEPTH) * OutputValue,
                           XF_PTUNAME(DEPTH) src_buf[K_ROWS][K_COLS],
                           unsigned char kernel[K_ROWS][K_COLS]) {
// clang-format off
    #pragma HLS INLINE
    // clang-format on

    XF_PTUNAME(DEPTH) packed_val = 0, depth = XF_PIXELWIDTH(DEPTH, XF_NPPC1) / PLANES;

Apply_dilate_Loop:
    for (ap_uint<5> c = 0, k = 0; c < PLANES; c++, k += depth) {
// clang-format off
        #pragma HLS LOOP_TRIPCOUNT min=1 max=PLANES
        #pragma HLS UNROLL
        // clang-format on
        XF_PTNAME(DEPTH) max = 0;
        for (ap_uint<13> k_rows = 0; k_rows < K_ROWS; k_rows++) {
// clang-format off
            #pragma HLS LOOP_TRIPCOUNT min=1 max=K_ROWS
            #pragma HLS UNROLL
            // clang-format on
            for (ap_uint<13> k_cols = 0; k_cols < K_COLS; k_cols++) {
// clang-format off
                #pragma HLS LOOP_TRIPCOUNT min=1 max=K_COLS
                #pragma HLS UNROLL
                // clang-format on
                if (kernel[k_rows][k_cols]) {
                    if (src_buf[k_rows][k_cols].range(k + (depth - 1), k) > max) {
                        max = src_buf[k_rows][k_cols].range(k + (depth - 1), k);
                    }
                }
            }
        }
        packed_val.range(k + (depth - 1), k) = max;
    }

    *OutputValue = packed_val;
    return;
}
```

源文件腐蚀部分核心代码

```
template <int PLANES, int DEPTH, int K_ROWS, int K_COLS>
void erode_function_apply(XF_PTUNAME(DEPTH) * OutputValue,
                                XF_TNAME(XF_8UC3,XF_NPPC1)&buf_value,
                          XF_PTUNAME(DEPTH) src_buf[K_ROWS][K_COLS]) {
// clang-format off
    #pragma HLS INLINE
    // clang-format on

    XF_PTUNAME(DEPTH) packed_val = 0, depth = XF_PIXELWIDTH(DEPTH, XF_NPPC1) / PLANES;

        // clang-format on
        ap_uint<8> min = 255;
        for (ap_uint<13> k_rows = 0; k_rows < K_ROWS; k_rows++) {
// clang-format off
            #pragma HLS LOOP_TRIPCOUNT min=1 max=K_ROWS
            #pragma HLS UNROLL
            // clang-format on
            for (ap_uint<13> k_cols = 0; k_cols < K_COLS; k_cols++) {
// clang-format off
                #pragma HLS LOOP_TRIPCOUNT min=1 max=K_COLS
                #pragma HLS UNROLL
                // clang-format on
                    if (src_buf[k_rows][k_cols].range(23, 16) < min) {
                        min = src_buf[k_rows][k_cols].range(23, 16);
                    }
            }
        }
        packed_val.range(15,0)=buf_value.range(15,0);
        packed_val.range(23,16) = min;

    *OutputValue = packed_val;
    return;
}
```

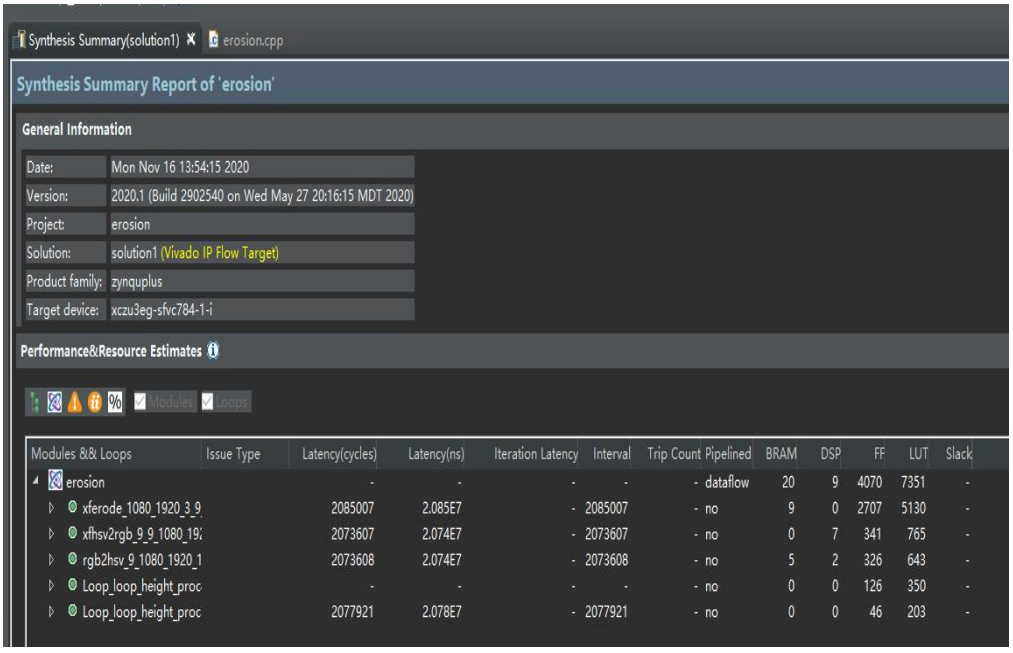
7.3 接口综合与工程优化

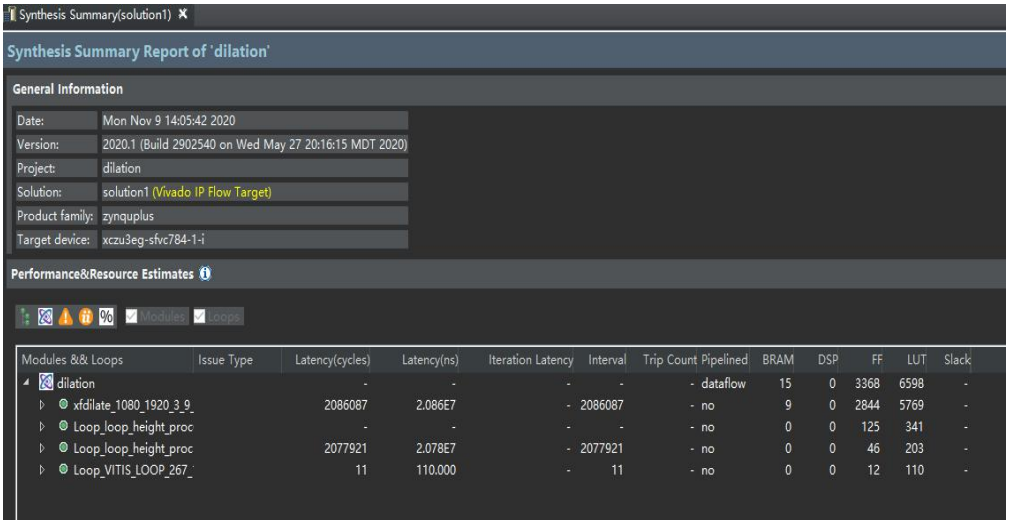
接口综合：

接口设置与上一章类似，Block level Interface 设置为 ap_ctrl_none 去除多余的关联信号，输入输出参数因为是连续变化的数据设置为 axi 类型

工程优化：

顶层函数设置为 Dataflow 降低运行延时，函数中因为有循环次数是变量的情况较多地方使用 tripcount 以便预估整体运行延时，对资源消耗不大的循环使用循环展开提升整体函数运行速度，函数延时在 20ms 左右能达到时序要求



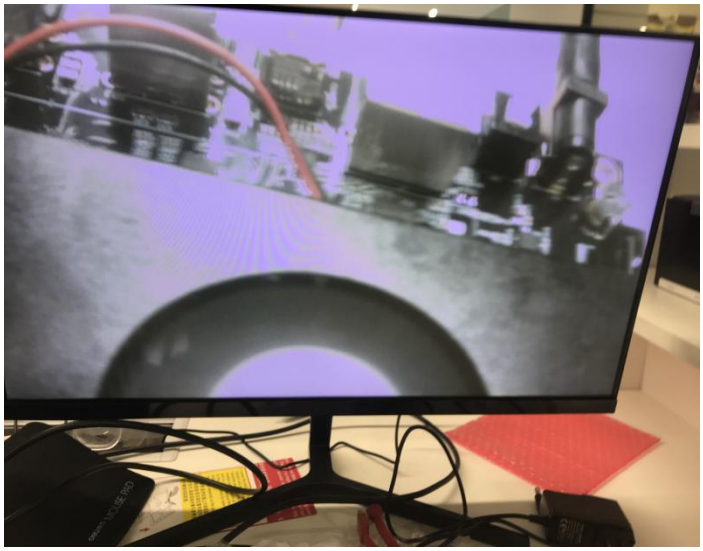


7.4 工程路径

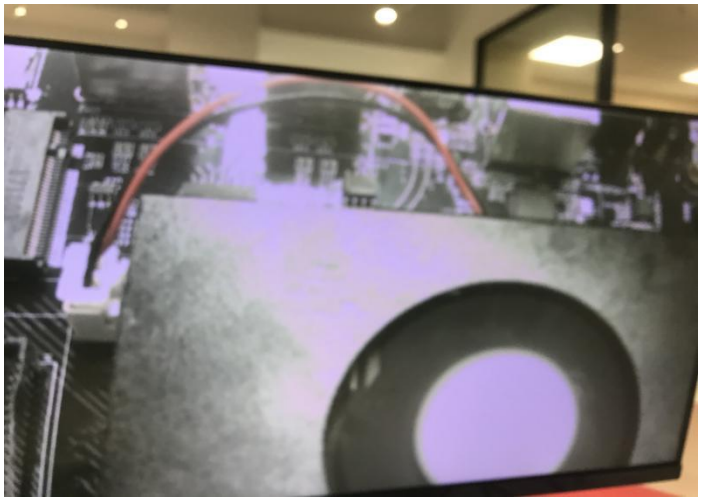
名称	路径
Vivado 工程	vivado/dilation
HLS 工程	hls/ dilation
Vivado 工程	vivado/erosion
HLS 工程	hls/ erosion

7.5 工程实现

膨胀操作显示效果



腐蚀操作显示效果



第八章 图像叠加

8.1 实验介绍

对视频进行灰度变换，转换为灰度图像之后叠加在原本的图像上，通过寄存器控制其大小和透明度以及显示位置，通过 DP 显示在显示屏上观察现象

8.2 实验源码

顶层文件参数定义

```
    #ifndef __overlaystream_h__
    #define __overlaystream_h__

    #include "ap_int.h"

    #include <hls_stream.h>
    #include <ap_axi_sdata.h>
    #include <string.h>

    #include "common/xf_common.hpp"
    #include "common/xf_infra.hpp"
    #include "common/xf_utility.hpp"
    #include "imgproc/xf_resize.hpp"
    #include "imgproc/xf_cvt_color.hpp"

    typedef xf::cv::ap_axiu<24,1,1,1> pixel_t;
    typedef hls::stream<pixel_t> vstream_t;

    #define IMG_MAX_ROWS    1080
    #define IMG_MAX_COLS    1920

    void overlaystream(vstream_t &video_in, vstream_t &video_out,unsigned int overlay_alpha);

#endif
```

源文件叠加部分

```
    template<int ROWS, int COLS>
    void overlyOnMat(xf::cv::Mat<XF_8UC3,ROWS, COLS,XF_NPPC1> &img_bottom, xf::cv::Mat<XF_8UC1,ROWS,COLS,XF_NPPC1>
&img_top, xf::cv::Mat<XF_8UC3,ROWS, COLS,XF_NPPC1>
&img_out,ap_uint<8>overly_alpha,ap_uint<32>overly_x,ap_uint<32>overly_y,ap_uint<32>overly_h,ap_uint<32>overly_w)
    {
      XF_TNAME(XF_8UC3,XF_NPPC1) pixelBottom;
      XF_TNAME(XF_8UC1,XF_NPPC1) pixelTop;
      XF_TNAME(XF_8UC3,XF_NPPC1) pixelMix;
```

```
    ap_uint<8> pixelTop_value;
  #pragma HLS ARRAY_PARTITION variable=pixelTop_value dim=1 complete
    ap_uint<8> pixelBottom_value[3];
  #pragma HLS ARRAY_PARTITION variable=pixelBottom_value dim=1 complete
    ap_uint<8> pixelMix_value[3];
  #pragma HLS ARRAY_PARTITION variable=pixelMix_value dim=1 complete
    unsigned int idx=0;
    ap_uint<8>overly_alpha_tmp=0;
    for(int row=0; row<ROWS; row++)
        {
    #pragma HLS loop_tripcount min=12 max=1080
            for(int col=0; col<COLS; col++)
            {
    #pragma HLS loop_tripcount min=3 max=1920
    #pragma HLS pipeline
                pixelBottom=img_bottom.read(row*COLS+col);
                pixelTop=img_top.read(row*COLS+col);
                if((row>=overly_y)&&(row<overly_h)&&(col>=overly_x)&&(col<=overly_w))
                {
                    overly_alpha_tmp=overly_alpha;
                }
                else
                {
                    overly_alpha_tmp=0;
                }
                ExtractPixel(pixelBottom,pixelBottom_value);
                pixelTop_value=pixelTop.range(7,0);

    pixelMix_value[0]=((255-overly_alpha_tmp)*pixelBottom_value[0]+overly_alpha_tmp*pixelTop_value)/255;

    pixelMix_value[1]=((255-overly_alpha_tmp)*pixelBottom_value[1]+overly_alpha_tmp*pixelTop_value)/255;

    pixelMix_value[2]=((255-overly_alpha_tmp)*pixelBottom_value[2]+overly_alpha_tmp*pixelTop_value)/255;
                PackPixel(pixelMix,pixelMix_value);
                img_out.write(row*COLS+col,pixelMix);
            }
        }
  }
```

源文件顶层函数部分

```
    void overlaystream(vstream_t &video_in, vstream_t &video_out,ap_uint<8>overly_alpha,ap_uint<32>
overly_x,ap_uint<32>overly_y,ap_uint<32>overly_h,ap_uint<32>overly_w)
    {
    #pragma HLS INTERFACE ap_vld port=overly_alpha register
    #pragma HLS INTERFACE ap_vld port=overly_w register
    #pragma HLS INTERFACE ap_vld port=overly_h register
    #pragma HLS INTERFACE ap_vld port=overly_y register
    #pragma HLS INTERFACE ap_vld port=overly_x register
    #pragma HLS ALLOCATION instances=resize limit=1 function
    #pragma HLS INTERFACE axis port=video_out register_mode=both register
    #pragma HLS INTERFACE ap_ctrl_none port=return
    #pragma HLS INTERFACE axis port=video_in

      xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1> img_in;
    #pragma HLS STREAM variable=img_in.data depth=1920 dim=1
      xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1> img_out;
    #pragma HLS STREAM variable=img_out.data depth=1920 dim=1
      xf::cv::Mat<XF_8UC1,IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1> img_src1;
    #pragma HLS STREAM variable=img_src1.data depth=1920 dim=1
      xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS,IMG_MAX_COLS,XF_NPPC1> img_dst1;
    #pragma HLS STREAM variable=img_dst1.data depth=1920 dim=1
      xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS,IMG_MAX_COLS, XF_NPPC1> img_src2;
    #pragma HLS STREAM variable=img_src2.data depth=1920 dim=1

    #pragma HLS dataflow

      xf::cv::AXIvideo2xfMat(video_in, img_in);
      xfrgb2gray<IMG_MAX_ROWS,IMG_MAX_COLS>(img_in,img_src1,img_src2);
      overlyOnMat<IMG_MAX_ROWS,IMG_MAX_COLS>(img_src2,img_src1,img_out,overly_alpha,overly_x,overly_y,overly_h,o
verly_w);
      xf::cv::xfMat2AXIvideo(img_out, video_out);
}
```

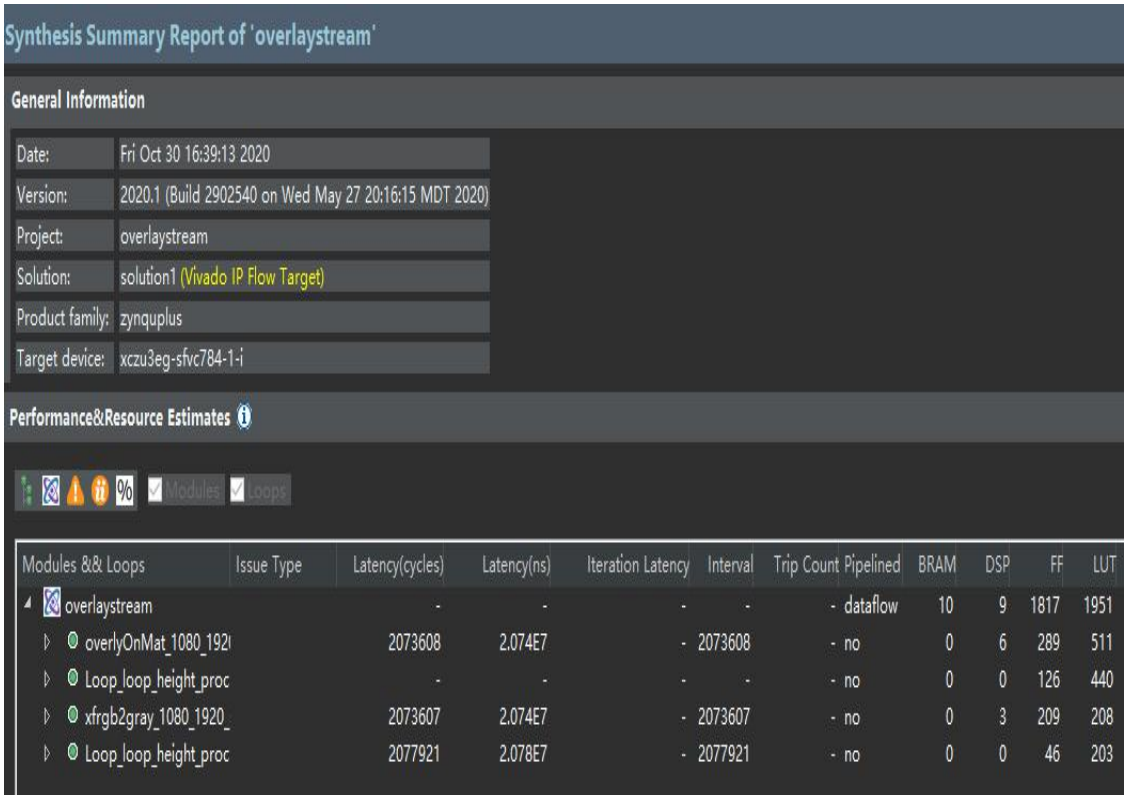
8.3 接口综合和工程优化：

接口综合：

接口设置与上一章基本相同，我们也可以尝试从 DDR 中将数据读出之后做对应处理，从 DDR 中读取数据时接口类型需要设置为 m_axi 型，显示位置和显示透明度的参数是每帧送入一次，因此还是将参数设置为 ap_vld 类型，有效信号和帧起始信号相接。

工程优化：

代码中顶层函数设置了 Dataflow，叠加部分的函数对内层函数设置了 Pipeline，减少了运行的延时，如果对外层函数设置 pipeline 会对内层函数做展开，但由于顶层函数设置为 Pipeline 函数之间的参数传递被综合为 fifo，只有一个输出端口，对内层函数展开实际是无法实现的，因此也无法对外层函数做 Pipeline 优化，整体运行延时在 20ms 左右能够满足帧数要求

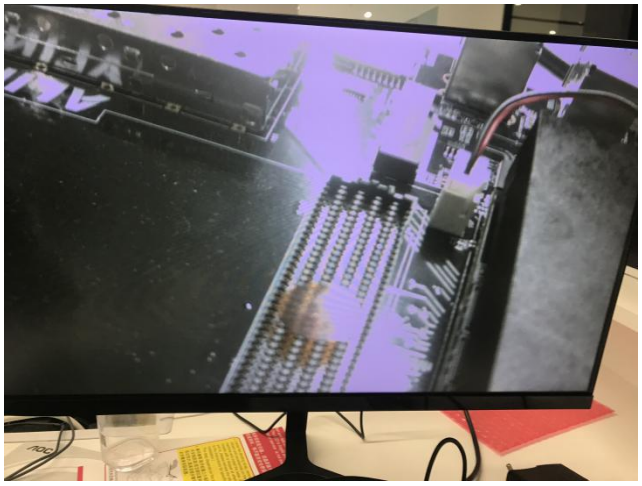


8.4 工程路径

名称	路径
Vivado 工程	vivado/video_on_video
HLS 工程	hls/ overlaystream

8.5 工程实现

设置对比度为 0 时图像：



对比度设置为 255 时图像：



叠加的图像位置和透明度都可以自行调节，透明度最大为 255 最小为 0，我们可以通过寄存器设置不同透明度观察现象

第九章 图像对比度调整

9.1 实验介绍

图像对比度调整通过对整体图像做直方图均衡化实现，直方图即图像灰度统计图，根据一帧图像的灰度统计图得到整体图像的亮度分布统计图，得到亮度分布统计图之后，我们首先对图像做 RGB 到 YCrCb 的变化，之后根据统计 Y 的值，为了提升帧数，将用于存储均衡化之后结果的数组设置为 static，下一帧图像到来之后的 Y 值用上一帧统计的结果替换，如果使用同一帧的统计值会增大整体处理的延时且效果和用上一帧统计结果相差不大，因此不建议使用这种方式，经过之前的操作之后我们能够对图像进行画质增强，将原本过度曝光或曝光不足的图片的对比度调整在一个更合适的范围内。

9.2 实验源码

顶层文件参数定义

```
#ifndef __CONTRASTADJ_H__
#define __CONTRASTADJ_H__

#include "ap_int.h"
#include "hls_stream.h"
#include "common/xf_utility.hpp"
#include "common/xf_infra.hpp"
#include "common/xf_common.hpp"
#include "imgproc/xf_cvt_color.hpp"

#define IMG_MAX_HEIGHT  1080
#define IMG_MAX_WIDTH   1920

void contrastadj(hls::stream<ap_axiu<24,1,1,1>> &src_axi, hls::stream<ap_axiu<24,1,1,1>> &dst_axi, int
adj);

#endif
```

源文件直方图统计部分

```
template<int ROWS, int COLS, int N>
void dualAryEqualize(
       xf::cv::Mat<XF_8UC3,ROWS, COLS,XF_NPPC1>    &_src,
       xf::cv::Mat<XF_8UC3,ROWS, COLS,XF_NPPC1>    &_dst,
       int filter)
{
  static ap_uint<8> map[N];
  ap_uint<32>hist_out1[N];
#pragma HLS ARRAY_PARTITION variable=hist_out1 dim=1 complete
  ap_uint<32>hist_out2[N];
#pragma HLS ARRAY_PARTITION variable=hist_out2 dim=1 complete
  unsigned int i,j=0;
  XF_TNAME(XF_8UC3,XF_NPPC1)src1;
  XF_TNAME(XF_8UC3,XF_NPPC1)src2;
  XF_TNAME(XF_8UC3,XF_NPPC1)dst1;
  XF_TNAME(XF_8UC3,XF_NPPC1)dst2;
  ap_uint<8> value1[3];
  ap_uint<8> value2[3];
  int pos1,pos2;
  for(i=0;i<N;i++)
  {
       hist_out1[i]=0;
       hist_out2[i]=0;
  }


  for(i=0;i<ROWS;i++)
  {
```

```
            for(j=0;j<COLS;j+=2)
            {
    #pragma HLS DEPENDENCE variable=src1 intra false
    #pragma HLS DEPENDENCE variable=src2 intra false

    #pragma HLS PIPELINE II=2
                src1=_src.read(i*COLS+j);
                src2=_src.read(i*COLS+j+1);
                ExtractPixel(src1,value1);
                ExtractPixel(src2,value2);
                pos1=(int)value1[0];
                pos2=(int)value2[0];
                hist_out1[pos1]+=1;
                hist_out2[pos2]+=1;
                value1[0]=map[pos1];
                value2[0]=map[pos2];
                PackPixel(dst1,value1);
                PackPixel(dst2,value2);
                _dst.write(i*COLS+j,dst1);
                _dst.write(i*COLS+j+1,dst2);
            }
      }
      int count_total = 0, tmp_hist;
      float scale;
      for(int i=0;i<N;i++)
      {
    #pragma HLS PIPELINE
            tmp_hist=hist_out1[i]+hist_out2[i];
            count_total += tmp_hist;
            scale = count_total*255/ROWS/COLS;
            scale=(scale-i)*0.01*filter+i;
            if(scale < 16)scale = 16;
            if(scale > 255)scale = 235;
            map[i] =  ap_uint<8>  (scale);
      }
}
```

源文件顶层函数部分

```
    void contrastadj(hls::stream<ap_axiu<24,1,1,1>> &src_axi,hls::stream<ap_axiu<24,1,1,1>> &dst_axi,int
adj)
    {

    #pragma HLS INTERFACE ap_ctrl_none port=return
    #pragma HLS INTERFACE ap_vld port=adj register

    #pragma HLS INTERFACE axis port=src_axi
    #pragma HLS INTERFACE axis port=dst_axi

      xf::cv::Mat<XF_8UC3,IMG_MAX_HEIGHT, IMG_MAX_WIDTH, XF_NPPC1> img1;
    #pragma HLS STREAM variable=img1.data depth=1 dim=1
      xf::cv::Mat<XF_8UC3,IMG_MAX_HEIGHT, IMG_MAX_WIDTH, XF_NPPC1> img2;
    #pragma HLS STREAM variable=img2.data depth=1 dim=1
      xf::cv::Mat<XF_8UC3,IMG_MAX_HEIGHT, IMG_MAX_WIDTH, XF_NPPC1> img3;
    #pragma HLS STREAM variable=img3.data depth=1 dim=1
      xf::cv::Mat<XF_8UC3,IMG_MAX_HEIGHT, IMG_MAX_WIDTH, XF_NPPC1> img4;
    #pragma HLS STREAM variable=img4.data depth=1 dim=1

    #pragma HLS dataflow

    // AXIvideoTest<IMG_MAX_HEIGHT,IMG_MAX_WIDTH>(src_axi,dst_axi);
     xf::cv::AXIvideo2xfMat<24,XF_8UC3,IMG_MAX_HEIGHT,IMG_MAX_WIDTH,XF_NPPC1>(src_axi, img1);
      xfrgb2ycrcb<IMG_MAX_HEIGHT,IMG_MAX_WIDTH>(img1, img2);
      dualAryEqualize<IMG_MAX_HEIGHT, IMG_MAX_WIDTH, 256>(img2, img3,adj);
       xfycrcb2rgb<IMG_MAX_HEIGHT,IMG_MAX_WIDTH>(img3, img4);
     xf::cv::xfMat2AXIvideo<24,XF_8UC3,IMG_MAX_HEIGHT,IMG_MAX_WIDTH,XF_NPPC1>(img4,dst_axi);
}
```

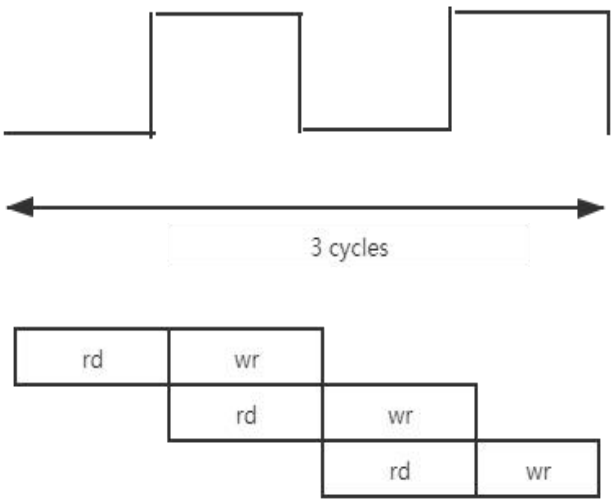
9.3 接口设置和工程优化：

接口设置：

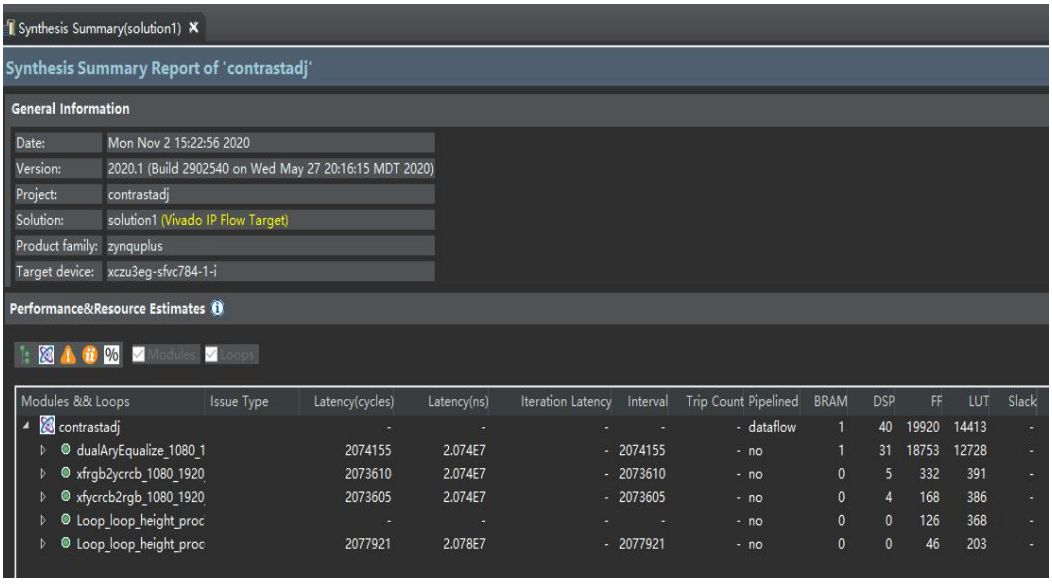
接口设置和上一章相似，Block level Interface 设置为 ap_none 去除多余的控制信号，输入输出因为是连续送入的数据设置为 axis 类型，对比度参数每帧送入一次

工程优化：

工程优化大部分与上一章类似，有一个需要注意的点在于我们 Pipeline 的间隔不能设置为 1 主要原因可以从这张图中看出来：



假设我们将间隔设置为 1，从图中我们可以看出我们从数组中读出的是它上一个时钟的数据，虽然我们可以将数组设置为 dual_port_ram，但我们无法在同一个时钟之内实现读出和写入的操作，这会带来一个特殊的情况就是我们在进行直方图统计时出现两个像素点的灰度值相同，此时根据代码我们会出现从同一个地址读出再写入的情况，而由于我们的读出和写入需要分两个时钟，第二个时钟读出的数据并不是该地址写入之后的数据，也就是增加的数据，而是增加之前的数据，这会导致我们统计出来的灰度值是实际灰度值的二分之一，会影响准确度，对此的解决方式为一次统计两个数据，用两个数组分别存储两个连续的像素统计结果，最终整体将两个数组相加得到整帧实际的统计结果，对其做直方图均衡化，并将得到的灰度值赋予下一帧的灰度值。IP 核整体运行延时在 21ms 左右能满足帧数要求：


Synthesis Summary(solution1)

Synthesis Summary Report of 'contrastadj'

General Information

Date:	Mon Nov 2 15:22:56 2020
Version:	2020.1 (Build 2902540 on Wed May 27 20:16:15 MDT 2020)
Project:	contrastadj
Solution:	solution1 (Vivado IP Flow Target)
Product family:	zynquplus
Target device:	xczu3eg-sfvc784-1-i

Performance&Resource Estimates

Modules Loops

Modules && Loops	Issue Type	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	Slack
contrastadj		-	-	-	-	-	dataflow	1	40	19920	14413	-
dualAryEqualize_1080_1		2074155	2.074E7	-	2074155	-	no	1	31	18753	12728	-
xfrgb2ycrcb_1080_1920		2073610	2.074E7	-	2073610	-	no	0	5	332	391	-
xfycrcb2rgb_1080_1920		2073605	2.074E7	-	2073605	-	no	0	4	168	386	-
Loop_loop_height_proc		-	-	-	-	-	no	0	0	126	368	-
Loop_loop_height_proc		2077921	2.078E7	-	2077921	-	no	0	0	46	203	-

9.4 工程路径

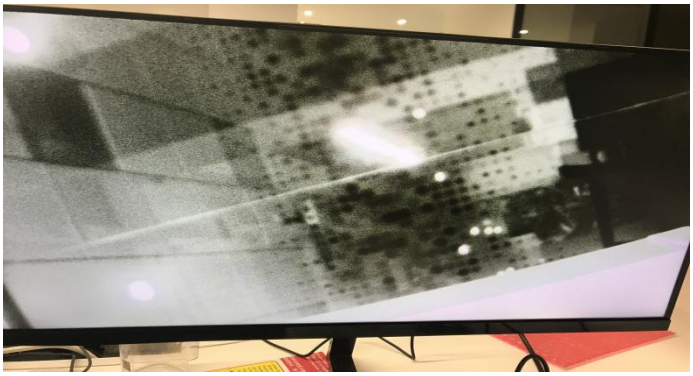
名称	路径
Vivado 工程	vivado/contrast_adj
HLS 工程	hls/contrast_adj

9.5 工程实现

对比度为 0 时的图像：



对比度设置为 100 的图像：



对比都设置为 200 的图像：



第十章 角点检测

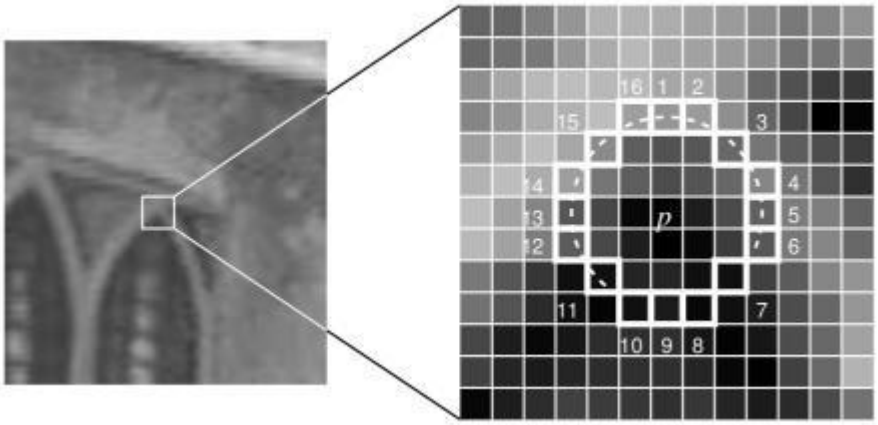
10.1 实验介绍

角点检测(Corner Detection)是计算机视觉系统中用来获得图像特征的一种方法，广泛应用于运动检测、图像匹配、视频跟踪、三维建模和目标识别等领域中。也称为特征点检测。

FAST（Features fromaccelerated segment test）是一种角点检测方法，最初是由 Edward Rosten 和 Tom Drummond 提出，该算法最突出的优点是它的计算效率。正如它的缩写名字，它很快而且事实上它比其他著名的特征点提取方法（如 SIFT，SUSAN，Harris）都要快。如果应用于机器学习方法的话，该算法能够取得更佳的效果。正因为它的快速特点，FAST 角点检测方法非常适用于实时视频处理的领域。

FAST 算法步骤

1) 从图片中选取一个像素 P，下面我们将判断它是否是一个特征点。我们首先把它的亮度值设为 Ip。
2) 设定一个合适的阈值 t。
3) 考虑以该像素点为中心的一个半径等于 3 像素的离散化的 Bresenham 圆，这个圆的边界上有 16 个像素（如图 1 所示）。



4) 现在，如果在这个大小为 16 个像素的圆上有 n 个连续的像素点，它们的像素值要么都比 Ip+t 大，要么都比 Ip−t 小，那么它就是一个角点。（如图 1 中的白色虚线所示）。n 的值可以设置为 12 或者 9，实验证明选择 9 可能会有更好的效果。

上面的算法中，对于图像中的每一个点，我们都要去遍历其邻域圆上的 16 个点的像素，效率较低。我们下面提出了一种高效的测试（high-speed test）来快速排除一大部分非角点的像素。该方法仅仅检查在位置 1，9，5 和 13 四个位置的像素，首先检测位置 1 和位置 9，如果它们都比阈值暗或比阈值亮，再检测位置 5 和位置 13。如果 PP 是一个角点，那么上述四个像素点中至少有 3 个应该必须都大于 Ip+t 或者小于 Ip−t，因为若是一个角点，超过四分之三圆的部分应该满足判断条件。如果不满足，那么 p 不可能是一个角点。对于所有点做上面这一部分初步的检测后，符合条件的将成为候选的角点，我们再对候选的角点，做完整的测试，即检测圆上的所有点。

上面的算法效率实际上是很高的，但是有一些缺点：

1) 当我们设置 n<12 时就不能使用快速算法来过滤非角点的点；
2) 检测出来的角点不是最优的，这是因为它的效率取决于问题的排序与角点的分布;
3) 对于角点分析的结果被丢弃了;
4) 多个特征点容易挤在一起。

可以通过机器学习和非极大值抑制的方法来增强鲁棒性。由于 opencv 中相关的函数没有使用机器学习，因此我们这里只介绍非极大值抑制的方法。由于分割测试并没有计算角点响应函数，因此常规的非极大值抑制方法并不适用于 FAST 算法。下面是 FAST 的非极大值抑制方法：

1) 计算得分函数，它的值 V 是特征点与其圆周上 16 个像素点的绝对差值中所有连续 10 个像素中的最小值的最大值，而且该值还要大于阈值 t；
2) 在 3×3 的特征点邻域内（而不是图像邻域），比较 V；
3) 剔除掉非极大值的特征点。

在 opencv 中，实现 FAST 算法的核心函数有两个，它们的原型为：

void FAST(InputArray image, vector<KeyPoint>& keypoints, int threshold, bool nonmaxSuppression=true)

void FASTX(InputArray image, vector<KeyPoint>& keypoints, int threshold, bool nonmaxSuppression, int type)

image 为输入图像，要求是灰度图像，keypoints 为检测到的特征点向量，threshold 为阈值 t，nonmaxSuppression 为是否进行非极大值抑制，true 表示进行非极大值抑制，type 为选取圆周像素点的个数，是 8（FastFeatureDetector::TYPE_5_8）、12（FastFeatureDetector::TYPE_7_12）还是 16（FastFeatureDetector::TYPE_9_16）。该参数是 FAST 函数和 FASTX 函数的区别，事实上，FAST 函数是调用 FASTX 函数，而传入的 type 值为 FastFeatureDetector::TYPE_9_16。

我们这里将检测到的角点叠加在同一帧图像上，先对图像做一个复制，之后将其中一张图像装换成单通道灰度图像并对其进行角点检测，另一张图像保持不变，完成角点检测之后判断其中某个点是否为角点，如果是角点将其标注为绿色，能清楚的看出叠加效果，如果我们要比较不同与之对于角点检测效果的影响可以对阈值进行适当调整，但阈值不宜超过 10，此时基本检测不到角点

10.2 实验源码

顶层文件参数设置

```
#ifndef DETECTCORNER_H
#define DETECTCORNER_H

#include "hls_stream.h"
#include "ap_int.h"
#include "string.h"

#include "common/xf_common.hpp"
#include "common/xf_utility.hpp"
#include "common/xf_infra.hpp"

#include "imgproc/xf_cvt_color.hpp"
```

```
    #include "xf_fast.hpp"
    #include "imgproc/xf_dilation.hpp"

    typedef ap_axiu<24,1,1,1> pixel_t;
    typedef hls::stream<pixel_t> stream_t;

    #define IMG_MAX_ROWS    1080
    #define IMG_MAX_COLS    1920
    #define NEW_K_ROWS 16
    #define NEW_K_COLS 16

    void detectCorner(unsigned int *pImgRGB888, unsigned int *pCorner, int rows, int cols, int threhold);

    #endif

#endif
```

源文件复制及整合部分

```
    template<int ROWS,int COLS>
    void
xfrgb2gray(xf::cv::Mat<XF_8UC3,ROWS,COLS,XF_NPPC1>&src,xf::cv::Mat<XF_8UC1,ROWS,COLS,XF_NPPC1>&dst1,xf::cv::Mat<
XF_8UC3,ROWS,COLS,XF_NPPC1>&dst2)
    {
      XF_TNAME(XF_8UC3,XF_NPPC1)pixel_src;
      XF_TNAME(XF_8UC1,XF_NPPC1)pixel_dst1;
      XF_TNAME(XF_8UC3,XF_NPPC1)pixel_dst2;
      ap_uint<8>rgb[3];
      ap_uint<8>gray;
      unsigned int i,j=0;
      for(i=0;i<ROWS;i++)
      {
            for(j=0;j<COLS;j++)
            {
                  pixel_src=src.read(i*COLS+j);
                  ExtractPixel(pixel_src,rgb);
                  gray=CalculateGRAY(rgb[0],rgb[1],rgb[2]);
                  pixel_dst1.range(7,0)=gray;
                  pixel_dst2=pixel_src;
                  dst1.write(i*COLS+j,pixel_dst1);
                  dst2.write(i*COLS+j,pixel_dst2);
            }
      }
    }

    template<int ROWS,int COLS>
    void
xfgray2rgb(xf::cv::Mat<XF_8UC1,ROWS,COLS,XF_NPPC1>&src1,xf::cv::Mat<XF_8UC3,ROWS,COLS,XF_NPPC1>&src2,xf::cv::Mat
<XF_8UC3,ROWS,COLS,XF_NPPC1>&dst)
    {
      unsigned int i,j=0;
      XF_TNAME(XF_8UC1,XF_NPPC1)pixel_src1;
      XF_TNAME(XF_8UC3,XF_NPPC1)pixel_src2;
      XF_TNAME(XF_8UC3,XF_NPPC1)pixel_dst;
      for(i=0;i<ROWS;i++)
      {
            for(j=0;j<COLS;j++)
            {
                  pixel_src1=src1.read(i*COLS+j);
                  pixel_src2=src2.read(i*COLS+j);
                  if(pixel_src1==255)
                  {
                  pixel_dst.range(7,0)=0x00;
                  pixel_dst.range(15,8)=pixel_src1;
                  pixel_dst.range(23,16)=0x00;
                  }
                  else
                  {
                        pixel_dst=pixel_src2;
                  }
                  dst.write(i*COLS+j,pixel_dst);
            }
      }
    }
```

同时将 xf_fast.hpp 内的 FastProc 函数改成这段

```
template <int NPC, int WORDWIDTH, int DEPTH, int WIN_SZ, int WIN_SZ_SQ>
void xFfastProc(XF_PTNAME(DEPTH) OutputValues[XF_NPIXPERCYCLE(NPC)],
                XF_PTNAME(DEPTH) src_buf[WIN_SZ][XF_NPIXPERCYCLE(NPC) + (WIN_SZ - 1)],
                ap_uint<8> win_size,
                uchar_t _threshold,
                XF_PTNAME(DEPTH) & pack_corners) {
// clang-format off
    #pragma HLS INLINE
    // clang-format on

    uchar_t kx = 0, ix = 0;

    // XF_SNAME(WORDWIDTH) tbuf_temp;
    XF_PTNAME(DEPTH) tbuf_temp = 0;

    ////////////////////////////////////////////
    // Main code goes here
    // Bresenham's circle score computation
    short int flag_d[(1 << XF_BITSHIFT(NPC))][NUM] = {0}, flag_val[(1 << XF_BITSHIFT(NPC))][NUM] = {0};

// clang-format off
    #pragma HLS ARRAY_PARTITION variable=flag_val dim=1
    #pragma HLS ARRAY_PARTITION variable=flag_d dim=1
    // clang-format on

    for (ap_uint<4> i = 0; i < 1; i++) {
// clang-format off
        #pragma HLS LOOP_TRIPCOUNT MAX=1
        #pragma HLS LOOP_FLATTEN off
        #pragma HLS PIPELINE II=1
        // clang-format on
        // Compute the intensity difference between the candidate pixel and pixels on the Bresenham's circle
        flag_d[i][0] = src_buf[3][3 + i] - src_buf[0][3 + i];  // tbuf4[3+i] - tbuf1[3+i];
        flag_d[i][1] = src_buf[3][3 + i] - src_buf[0][4 + i];  // tbuf4[3+i] - tbuf1[4+i];
        flag_d[i][2] = src_buf[3][3 + i] - src_buf[1][5 + i];  // tbuf4[3+i] - tbuf2[5+i];
        flag_d[i][3] = src_buf[3][3 + i] - src_buf[2][6 + i];  // tbuf4[3+i] - tbuf3[6+i];
        flag_d[i][4] = src_buf[3][3 + i] - src_buf[3][6 + i];  // tbuf4[3+i] - tbuf4[6+i];
        flag_d[i][5] = src_buf[3][3 + i] - src_buf[4][6 + i];  // tbuf4[3+i] - tbuf5[6+i];
        flag_d[i][6] = src_buf[3][3 + i] - src_buf[5][5 + i];  // tbuf4[3+i] - tbuf6[5+i];
        flag_d[i][7] = src_buf[3][3 + i] - src_buf[6][4 + i];  // tbuf4[3+i] - tbuf7[4+i];
        flag_d[i][8] = src_buf[3][3 + i] - src_buf[6][3 + i];  // tbuf4[3+i] - tbuf7[3+i];
        flag_d[i][9] = src_buf[3][3 + i] - src_buf[6][2 + i];  // tbuf4[3+i] - tbuf7[2+i];
        flag_d[i][10] = src_buf[3][3 + i] - src_buf[5][1 + i]; // tbuf4[3+i] - tbuf6[1+i];
        flag_d[i][11] = src_buf[3][3 + i] - src_buf[4][0 + i]; // tbuf4[3+i] - tbuf5[0+i];
        flag_d[i][12] = src_buf[3][3 + i] - src_buf[3][0 + i]; // tbuf4[3+i] - tbuf4[0+i];
        flag_d[i][13] = src_buf[3][3 + i] - src_buf[2][0 + i]; // tbuf4[3+i] - tbuf3[0+i];
        flag_d[i][14] = src_buf[3][3 + i] - src_buf[1][1 + i]; // tbuf4[3+i] - tbuf2[1+i];
        flag_d[i][15] = src_buf[3][3 + i] - src_buf[0][2 + i]; // tbuf4[3+i] - tbuf1[2+i];
        // Repeating the first 9 values
        flag_d[i][16] = flag_d[i][0];
        flag_d[i][17] = flag_d[i][1];
        flag_d[i][18] = flag_d[i][2];
        flag_d[i][19] = flag_d[i][3];
        flag_d[i][20] = flag_d[i][4];
        flag_d[i][21] = flag_d[i][5];
        flag_d[i][22] = flag_d[i][6];
        flag_d[i][23] = flag_d[i][7];
        flag_d[i][24] = flag_d[i][8];

        // Classification of pixels on the Bresenham's circle into brighter, darker or similar w.r.t.
        // the candidate pixel
        for (ap_uint<4> j = 0; j < 8; j++) {
// clang-format off
            #pragma HLS unroll
            // clang-format on
            if (flag_d[i][j] > _threshold)
                flag_val[i][j] = 1;
            else if (flag_d[i][j] < -_threshold)
                flag_val[i][j] = 2;
            else
                flag_val[i][j] = 0;

            if (flag_d[i][j + 8] > _threshold)
                flag_val[i][j + 8] = 1;
            else if (flag_d[i][j + 8] < -_threshold)
                flag_val[i][j + 8] = 2;
            else
                flag_val[i][j + 8] = 0;
            // Repeating the first 9 values
            flag_val[i][j + PSize] = flag_val[i][j];
        }
        flag_val[i][PSize / 2 + PSize] = flag_val[i][PSize / 2];
```

```
            flag_d[i][PSize / 2 + PSize] = flag_d[i][PSize / 2];

            // Bresenham's circle score computation complete

            // Decision making for corners
            uchar_t core = 0;
            uchar_t iscorner = 0;
            uchar_t count = 1;
            for (ap_uint<5> c = 1; c < PSize + PSize / 2 + 1; c++) {
    // clang-format off
                #pragma HLS LOOP_TRIPCOUNT MAX=25
                #pragma HLS UNROLL
                // clang-format on
                if ((flag_val[i][c - 1] == flag_val[i][c]) && flag_val[i][c] > 0) {
                    count++;
                    if (count > PSize / 2) {
                        iscorner = 1; // Candidate pixel is a corner
                    }
                } else {
                    count = 1;
                }
            } // Corner position computation complete
            // NMS Score Computation
            if (iscorner) {
                xFCoreScore(flag_d[i], _threshold, &core);
                pack_corners.range(ix + 7, ix) = 255;
            } else
                pack_corners.range(ix + 7, ix) = 0;
            ix += 8;
            // Pack the 8-bit score values into 64-bit words
            tbuf_temp.range(kx + 7, kx) = core; // Set bits in a range of positions.
            kx += 8;
        }
        // return tbuf_temp;

        OutputValues[0] = pack_corners; // array[(WIN_SZ_SQ)>>1];
        return;
    }
```

源文件顶层函数部分

```
    void detectCorner(hls::stream<ap_axiu<24,1,1,1>>&video_in, hls::stream<ap_axiu<24,1,1,1>>&video_out,int
threshold)
    {
    #pragma HLS INTERFACE ap_vld port=threshold register
    #pragma HLS INTERFACE axis port=video_out register_mode=both register
    #pragma HLS INTERFACE axis port=video_in register_mode=both register
    #pragma HLS INTERFACE ap_ctrl_none port=return
    #pragma HLS DATAFLOW


      xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1> img_in;
    #pragma HLS STREAM variable=img_in.data depth=1920 dim=1
      xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1> img_out;
    #pragma HLS STREAM variable=img_out.data depth=1920 dim=1
      xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1> img_rgb_src;
    #pragma HLS STREAM variable=img_rgb_src.data depth=1920 dim=1
      xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1> img_rgb_dst;
    #pragma HLS STREAM variable=img_rgb_dst.data depth=1920 dim=1
      xf::cv::Mat<XF_8UC1,IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1> img_gray_src;
    #pragma HLS STREAM variable=img_gray_src.data depth=1920 dim=1
      xf::cv::Mat<XF_8UC1,IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1> img_gray_dst;
    #pragma HLS STREAM variable=img_gray_dst.data depth=1920 dim=1


      unsigned char kernel[NEW_K_ROWS][NEW_K_COLS];
      #pragma HLS array_partition variable=kernel dim=0
      // clang-format on
      for (unsigned char i = 0; i < NEW_K_ROWS; i++) {
            for (unsigned char j = 0; j < NEW_K_COLS; j++) {
                  kernel[i][j] = 1; // _kernel[i*NEW_K_COLS+j];
            }
      }

      xf::cv::AXIvideo2xfMat(video_in,img_in);
      xfrgb2gray<IMG_MAX_ROWS,IMG_MAX_COLS>(img_in,img_gray_src,img_rgb_src);
      xf::cv::fast<0,XF_8UC1,IMG_MAX_ROWS,IMG_MAX_COLS,XF_NPPC1>(img_gray_src,img_rgb_src,img_gray_dst,img_rgb_d
st,threshold);
    //      xf::cv::xfdilate<IMG_MAX_ROWS,IMG_MAX_COLS, XF_CHANNELS(XF_8UC1, XF_NPPC1), XF_8UC1, XF_NPPC1, 0,
(IMG_MAX_COLS >> XF_BITSHIFT(XF_NPPC1)) + (NEW_K_ROWS >> 1),
    //                      NEW_K_ROWS,
NEW_K_COLS>(mask,dmask,matGray.rows,matGray.cols>>XF_BITSHIFT(XF_NPPC1),kernel);
    //
      overlyOnMat<IMG_MAX_ROWS,IMG_MAX_COLS>(img_rgb_dst,img_gray_dst,img_out,overly_alpha,overly_x,overly_y,ove
```

```
rly_h,overly_w);
      xfgray2rgb<IMG_MAX_ROWS,IMG_MAX_COLS>(img_gray_dst,img_rgb_dst,img_out);
      xf::cv::xfMat2AXIvideo(img_out,video_out);
}
```

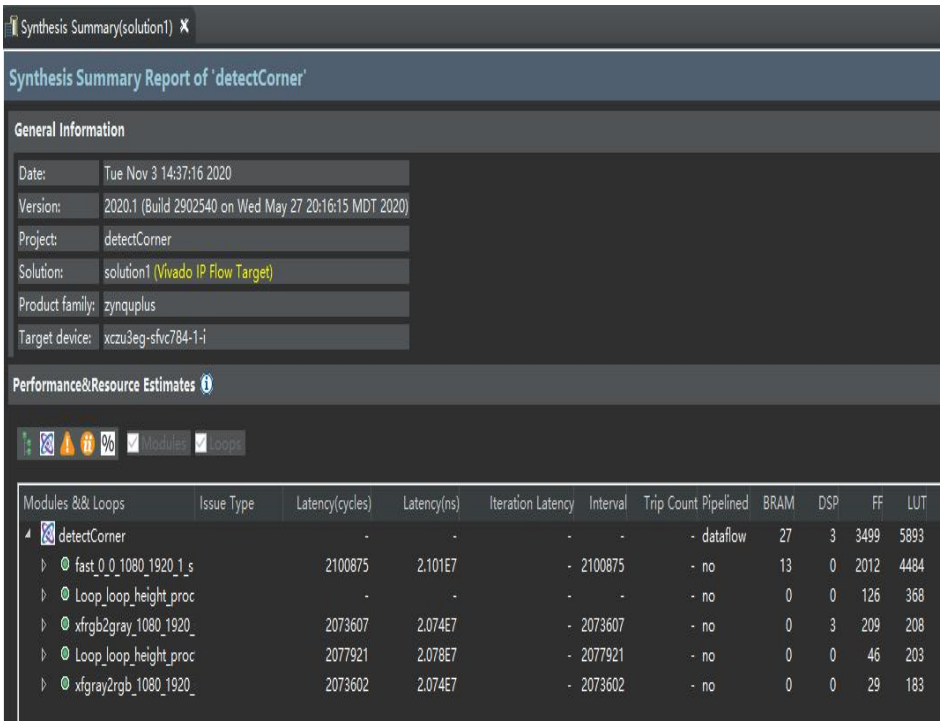
10.3 接口设置与工程优化

接口设置:

接口设置与上一章相似，输入输出因为是连续数据设置为 axi 类型，参数设置为 ap_vld 类型，每一帧送一次，保证图像较好的显示效果

工程优化:

工程优化与上一章相似，为提升图像帧数使用了 Dataflow 优化，要注意使用 Dataflow 优化时不能出现函数输出参数绕过中间某个函数之间做下一函数输入的情况，此时为了达到叠加效果需要对 cv::xf::fast 函数做修改，对其增加输入和输出，在下修改的 xf_fast.hpp 文件在工程中的 source 文件夹内，原先的 xf_fast.hpp 函数只能实现检测焦点的效果达不到叠加的效果。整体优化之后运行时间在 21ms 左右，能满足帧数要求

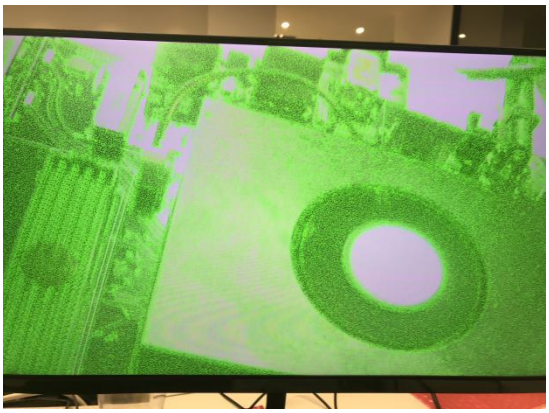


10.4 工程路径

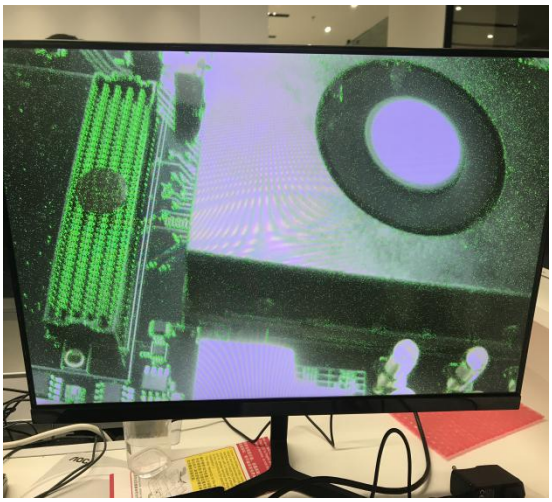
名称	路径
Vivado 工程	vivado/detectCorner
HLS 工程	hls/ detectCorner

10.5 工程实现

烧录之后阈值设置为 0 时显示效果



阈值设置为 5 时显示效果



阈值设置为 8 时显示效果



第十一章 SOBEL 算子实现边缘检测

11.1 实验介绍

Sobel 算子，也被称为 Sobel-Feldman 算子，或者 Sobel 滤波，是在图像处理和计算机视觉得到广泛应用的一种图像边缘检测算法。它由斯坦福大学人工智能实验室（SAIL）的 Irwin Sobel 和 Gray Feldman 而得名。

Sobel 算子使用两个（3x3）矩阵来对原图进行卷积运算以计算出两个方向的灰度差分（偏导）的估计值（一个水平方向、一个竖直方向）。我们假定 A 是原始图像（彩色图像需先转换为灰度图像），Gx 和 Gy 分别是在横向及纵向的灰度偏导的近似值（即两个方向上对原图的平面卷积结果）。数学表达如下：

$$\mathbf{G}_x = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} * \mathbf{A} \quad \text{and} \quad \mathbf{G}_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} * \mathbf{A}$$

对应的计算过程如下：

Gx = [f(x+1,y-1)+2*f(x+1,y)+f(x+1,y+1)] - [f(x-1,y-1)+2*f(x-1,y)+f(x-1,y+1)]

Gy = [f(x-1,y-1) + 2f(x,y-1) + f(x+1,y-1)] - [f(x-1, y+1) + 2*f(x,y+1)+f(x+1,y+1)

上式中，f(x,y)为图像 A 中(x,y)处的灰度值。由此便可以计算出每个点的 Gx 和 Gy。

对于图像中的每个点，其梯度的估计值 G 便可以通过两个方向的梯度 Gx 和 Gy 借由下式得出：

$$\mathbf{G} = \sqrt{\mathbf{G}_x^2 + \mathbf{G}_y^2}$$

11.2 实验源码

头文件参数定义

```
#ifndef __edge_detector_h__
#define __edge_detector_h__

#include "ap_axi_sdata.h"
#include "hls_stream.h"
#include "ap_int.h"

#include "common/xf_common.hpp"
```

```
    #include "common/xf_utility.hpp"
    #include "common/xf_infra.hpp"

    #include "imgproc/xf_cvt_color.hpp"
    #include "imgproc/xf_sobel.hpp"

    typedef ap_axiu<24, 1, 1, 1> us_pixel_t;
    typedef hls::stream<us_pixel_t> ustream_t;

    #define IMG_MAX_ROWS 1080
    #define IMG_MAX_COLS 1920

    void edge_detector(ustream_t &src, ustream_t &dst, unsigned char threshold);

#endif
```

源文件主要部分

```
    template<int ROWS,int COLS>
    void thresholding(xf::cv::Mat<XF_8UC1,IMG_MAX_ROWS, IMG_MAX_COLS,XF_NPPC1> &src,
           xf::cv::Mat<XF_8UC1,IMG_MAX_ROWS, IMG_MAX_COLS,XF_NPPC1> &dst,
           ap_uint<8>&threshold)
    {
      XF_TNAME(XF_8UC1,XF_NPPC1) pixel_src, pixel_dst;
      unsigned int i,j=0;
      ap_uint<8>value;
      if(threshold==0)
      {
            for(i=0;i<ROWS;i++)
            {
                  for(j=0;j<COLS;j++)
                  {
    #pragma HLS PIPELINE
                        pixel_src=src.read(i*COLS+j);
                        dst.write(i*COLS+j,pixel_src);
                  }
            }
      }
      else
      {
            for(i=0;i<ROWS;i++)
            {
                  for(j=0;j<COLS;j++)
                  {
    #pragma HLS PIPELINE
                        pixel_src=src.read(i*COLS+j);
                        if(pixel_src>threshold)
                        {
                              pixel_dst=0;
                        }
                        else
                        {
                              pixel_dst=255;
                        }
                        dst.write(i*COLS+j,pixel_dst);
                  }
            }
      }
}


    template<int ROWS,int COLS>
    void thresholding(xf::cv::Mat<XF_8UC1,IMG_MAX_ROWS, IMG_MAX_COLS,XF_NPPC1> &src,
           xf::cv::Mat<XF_8UC1,IMG_MAX_ROWS, IMG_MAX_COLS,XF_NPPC1> &dst,
           ap_uint<8>&threshold)
    {
      XF_TNAME(XF_8UC1,XF_NPPC1) pixel_src, pixel_dst;
      unsigned int i,j=0;
      ap_uint<8>value;
      if(threshold==0)
      {
            for(i=0;i<ROWS;i++)
            {
                  for(j=0;j<COLS;j++)
                  {
    #pragma HLS PIPELINE
                        pixel_src=src.read(i*COLS+j);
                        dst.write(i*COLS+j,pixel_src);
                  }
            }
      }
      else
      {
            for(i=0;i<ROWS;i++)
```

```
            {
                for(j=0;j<COLS;j++)
                {
    #pragma HLS PIPELINE
                    pixel_src=src.read(i*COLS+j);
                    if(pixel_src>threshold)
                    {
                        pixel_dst=0;
                    }
                    else
                    {
                        pixel_dst=255;
                    }
                    dst.write(i*COLS+j,pixel_dst);
                }
            }
        }
}
```

源文件顶层函数部分

```
    void edge_detector(ustream_t &src, ustream_t &dst, ap_uint<8> threshold)
    {
    #pragma HLS DATAFLOW
    #pragma HLS INTERFACE axis port=src
    #pragma HLS INTERFACE axis port=dst
    #pragma HLS INTERFACE ap_vld port=threshold register
    #pragma HLS INTERFACE ap_ctrl_none port=return

      xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS, IMG_MAX_COLS,XF_NPPC1> srcImg;
    #pragma HLS stream variable=srcImg.data depth=1920 dim=1
      xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS, IMG_MAX_COLS,XF_NPPC1> rgbSobel;
    #pragma HLS stream variable=rgbSobel.data depth=1920 dim=1
      xf::cv::Mat<XF_8UC1,IMG_MAX_ROWS, IMG_MAX_COLS,XF_NPPC1> grayImg;
    #pragma HLS stream variable=grayImg.data depth=1920 dim=1
      xf::cv::Mat<XF_8UC1,IMG_MAX_ROWS, IMG_MAX_COLS,XF_NPPC1> sobelImg_x;
    #pragma HLS stream variable=sobelImg_x.data depth=1920 dim=1
      xf::cv::Mat<XF_8UC1,IMG_MAX_ROWS, IMG_MAX_COLS,XF_NPPC1> sobelImg_y;
    #pragma HLS stream variable=sobelImg_y.data depth=1920 dim=1
      xf::cv::Mat<XF_8UC1,IMG_MAX_ROWS, IMG_MAX_COLS,XF_NPPC1> sobelImg;
    #pragma HLS stream variable=sobelImg.data depth=1920 dim=1
      xf::cv::Mat<XF_8UC1,IMG_MAX_ROWS, IMG_MAX_COLS,XF_NPPC1> thresholdImg;
    #pragma HLS stream variable=thresholdImg.data depth=1920 dim=1

      xf::cv::AXIvideo2xfMat(src, srcImg);
      xfrgb2gray<IMG_MAX_ROWS,IMG_MAX_COLS>(srcImg, grayImg);
        xf::cv::xFSobelFilter3x3<XF_8UC1, XF_8UC1,IMG_MAX_ROWS, IMG_MAX_COLS, XF_CHANNELS(XF_8UC1,XF_NPPC1),
XF_DEPTH(XF_8UC1,XF_NPPC1), XF_DEPTH(XF_8UC1,XF_NPPC1),
                    XF_NPPC1, XF_WORDWIDTH(XF_8UC1,XF_NPPC1), XF_WORDWIDTH(XF_8UC1,XF_NPPC1), (IMG_MAX_COLS >>
XF_BITSHIFT(XF_NPPC1)),false>(
          grayImg,sobelImg_x,sobelImg_y,grayImg.rows,grayImg.cols>>XF_BITSHIFT(XF_NPPC1));
      AddWeightedKernel<IMG_MAX_ROWS,IMG_MAX_COLS>(sobelImg_x,0.5f,sobelImg_y,0.5f,0.0f,sobelImg);
      thresholding<IMG_MAX_ROWS,IMG_MAX_COLS>(sobelImg, thresholdImg,threshold);
      xfgray2rgb<IMG_MAX_ROWS,IMG_MAX_COLS>(thresholdImg, rgbSobel);
      xf::cv::xfMat2AXIvideo(rgbSobel, dst);
}
```

11.3 接口综合与工程优化

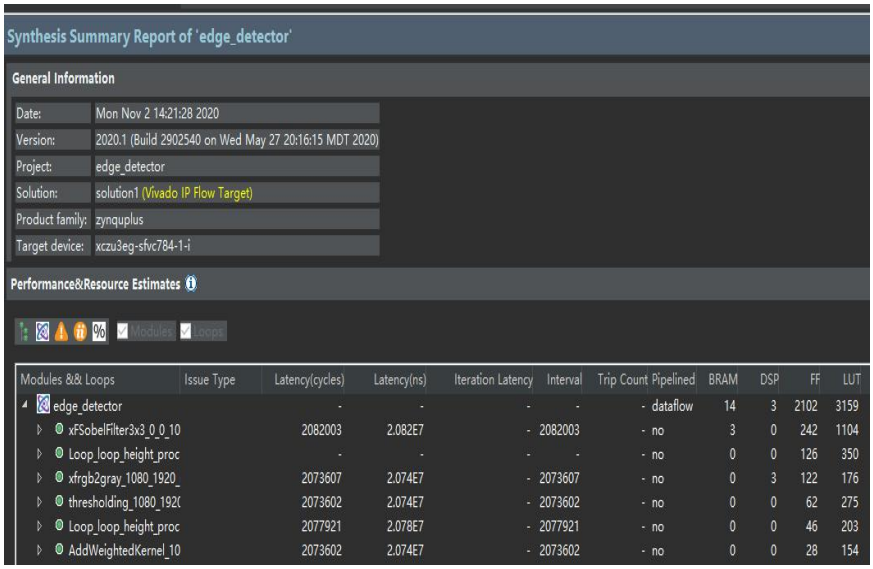
接口综合：

接口设置与之前相同,Block Level Interface 设置为 ap_none 类型，输入输出参数因为是连续变化的数据设置为 axi 类型，阈值设置参数设置为 ap_vld 类型，每帧送入一次，达到较好的显示效果。

工程优化：

工程优化与上一章类似，注意由于 threshold 值为 0 时输出结果比较特殊，对其现在循环之外判断是否为 0，减少循环中的参数判断，提升运行速度，由于使用了 Dataflow，为了避免出现

deadlock 死机的情况，我们要尽量保证函数中的延时尽量接近，整体工程运行延时在 20ms 左右能满足帧数要求

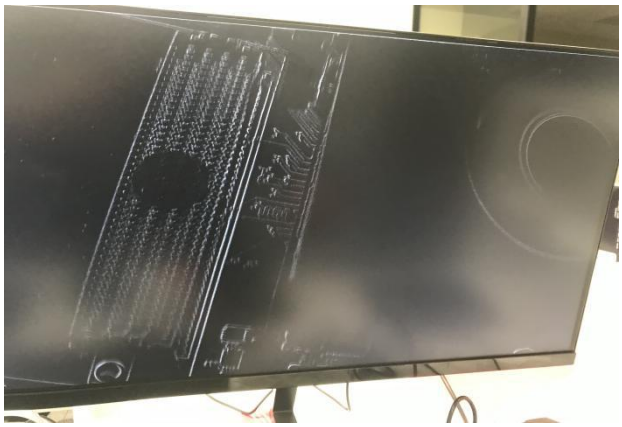


11.4 工程路径

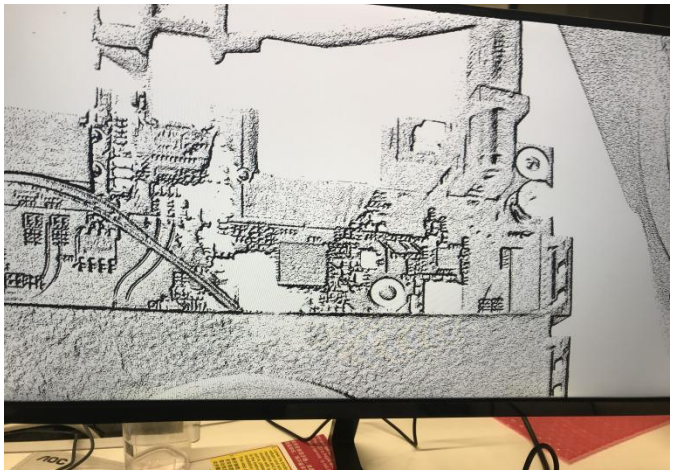
名称	路径
Vivado 工程	vivado/edge_detector
HLS 工程	hls/ edge_detector

11.5 工程实现

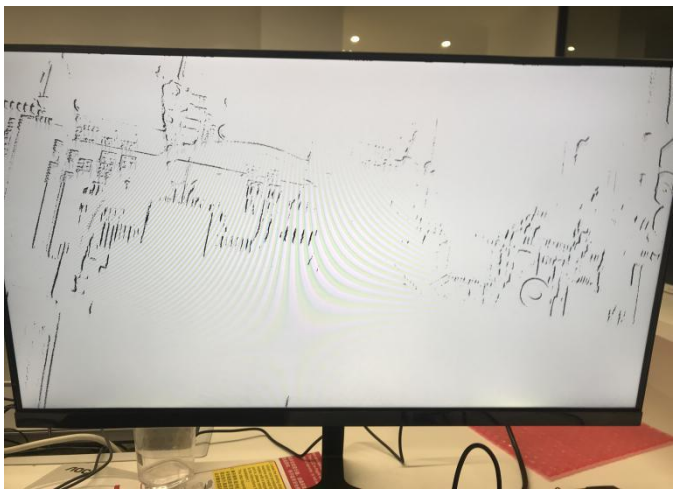
烧录之后当阈值为 0 时显示效果



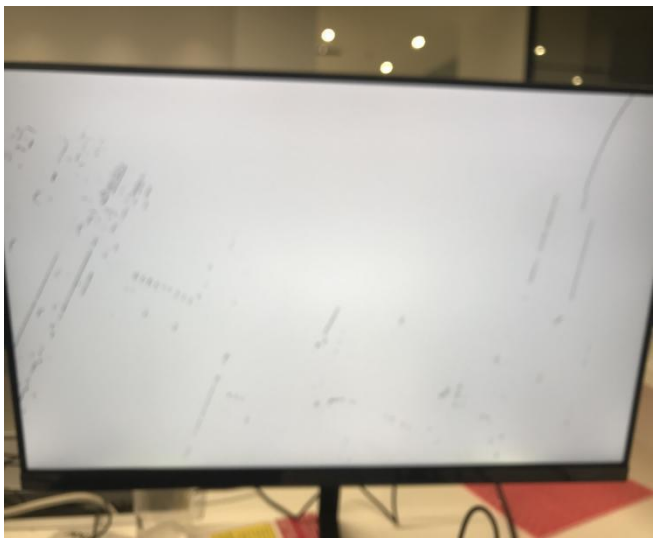
当阈值为 10 时显示效果



当阈值为 50 时显示效果



当阈值为 100 时显示效果



第十二章 Canny 算子实现边缘检测

12.1 实验介绍

相较于前一章的 Sobel 算子实现边缘检测，Canny 算子对噪声抗干扰性强，对于若边缘检测更为准确，经过 canny 算子勾勒出来的边框更为细致，能清晰的显示画面中的弱边缘，其运算主要分为以下几步，首先对图像做 3x3 高斯滤波，之后对画面进行做一阶偏导有限差分计算梯度幅值和方向，完成后作非最大值的抑制去除虚假边缘，最后进行双阈值判断连接边缘，实验中做梯度幅值和方向计算通过 sobel 实现，通常我们所说的边缘点就是灰度值变化较大的点，得到这些点之后在周边寻找最大值点，双阈值判断指的是如果目标点小于较小阈值认为其不是边缘将其除去，如果大于较小阈值小于较大阈值对该点置一个在 0 到 255 之间的数，认为其为弱边缘，如果大于较大阈值对其置 0 将该点涂成黑色，认为其属于强边缘。

12.2 实验源码：

头文件参数定义

```
#ifndef __edge_canny_detector_h__
#define __edge_canny_detector_h__

#include "ap_int.h"
#include "hls_stream.h"
#include "ap_axi_sdata.h"

#include "common/xf_common.hpp"
#include "common/xf_infra.hpp"
#include "common/xf_utility.hpp"

#include "xf_canny.hpp"
#include "imgproc/xf_cvt_color.hpp"

typedef ap_axiu<24,1,1,1> pixel;
typedef hls::stream<pixel> pixel_t ;

#define IMG_MAX_ROWS 1080
#define IMG_MAX_COLS 1920
#define FILTER_TYPE 3

void edge_canny_detector(pixel_t &src,pixel_t &dst,ap_int<32> lowthreshold,ap_int<32> highthreshold);

#endif
```

源文件主要部分

```
template<int ROWS,int COLS>
void xfrgb2gray(xf::cv::Mat<XF_8UC3,ROWS,COLS,XF_NPPC1> &src,xf::cv::Mat<XF_8UC1,ROWS,COLS,XF_NPPC1> &dst)
{
  XF_TNAME(XF_8UC3,XF_NPPC1)rgb_packed;
  XF_TNAME(XF_8UC1,XF_NPPC1)gray_packed;
  ap_uint<8>rgb[3];
  ap_uint<8>gray;
  unsigned int i,j=0;
  for(i=0;i<ROWS;i++)
  {
        for(j=0;j<COLS;j++)
        {
#pragma HLS PIPELINE
              rgb_packed=src.read(i*COLS+j);
              ExtractPixel(rgb_packed,rgb);
              gray=CalculateGRAY(rgb[0],rgb[1],rgb[2]);
              gray_packed.range(7,0)=gray;
```

```
                dst.write(i*COLS+j,gray_packed);
            }
        }
}

    template<int ROWS,int COLS>
    void xfgray2rgb(xf::cv::Mat<XF_8UC1,ROWS,COLS,XF_NPPC1> &src,xf::cv::Mat<XF_8UC3,ROWS,COLS,XF_NPPC1> &dst)
    {
    #pragma HLS INLINE off
      XF_TNAME(XF_8UC3,XF_NPPC1)rgb_packed;
      XF_TNAME(XF_8UC1,XF_NPPC1)gray_packed;
      ap_uint<8>rgb[3];
      ap_uint<8>gray;
      unsigned i,j,k=0;
      for(i=0;i<ROWS;i++)
      {
            for(j=0;j<COLS;j++)
            {
    #pragma HLS PIPELINE
                gray_packed=src.read(i*COLS+j);
                gray=gray_packed.range(7,0);
                rgb_packed.range(7,0)=gray;
                rgb_packed.range(15,8)=gray;
                rgb_packed.range(23,16)=gray;
                dst.write(i*COLS+j,rgb_packed);
            }
      }
    }

    template<int ROWS,int COLS>
    void
duplicate(xf::cv::Mat<XF_16SC1,ROWS,COLS,XF_NPPC1>&src1,xf::cv::Mat<XF_16SC1,ROWS,COLS,XF_NPPC1>&src2,xf::cv::Ma
t<XF_16SC1,ROWS,COLS,XF_NPPC1> &dst1,xf::cv::Mat<XF_16SC1,ROWS,COLS,XF_NPPC1>
&dst2,xf::cv::Mat<XF_16SC1,ROWS,COLS,XF_NPPC1> &dst3,xf::cv::Mat<XF_16SC1,ROWS,COLS,XF_NPPC1> &dst4)
    {
      XF_TNAME(XF_16SC1,XF_NPPC1)pixel_src1;
      XF_TNAME(XF_16SC1,XF_NPPC1)pixel_src2;
      unsigned int i,j=0;
      for(i=0;i<ROWS;i++)
      {
            for(j=0;j<COLS;j++)
            {
    #pragma HLS PIPELINE
                pixel_src1=src1.read(i*COLS+j);
                pixel_src2=src2.read(i*COLS+j);
                dst1.write(i*COLS+j,pixel_src1);
                dst2.write(i*COLS+j,pixel_src1);
                dst3.write(i*COLS+j,pixel_src2);
                dst4.write(i*COLS+j,pixel_src2);
            }
      }
}
```

源文件顶层函数部分

```
    void edge_canny_detector(pixel_t &src,pixel_t &dst,ap_int<8>&lowthreshold,ap_int<8>&highthreshold)
    {

    #pragma HLS INTERFACE ap_ctrl_none port=return
    #pragma HLS INTERFACE ap_vld port=highthreshold register
    #pragma HLS INTERFACE ap_vld port=lowthreshold register
    #pragma HLS INTERFACE axis port=dst register_mode=both depth=16 register
    #pragma HLS INTERFACE axis port=src register_mode=both depth=16 register
    #pragma HLS DATAFLOW
      xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS,IMG_MAX_COLS,XF_NPPC1>rgb_img_src;
    #pragma HLS STREAM variable=rgb_img_src.data depth=1920 dim=1
      xf::cv::Mat<XF_8UC1,IMG_MAX_ROWS,IMG_MAX_COLS,XF_NPPC1>gray_img_src;
    #pragma HLS STREAM variable=gray_img_src.data depth=1920 dim=1
      xf::cv::Mat<XF_8UC1,IMG_MAX_ROWS,IMG_MAX_COLS,XF_NPPC1>gray_img_dst;
    #pragma HLS STREAM variable=gray_img_dst.data depth=1920 dim=1
      xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS,IMG_MAX_COLS,XF_NPPC1>rgb_img_dst;
    #pragma HLS STREAM variable=rgb_img_dst.data depth=1920 dim=1

      int img_height=rgb_img_src.rows;
      int img_width=rgb_img_src.cols;
        xf::cv::Mat<XF_8UC1, IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1> gaussian_mat(img_height, img_width);
    #pragma HLS STREAM variable=gaussian_mat.data depth=1920 dim=1
        xf::cv::Mat<XF_16SC1, IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1> gradx_mat(img_height, img_width);
    #pragma HLS STREAM variable=gradx_mat.data depth=1920 dim=1
        xf::cv::Mat<XF_16SC1, IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1> gradx1_mat(img_height, img_width);
    #pragma HLS STREAM variable=gradx1_mat.data depth=1920 dim=1
        xf::cv::Mat<XF_16SC1, IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1> gradx2_mat(img_height, img_width);
    #pragma HLS STREAM variable=gradx2_mat.data depth=1920 dim=1
        xf::cv::Mat<XF_16SC1,IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1> grady_mat(img_height, img_width);
```

```
    #pragma HLS STREAM variable=grady_mat.data depth=1920 dim=1
        xf::cv::Mat<XF_16SC1, IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1> grady1_mat(img_height, img_width);
    #pragma HLS STREAM variable=grady1_mat.data depth=1920 dim=1
        xf::cv::Mat<XF_16SC1, IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1> grady2_mat(img_height, img_width);
    #pragma HLS STREAM variable=grady2_mat.data depth=1920 dim=1
        xf::cv::Mat<XF_16SC1, IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1, 1920*3> magnitude_mat(img_height, img_width);
    #pragma HLS STREAM variable=magnitude_mat.data depth=5760 dim=1
        xf::cv::Mat<XF_8UC1, IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1, 1920*3> phase_mat(img_height, img_width);
    #pragma HLS STREAM variable=phase_mat.data depth=5760 dim=1
        xf::cv::Mat<XF_8UC1, IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1> nms_mat(img_height, img_width);
    #pragma HLS STREAM variable=nms_mat.data depth=1920 dim=1

    // clang-format off
    // clang-format on

    // clang-format off

        // clang-format on

      xf::cv::AXIvideo2xfMat(src,rgb_img_src);
      xfrgb2gray<IMG_MAX_ROWS,IMG_MAX_COLS>(rgb_img_src,gray_img_src);
        xFAverageGaussianMask3x3<XF_8UC1, XF_8UC1, IMG_MAX_ROWS, IMG_MAX_COLS, XF_DEPTH(XF_8UC1,XF_NPPC1),
XF_NPPC1, XF_WORDWIDTH(XF_8UC1,XF_NPPC1), (IMG_MAX_COLS >> XF_BITSHIFT(XF_NPPC1))>(
              gray_img_src, gaussian_mat, img_height, img_width);
        xFSobel<XF_8UC1, XF_16SC1, IMG_MAX_ROWS, IMG_MAX_COLS, XF_DEPTH(XF_8UC1,XF_NPPC1),
XF_DEPTH(XF_16SC1,XF_NPPC1),XF_NPPC1,XF_WORDWIDTH(XF_8UC1,XF_NPPC1), XF_WORDWIDTH(XF_16SC1,XF_NPPC1), 3, false>(
            gaussian_mat, gradx_mat, grady_mat, XF_BORDER_REPLICATE, img_height, img_width);
        duplicate<IMG_MAX_ROWS,IMG_MAX_COLS>(gradx_mat,grady_mat,gradx1_mat,gradx2_mat,grady1_mat,grady2_mat);
        xf::cv::magnitude<1, XF_16SC1,XF_16SC1, IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1, 1920*3>(gradx1_mat,
grady1_mat, magnitude_mat);
        xFAngle<XF_16SC1, XF_8UC1, IMG_MAX_ROWS, IMG_MAX_COLS, XF_16SP, XF_8UP, XF_NPPC1, XF_16UW, XF_8UW, 1920*3>(
             gradx2_mat, grady2_mat, phase_mat, img_height, img_width);
        xFSuppression3x3<XF_16SC1, XF_8UC1, XF_8UC1, IMG_MAX_ROWS, IMG_MAX_COLS, XF_16SP, XF_8UP, XF_8UP, XF_NPPC1,
XF_16UW, XF_8UW, XF_8UW, (IMG_MAX_COLS >> XF_BITSHIFT(XF_NPPC1)), 1920*3, 1920*3> (magnitude_mat, phase_mat, nms_mat,
lowthreshold,highthreshold, img_height, img_width);
    //    nPackNMS<IMG_MAX_ROWS,IMG_MAX_COLS>(nms_mat,gray_img_dst);

    //    AddWeightedKernel<IMG_MAX_ROWS,IMG_MAX_COLS>(gradx_mat,0.5f,grady_mat,0.5f,0.0f,gray_img_dst);
      xfgray2rgb<IMG_MAX_ROWS,IMG_MAX_COLS>(nms_mat,rgb_img_dst);
      xf::cv::xfMat2AXIvideo(rgb_img_dst,dst);
}
```

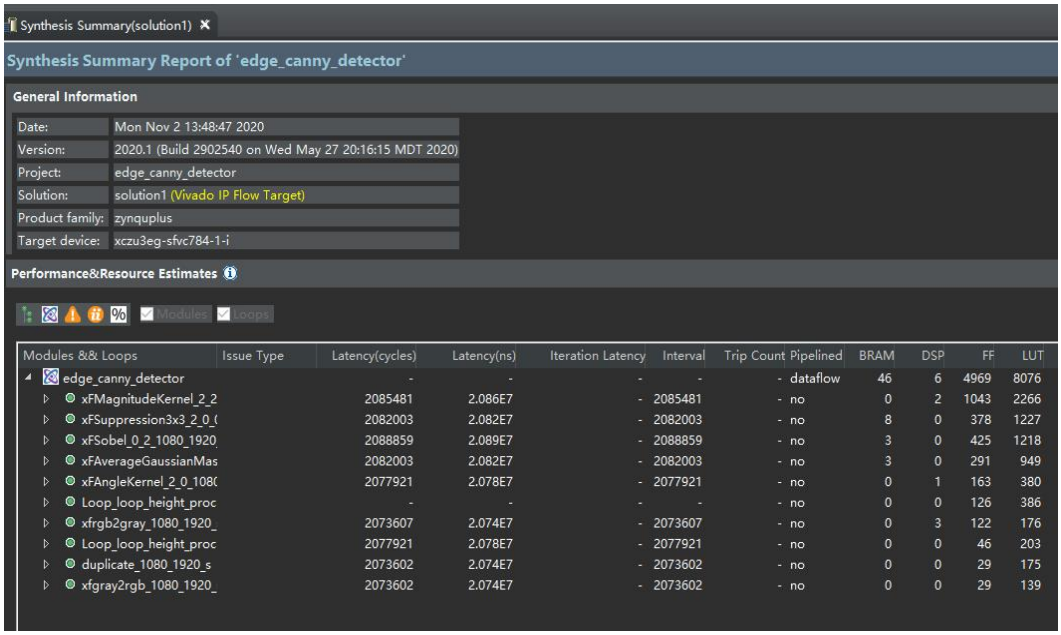
12.3 接口综合和工程优化

接口综合：

对于接口的设置和之前章节所说相同，Block Level Interface 设置为 ap_ctrl_hs 消除多余的关联信号，输入输出参数因为是连续变化的数据将其设置为 axi 类型接口，low_threshold 和 high_threshold 设置为 ap_vld 类型接口，有效信号和帧起始信号相接，如果在一帧画面运行到一般时送入阈值会导致画面上下边缘显示不一致

工程优化：

整体优化方式与上一章类似使用 Dataflow 执行函数，在下一开始在运行 xfopencv 库自带的 xf_canny 函数时出现画面彩条现实的情况，对此对 xf_canny 函数相关的 hpp 文件进行了修改，程序中使用的 xf_canny.hpp 文件中的函数是经过重写之后的文件，在下将其放在 source 文件夹内，有三个文件夹均进行了对应修改，验证显示效果正常，整体函数运行时间在 20ms 左右能满足帧数要求：



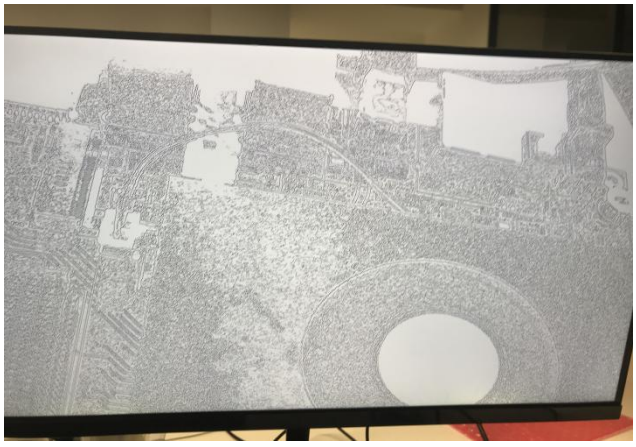
Modules && Loops	Issue Type	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT
edge_canny_detector		-	-	-	-	-	dataflow	46	6	4969	8076
xFMagnitudeKernel_2_2		2085481	2.086E7	-	2085481	-	no	0	2	1043	2266
xFSuppression3x3_2_0_(		2082003	2.082E7	-	2082003	-	no	8	0	378	1227
xFSobel_0_2_1080_1920_		2088859	2.089E7	-	2088859	-	no	3	0	425	1218
xFAverageGaussianMas		2082003	2.082E7	-	2082003	-	no	3	0	291	949
xFAngleKernel_2_0_108(		2077921	2.078E7	-	2077921	-	no	0	1	163	380
Loop_loop_height_proc		-	-	-	-	-	no	0	0	126	386
xfrgb2gray_1080_1920_		2073607	2.074E7	-	2073607	-	no	0	3	122	176
Loop_loop_height_proc		2077921	2.078E7	-	2077921	-	no	0	0	46	203
duplicate_1080_1920_s		2073602	2.074E7	-	2073602	-	no	0	0	29	175
xfgray2rgb_1080_1920_		2073602	2.074E7	-	2073602	-	no	0	0	29	139

12.4 工程路径

名称	路径
Vivado 工程	vivado/edge_canny
HLS 工程	hls/ edge_canny_detector

12.5 工程实现

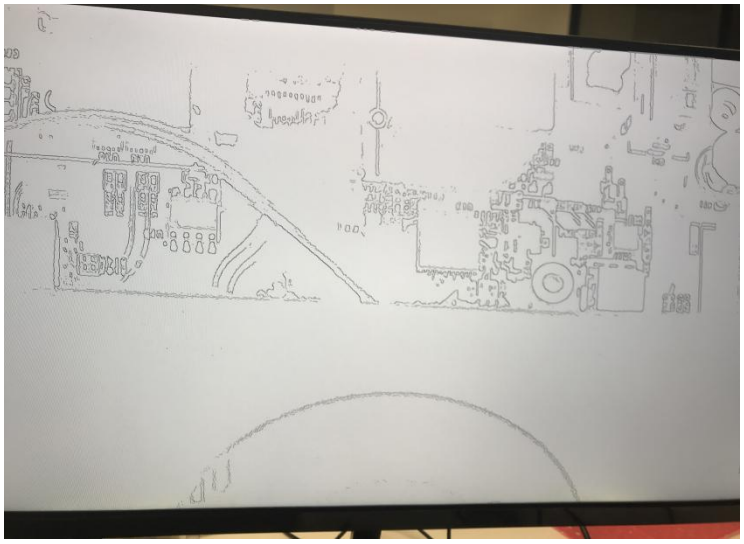
低阈值设置为 10，高阈值设置为 20 时显示画面



低阈值设置为 20，高阈值设置为 30 时显示画面



低阈值设置为 50，高阈值设置为 100 时显示画面

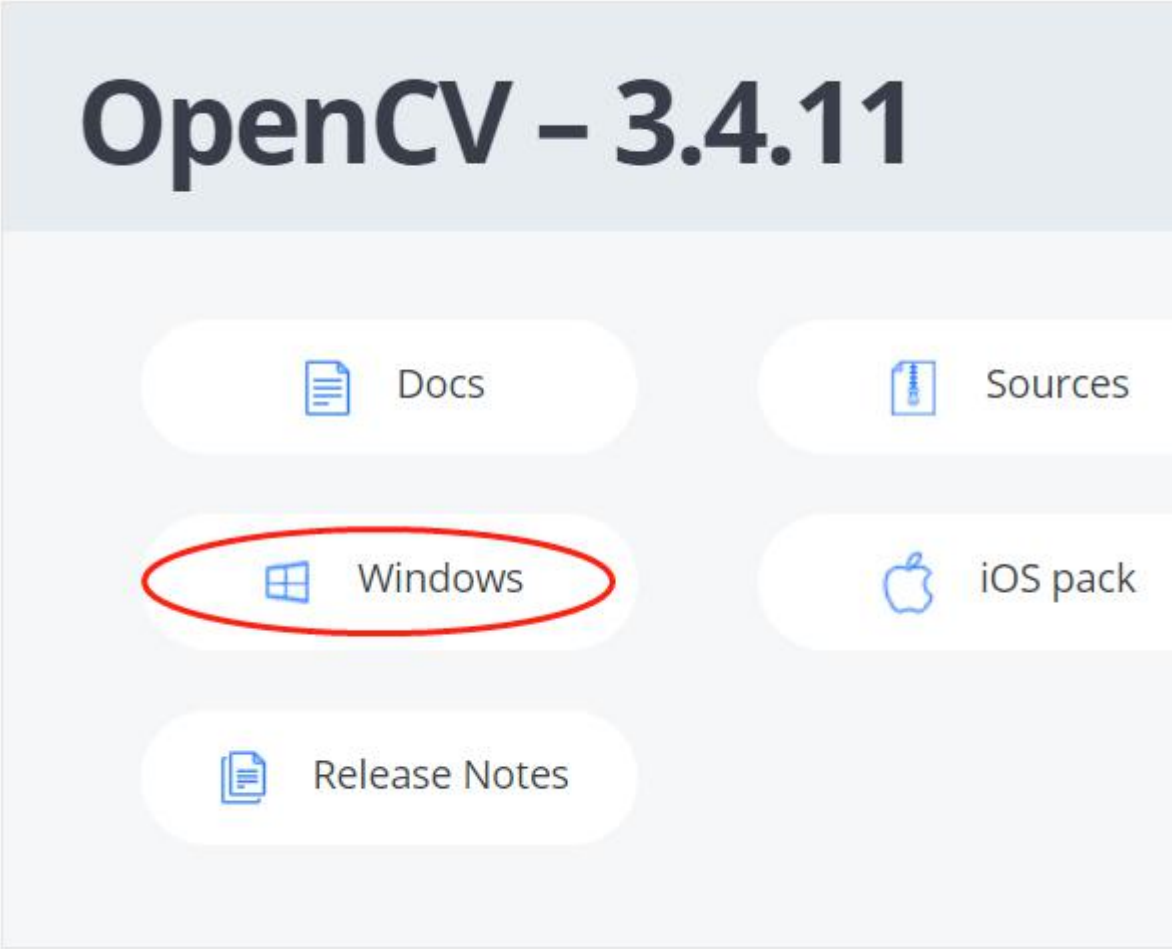


第十三章 如何用 vitis HLS 实现 opencv 仿真

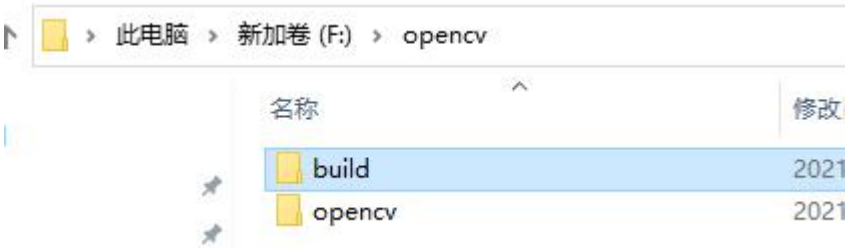
13.1 opencv 仿真环境搭建

在 2020.1 以上的版本中赛灵思去除了 opencv 的库函数文件，对于图像的仿真比之前的版本有很大的不同，如果要进行图像仿真需要先手动搭建一个 opencv 的仿真环境，将相应的库函数安装好，否则无法通过编译，虽然不影响综合也能在板卡实现功能，但是无法仿真的问题会对开发有很大的不便，我们根据以下步骤对 opencv 仿真环境搭建

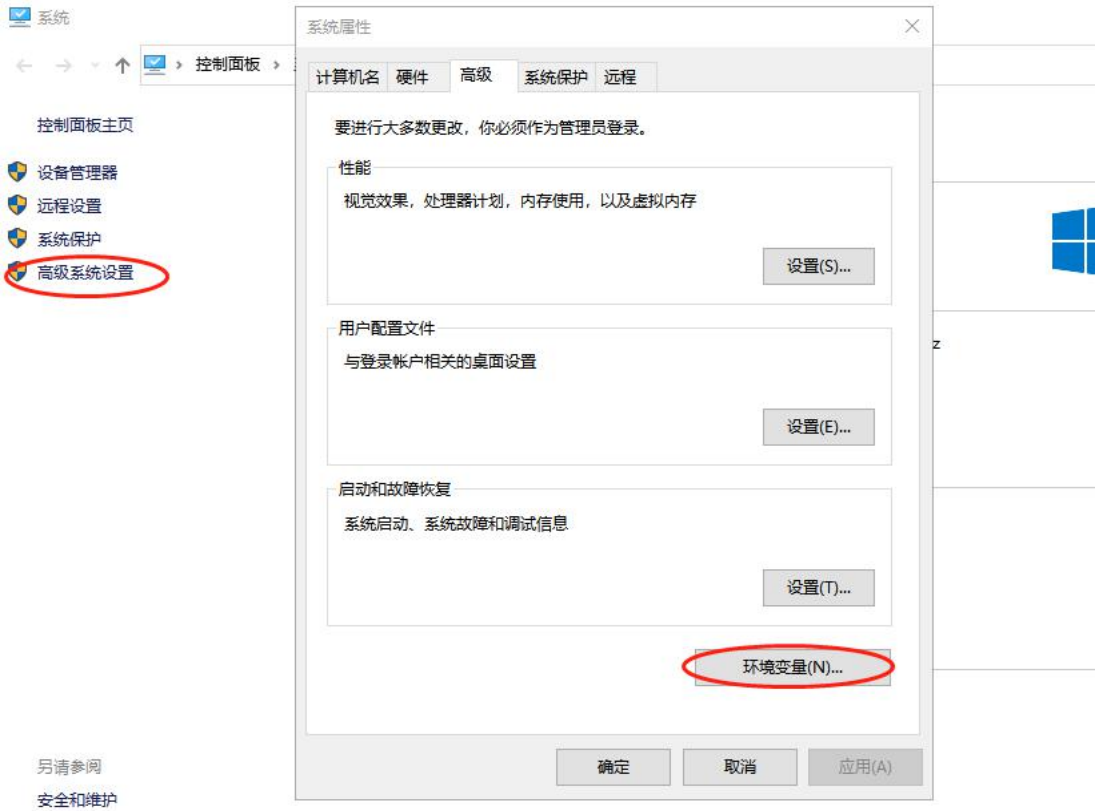
1、登录 opencv 官网：https://opencv.org/releases/ 安装 opencv3.4.11



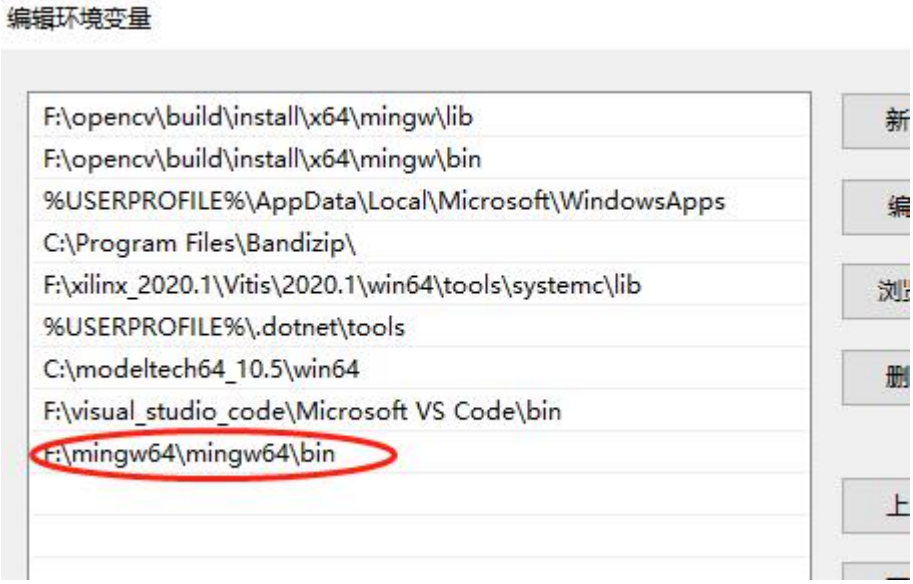
选择 windows,下载 windows 版本,下载完之后新建一个文件夹将文件解压在里面，并在新建的文件夹内再新建一个 build



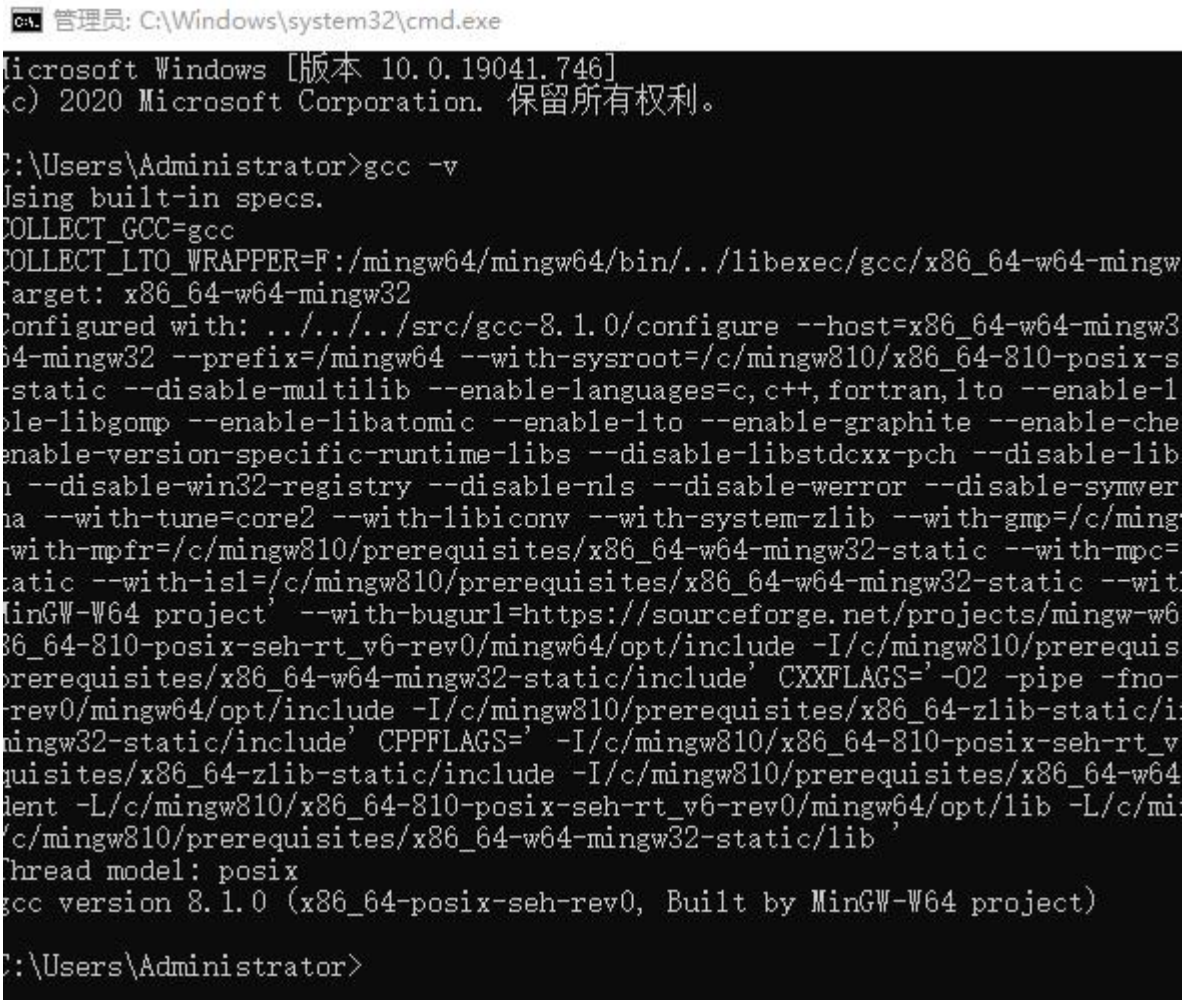
3、完成之后配置环境变量，我的电脑右键属性->高级系统设置->环境变量



2、在 path 中添加安装的 mingGW-GW64 路径



3、打开 cmd,输入 gcc -v，显示如下画面说明配置成功

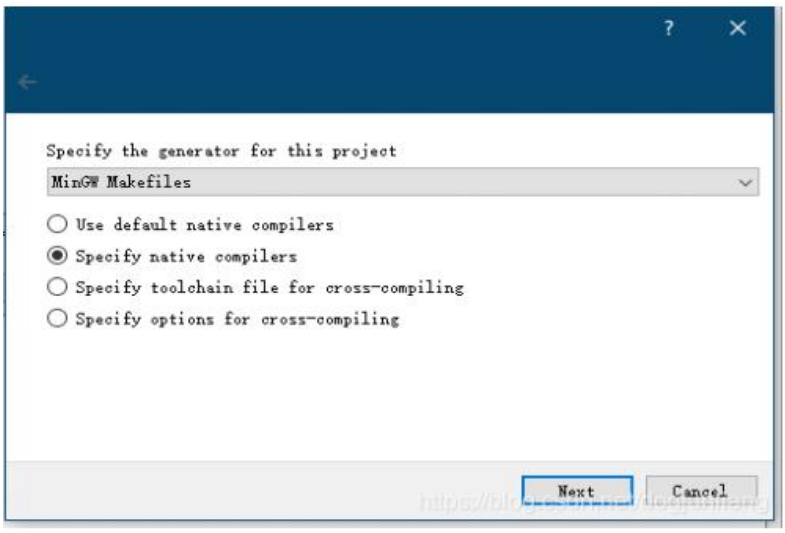


4、下载安装 Cmake 网址如下：https://cmake.org/download/ ,安装过程中的选项都不需要修改

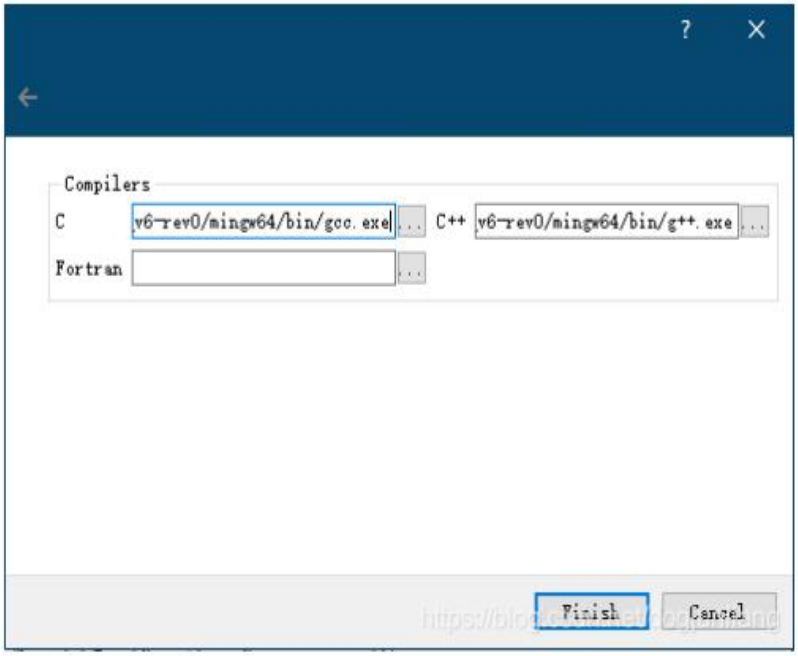
5、完成之后打开 cmake-gui，在 where is the source code 选择安装的 opencv 的 source 文件夹路径，在 where to build the binary 选择第一步新建的 build 文件夹路径：



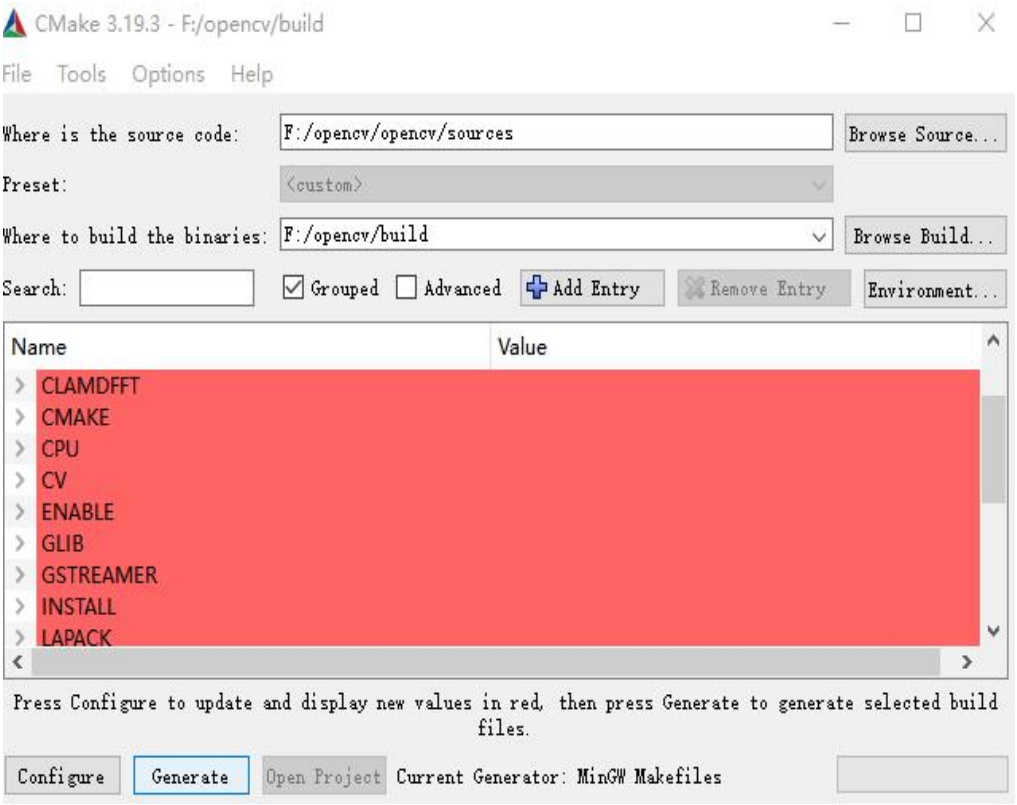
6、点击 configure，按下图的方式进行设置



编译器选择安装的 mingGW-GW-64 内的编译器



7、单击 configure，会看到如下界面



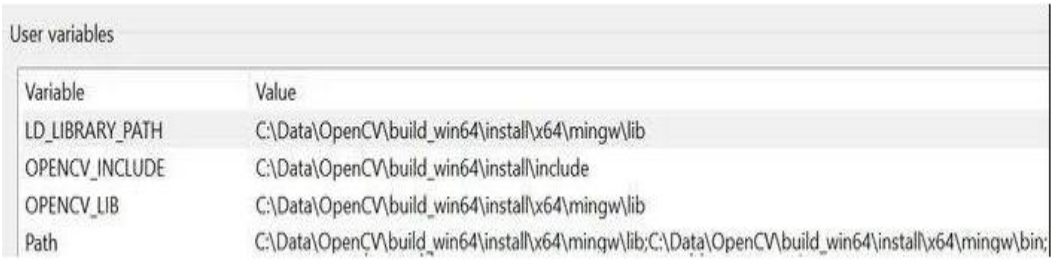
其中勾选 ENABLE_CXX11,勾选 WITH_OPENGL,不勾选 OPENCV_ENABLE_ALLOCATOR_STATS,其余部分保持默认

8、单击 configure,出现 config done 之后单击 Generate，出现 generate done 之后将 cmake-gui 关闭

9、右键开始菜单，输入 cmd,在 windows 下输入 cd + 文件夹路径，进入第一步新建的 build 文件夹，进入之后输入 mingw32-make,开始进行编译。

当编译完成之后输入 mingw32-make install,之后会在 build 文件夹内生成一个 install 文件夹。

10、关闭 cmd，按照第 4 步的步骤打开环境变量，新建三个环境变量，path 中添加两个路径，配置方式如图所示：



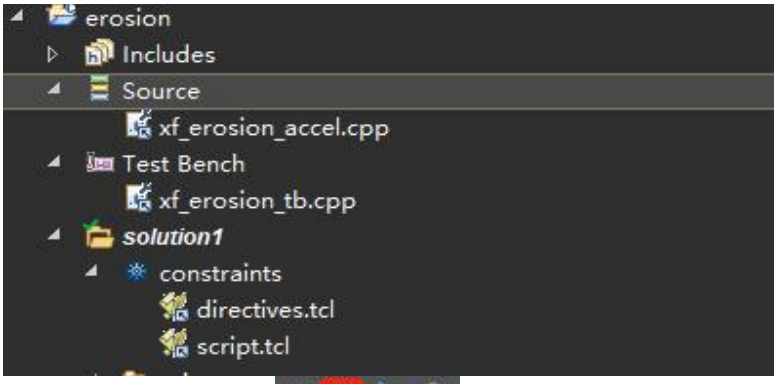
11、重启电脑此时我们的 opencv 环境已经搭建完成

13.2 Vitis HLS 配置

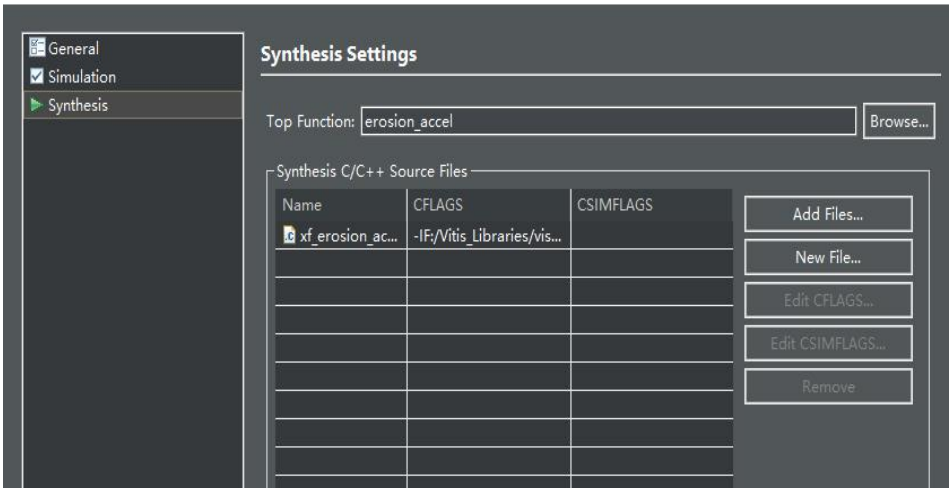
Vitis hls 按照如下方式进行配置:

1、新建一个工程，命名为 erosion，步骤按照 1.1 所述，完成之后将 opencv 库函数压缩

包内的 erosion 文件夹部分的文件复制到 source 文件夹内，将 build 文件夹内的 xf_config_param.h 复制到外面，之后将 xf_erosion_accel.cpp 文件添加到 source 内，之后将 xf_erosion_tb.cpp 添加进 test bench 内，最终效果如图所示：

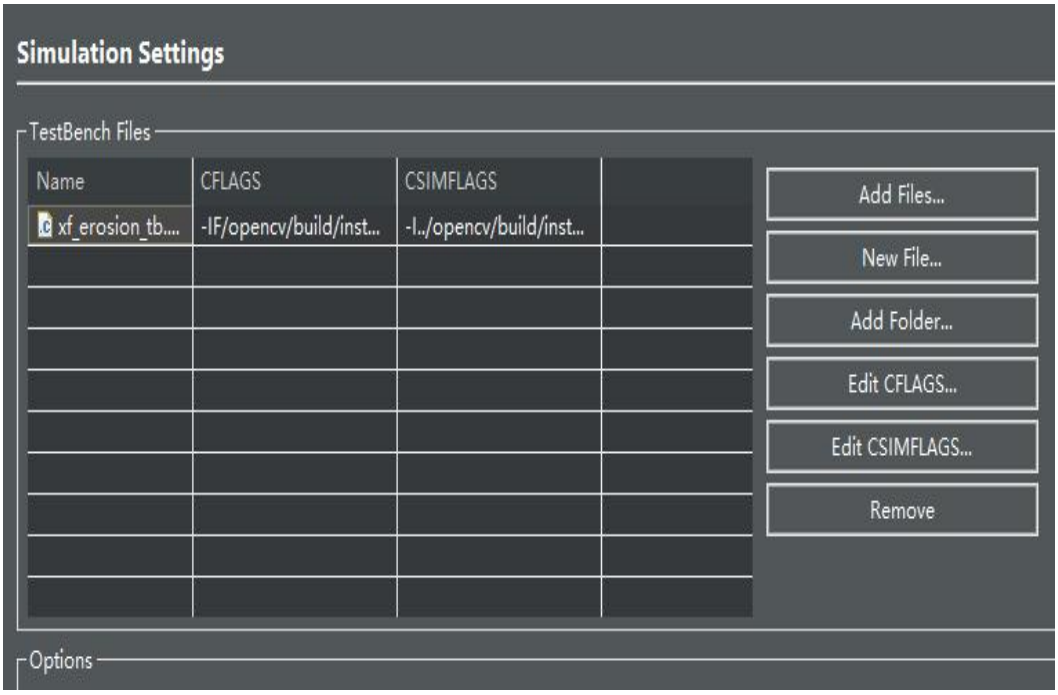


2、单击 project setting  ，选择 synthesis,选择 edit_cflags 输入 -I <get_path> -std=c++0x

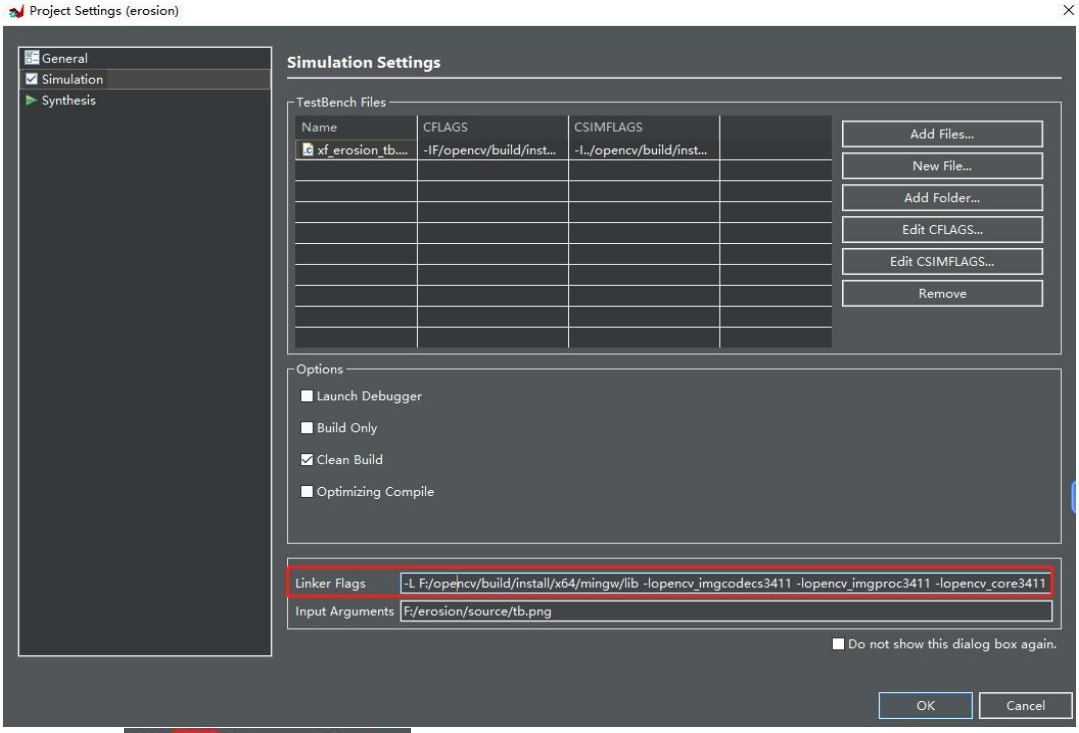


get_path 中输入电脑中安装的赛灵思提供的 opencv 的库函数路径

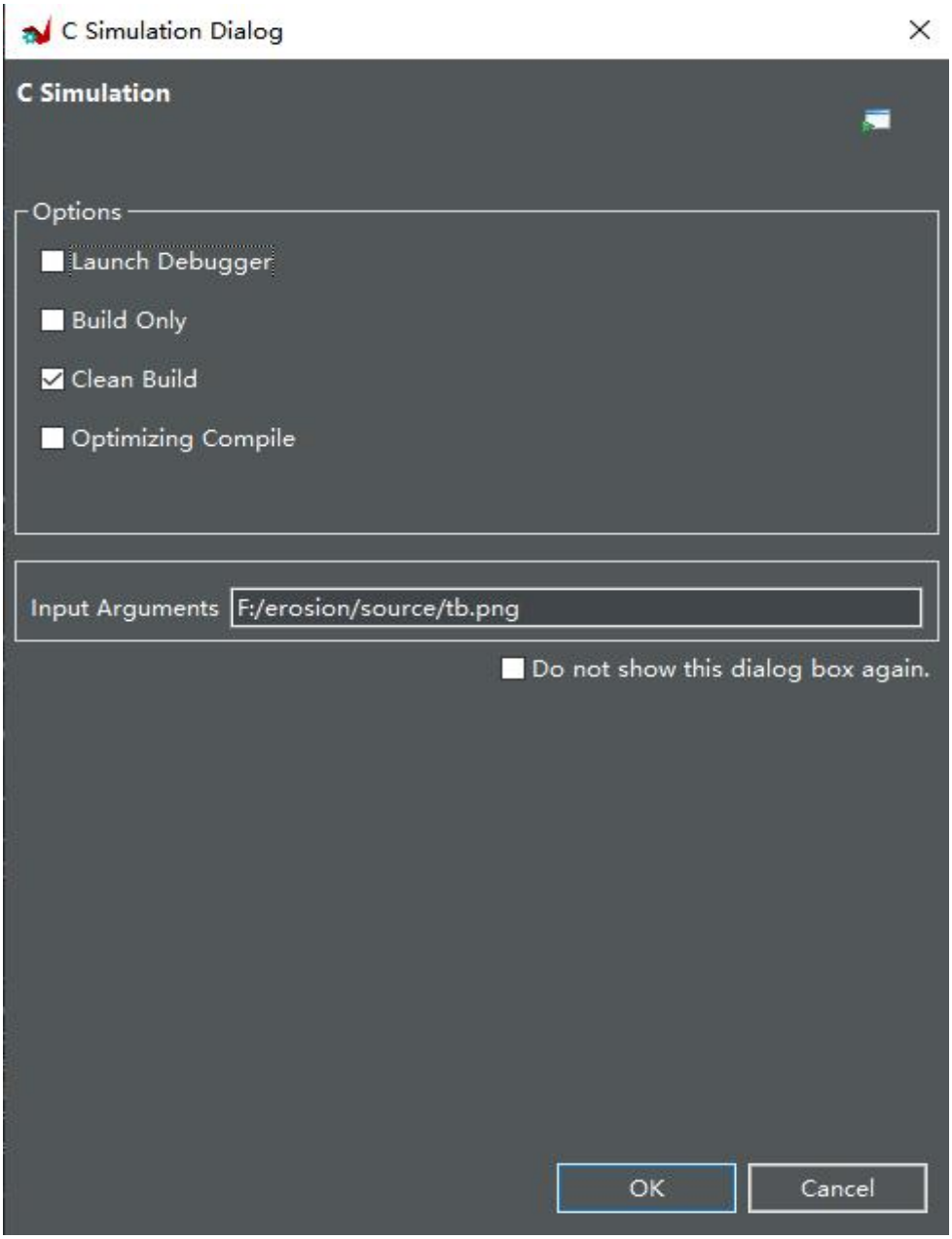
3、选择 simulation,选择 edit_csimflags 输入-I <get_path_opencv> -I <get_path> -std=c++0x,get_path_opencv 中输入从 opencv 官方下载的 opencv 库函数安装路径注意是 install 文件夹内的 include 文件夹路径，get_path 中输入安装的赛灵思提供的 opencv 路径(赛灵思提供的 opencv 库函数属于可综合，opencv 提供的函数只能用于仿真，两者差别较大)



在 Linker flags 内输入如下代码”-L F:/opencv/build/install/x64/mingw/lib -lopencv_imgcodecs3411 -lopencv_imgproc3411 -lopencv_core3411 -lopencv_highgui3411 -lopencv_flann3411 -lopencv_features2d3411”（3411 指的是 opencv 版本，如果安装了其他版本需要做对应修改）

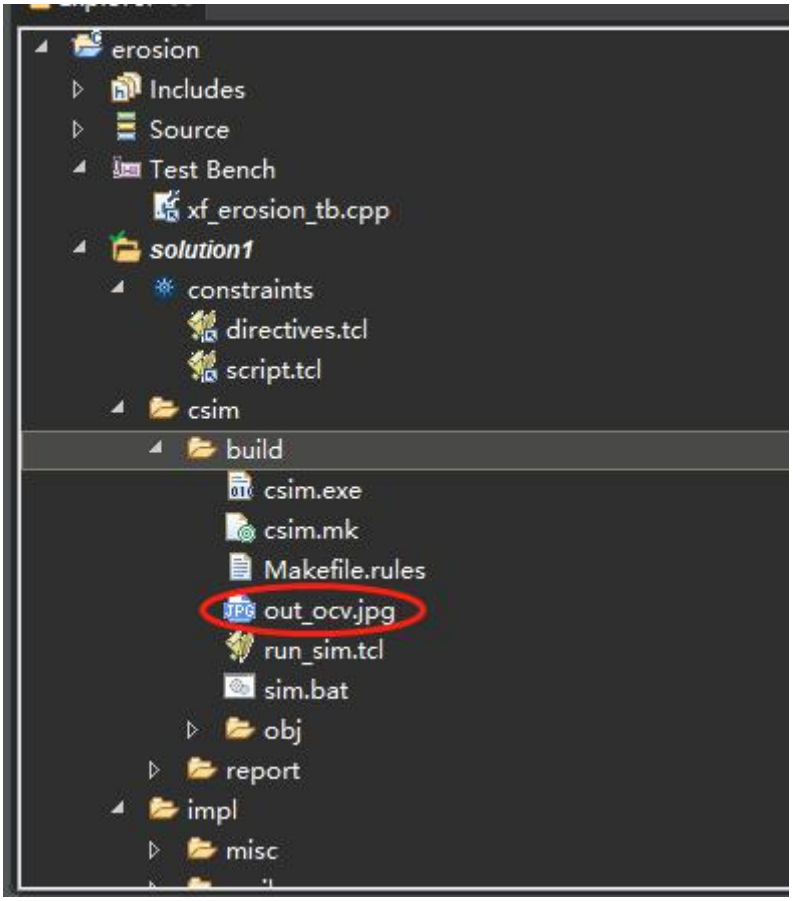


4、单击图标  在 input arguments 中输入仿真图片的路径

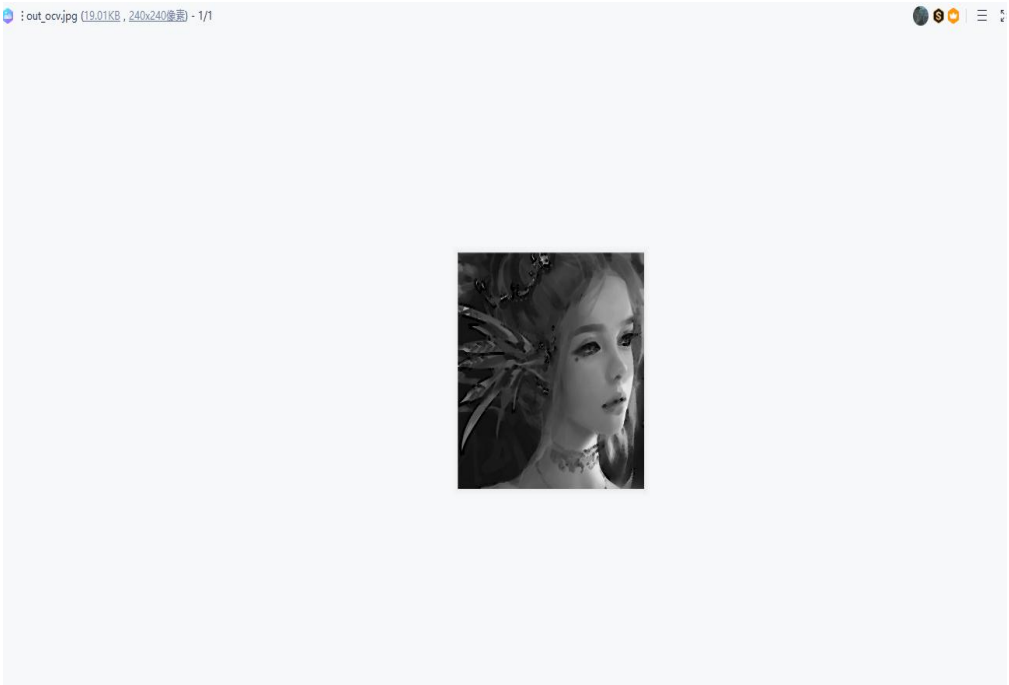


注意图片分辨率要和代码中设定的图片分辨率相同

5、通过仿真之后双击打开左侧的 out_ocv.jpg 图片


erosion
Includes
Source
Test Bench
xf_erosion_tb.cpp
solution1
constraints
directives.tcl
script.tcl
csim
build
csim.exe
csim.mk
Makefile.rules
out_ocv.jpg
run_sim.tcl
sim.bat
obj
report
impl
misc

6、观察运行效果


out_ocv.jpg (19.01KB , 240x240像素) - 1/1

13.3 备注

我们提供了一个名字叫 erosion 的工程作为仿真实例，下载后将其中路径改为自己安装的路径，图片路径改为自己下载的图片路径，之后选择 c 仿真就能看到运行之后的效果，如果运行出错检查一下路径是否配错，以及 opencv 是否正确安装。